

Prirodno-matematički fakultet

Univerzitet u Kragujevcu



Spotify and Youtube - istraživanje popularnosti pesama

Predmet:

Uvod u nauku o podacima

Predmetni profesor:

Branko Arsić

Članovi tima:

Katarina Dimitrijević 44/2022

Sandra Miladinović 82/2023

Sadržaj

Uvod	5
Opis problema i motivacija	5
Izvor podataka	5
Pregled podataka	5
Eksplorativna analiza podataka (EDA)	12
Univarijantna analiza	13
Stream	14
Views	16
Najpopularnije pesme po Stream	18
Audio karakteristike	19
Reakcije publike na YouTube-u	23
Kategorijske promenljive (Album_type, Licensed, Official_video)	24
Izvođač i kanal	25
Bivarijantna analiza - Stream	30
Artist	31
Numeričke promenljive	33
Album_type	34
Licensed	36
Official_video	39
Zaključak bivarijantne analize nad kolonom Stream	41
Bivarijantna analiza - Views	41
Artist	41
Channel	43
Numeričke promenljive	44
Album_type	47
Licensed	49
Official_video	52
Zaključak - kategorijske promenljive	54
Views naspram Stream	54
Ukupan zaključak bivarijantne analize nad kolonom Views	56
Izbor ciljne promenljive - Stream ili Views	56

Čišćenje podataka	57
Duplikati.....	57
Anomalije	58
Likes, Views, Comments, Streams	58
Loudness	59
Duration_ms.....	60
Audio karakteristike	62
Nedostajuće vrednosti	63
Audio karakteristike	65
Views.....	66
Description	67
Podela na trening i test skup	68
Likes i Comments	68
Stream.....	71
Čuvanje očišćenih podataka	74
Struktura podataka	74
Feature engineering.....	77
Artist	77
Engagement rate (Likes, Comments)	81
PCA (Views, Likes, Comments)	83
Mood kvadrant (Valence x Energy)	87
Danceability i Energy (Party index)	88
Binarna klasifikacija audio karakteristika	90
Key.....	91
Kolaboracija (feat. u nazivu pesme)	92
Licensed i Official_video	93
Multivarijantna analiza	94
Izbor prediktora i dva modela	95
Provera multikolinearnosti (VIF)	95
Osnovni modeli	96
Dijagnostika modela	98
Sinergija (interakcioni efekti)	100
Pretprocesiranje podataka	101

Modeli	103
Priprema za modele	103
Linearni modeli.....	105
Feature selection	105
Stepwise selekcija	106
Obična linearna regresija.....	108
Priprema za Lasso i Ridge regresiju.....	108
Lasso regresija.....	109
Ridge regresija	110
Poređenje linearnih modela	110
Napredni modeli mašinskog učenja	111
Priprema za napredne modele	111
Modeli sa Youtube signalom	112
Random Forest	112
XGBoost.....	116
LightGBM	119
Dodatna analiza: uticaj uključivanja pesama bez YouTube spota (Has_youtube = FALSE)	123
Random Forest.....	123
XGBoost.....	125
LightGBM	126
Uticaj bez Youtube signala.....	128
Random Forest.....	128
XGBoost.....	130
LightGBM	131
Poređenje nelinearnih modela	132
Poređenje modela.....	134
Predviđene naspram stvarnih vrednosti (LightGBM)	136
Zaključak	138
Literatura.....	139

Uvod

Predviđanje popularnosti muzike ima veliki značaj u savremenoj muzičkoj industriji. Diskografske kuće i menadžeri koriste ovakve analize kako bi procenili potencijal pesme pre ili nakon objavljivanja, radi donošenja različitih marketinških odluka. Same streaming platforme, kao što su Spotify ili YouTube, oslanjaju se na slične principe prilikom kreiranja algoritamskih preporuka i plejlista, gde razumevanje faktora koji utiču na popularnost direktno utiče na to koja će muzika biti preporučena publici. Generalno, ovakve analize doprinose i boljem razumevanju same muzičke industrije, koliko na uspeh pesme utiče njen zvuk (audio karakteristike), koliko reputacija izvođača, a koliko spoljašnji faktori poput vidljivosti na drugim platformama.

Za potrebe ovog istraživanja korišćen je R programski jezik jer nudi veliki izbor biblioteka za statističku analizu, obradu podataka i vizualizaciju.

Opis problema i motivacija

Problem predviđanja popularnosti pesama je veoma kompleksan, jer postoji mnogo faktora koji utiču na nju a ne mogu se kvantifikovati, na primer postoje šanse da je pesma viralna na TikTok-u i da se je zbog toga popularna na ostalim platformama, što je jako teško pretvoriti u podatak koji se na neki način može analizirati i koristiti. Glavni cilj ovog projekta je analiza koliko podaci koji mogu da se kvantifikuju utiču na ove pesme, kao i kreiranje modela koji mogu da predviđaju popularnost pesme na određenim platformama, u skladu sa dostupnim karakteristikama.

Izvor podataka

Za potrebe istraživanja korišćen je [Spotify and YouTube](#) dataset, preuzet sa Kaggle platforme. Sastoji se od preko 20.000 redova, pri čemu je prikupljeno po 10 top pesama za svakog izvođača u datasetu. Pored toga, sadrži 28 kolona, gde se neke kolone fokusiraju na same audio karakteristike pesme, neke na same metrike popularnosti, a neke na karakteristike Youtube videa.

Pregled podataka

Na početku ćemo učitati podatke iz csv fajla preuzetog sa kaggle-a.

```
library(tidyverse)
library(moments)
library(scales)
library(reshape2)
library(rstatix)
library(psych)
library(car)
library(randomForest)
library(xgboost)
library(lightgbm)
```

```
library(ranger)
library(glmnet)
```

```
data = read.csv("Spotify_Youtube.csv", header = T)
```

Nakon učitavanja podataka, funkcijom `str` proveravamo strukturu podataka, pri čemu možemo da primetimo da imamo 16 kolona numeričkog tipa i 12 kolona znakovnog tipa.

```
str(data)

## 'data.frame':    20718 obs. of  28 variables:
## $ X                : int  0 1 2 3 4 5 6 7 8 9 ...
## $ Artist           : chr  "Gorillaz" "Gorillaz" "Gorillaz" "Gorillaz" ...
## $ Url_spotify      : chr
## "https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ"
## "https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ"
## "https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ"
## "https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ" ...
## $ Track            : chr  "Feel Good Inc." "Rhinestone Eyes" "New Gold
## (feat. Tame Impala and Bootie Brown)" "On Melancholy Hill" ...
## $ Album            : chr  "Demon Days" "Plastic Beach" "New Gold (feat.
## Tame Impala and Bootie Brown)" "Plastic Beach" ...
## $ Album_type       : chr  "album" "album" "single" "album" ...
## $ Uri              : chr  "spotify:track:0d28khcov6AiegSCpG5TuT"
## "spotify:track:1foMv2HQwfQ2vntFf9HFeG" "spotify:track:64dLd6rVqDLtkXFYrEUHIU"
## "spotify:track:0q6LuUqGLUiCPP1cbdwFs3" ...
## $ Danceability     : num  0.818 0.676 0.695 0.689 0.663 0.76 0.716 0.726
## 0.741 0.625 ...
## $ Energy           : num  0.705 0.703 0.923 0.739 0.694 0.891 0.897 0.815
## 0.913 0.877 ...
## $ Key              : num  6 8 1 2 10 11 4 11 2 10 ...
## $ Loudness         : num  -6.68 -5.82 -3.93 -5.81 -8.63 ...
## $ Speechiness      : num  0.177 0.0302 0.0522 0.026 0.171 0.0372 0.0629
## 0.0313 0.0465 0.162 ...
## $ Acousticness     : num  8.36e-03 8.69e-02 4.25e-02 1.51e-05 2.53e-02
## 2.29e-02 1.20e-02 7.99e-03 3.43e-03 3.15e-02 ...
## $ Instrumentalness : num  0.00233 0.000687 0.0469 0.509 0 0.0869 0.262
## 0.081 0.103 0.0811 ...
## $ Liveness         : num  0.613 0.0463 0.116 0.064 0.0698 0.298 0.325
## 0.112 0.325 0.672 ...
## $ Valence          : num  0.772 0.852 0.551 0.578 0.525 0.966 0.358 0.462
## 0.643 0.865 ...
## $ Tempo            : num  138.6 92.8 108 120.4 168 ...
## $ Duration_ms      : num  222640 200173 215150 233867 340920 ...
## $ Url_youtube      : chr  "https://www.youtube.com/watch?v=HyHNuVaZJ-k"
## "https://www.youtube.com/watch?v=yYDmaexVHic"
## "https://www.youtube.com/watch?v=qJa-VFwPpYA"
## "https://www.youtube.com/watch?v=04mfKJWDSzI" ...
## $ Title            : chr  "Gorillaz - Feel Good Inc. (Official Video)"
## "Gorillaz - Rhinestone Eyes [Storyboard Film] (Official Music Video)"
```

```

"Gorillaz - New Gold ft. Tame Impala & Bootie Brown (Official Visualiser)"
"Gorillaz - On Melancholy Hill (Official Video)" ...
## $ Channel      : chr  "Gorillaz" "Gorillaz" "Gorillaz" "Gorillaz" ...
## $ Views       : num  6.94e+08 7.20e+07 8.44e+06 2.12e+08 6.18e+08 ...
## $ Likes       : num  6220896 1079128 282142 1788577 6197318 ...
## $ Comments    : num  169907 31003 7399 55229 155930 ...
## $ Description  : chr  "Official HD Video for Gorillaz' fantastic track
Feel Good Inc.\n\nFollow Gorillaz online:\nhhttp://gorillaz.com\"|
__truncated__ "The official video for Gorillaz - Rhinestone
Eyes\n\nRhinestone Eyes is taken from the 2010 album Plastic Beach"|
__truncated__ "Gorillaz - New Gold ft. Tame Impala & Bootie Brown (Official
Visualiser)\n\nPre-order Cracker Island album: htt"| __truncated__ "Follow
Gorillaz online:\nhhttp://gorillaz.com
\nhttp://facebook.com/Gorillaz\nhttp://twitter.com/GorillazBand\nh"|
__truncated__ ...
## $ Licensed     : chr  "True" "True" "True" "True" ...
## $ official_video : chr  "True" "True" "True" "True" ...
## $ Stream       : num  1.04e+09 3.10e+08 6.31e+07 4.35e+08 6.17e+08 ...

```

Spisak kolona koje skup sadrži i kratak opis:

1. X – Jedinstveni identifikator.
2. Artist – Izvođač pesme.
3. Url_spotify – Link ka stranici izvođača na Spotify-u.
4. Track – Naziv pesme.
5. Album – Naziv albuma na kome se pesma nalazi.
6. Album_type – Tip izdanja: album, single ili compilation.
7. Uri – Jedinstveni Spotify identifikator pesme.
8. Danceability – Mera (0–1) koliko je pesma pogodna za ples, na osnovu tempa, ritmičke stabilnosti i jačine bita.
9. Energy – Mera (0–1) intenziteta i aktivnosti pesme; brze, glasne pesme imaju više vrednosti.
10. Key – Tonalitet pesme, kodiran brojevima 0–11, -1 ako tonalitet nije detektovan.
11. Loudness – Prosečna jačina zvuka pesme u decibelima (dB), tipično u opsegu od -60 do 0.
12. Speechiness – Mera (0–1) prisustva govora u snimku; visoke vrednosti ukazuju na govorni sadržaj, a ne pevanu muziku.

13. Acousticness – Mera pouzdanosti (0–1) da je pesma akustična, bez elektronske produkcije.
14. Instrumentalness – Mera (0–1) verovatnoće da pesma nema vokal.
15. Liveness – Mera (0–1) prisustva publike u snimku; visoke vrednosti ukazuju da je pesma snimljena uživo.
16. Valence – Mera (0–1) muzičke “pozitivnosti”; više vrednosti zvuče vedrije, niže tmurnije.
17. Tempo – Procenjeni tempo pesme u otkucajima u minuti (BPM).
18. Duration_ms – Trajanje pesme u milisekundama.
19. Url_youtube – Link ka YouTube video spotu pesme.
20. Title – Naziv YouTube videa.
21. Channel – Naziv YouTube kanala na kome je video objavljen.
22. Views – Broj pregleda YouTube videa.
23. Likes – Broj lajkova na YouTube videu.
24. Comments – Broj komentara na YouTube videu.
25. Description – Opis YouTube videa koji je uneo autor.
26. Licensed – Da li je video registrovan kao licencirani sadržaj.
27. Official_video – Da li je video zvaničan spot pesme, za razliku od lyrics videa, izvedbe uživo ili fan-uploada.
28. Stream – Broj stream-ova pesme na Spotify-u.

Pored toga, pogledaćemo i prvih nekoliko redova, kako bismo imali bolji u podatke sa kojima radimo.

```
head(data)
```

```
##   X   Artist                                     Url_spotify
## 1 0 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
## 2 1 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
## 3 2 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
## 4 3 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
## 5 4 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
## 6 5 Gorillaz https://open.spotify.com/artist/3AA28KZvwAUcZuOKwyblJQ
##   Track
## 1 Feel Good Inc.
## 2 Rhinestone Eyes
```



```

## 3 New Gold (feat. Tame Impala and Bootie Brown)
## 4 On Melancholy Hill
## 5 Clint Eastwood
## 6 DARE
## Album Album_type
## 1 Demon Days album
## 2 Plastic Beach album
## 3 New Gold (feat. Tame Impala and Bootie Brown) single
## 4 Plastic Beach album
## 5 Gorillaz album
## 6 Demon Days album
## Uri Danceability Energy Key Loudness
## 1 spotify:track:0d28khcov6AiegSCpG5TuT 0.818 0.705 6 -6.679
## 2 spotify:track:1foMv2HQwfQ2vntFf9HFeG 0.676 0.703 8 -5.815
## 3 spotify:track:64dLd6rVqDLtkXFYrEUHIU 0.695 0.923 1 -3.930
## 4 spotify:track:0q6LuUqGLUiCPP1cbdwFs3 0.689 0.739 2 -5.810
## 5 spotify:track:7yMiX7n9SBvadzox8T5jzT 0.663 0.694 10 -8.627
## 6 spotify:track:4Hff1IjRbLGeLgFgxvHflk 0.760 0.891 11 -5.852
## Speechiness Acousticness Instrumentalness Liveness Valence Tempo
## 1 0.1770 8.36e-03 0.002330 0.6130 0.772 138.559
## 2 0.0302 8.69e-02 0.000687 0.0463 0.852 92.761
## 3 0.0522 4.25e-02 0.046900 0.1160 0.551 108.014
## 4 0.0260 1.51e-05 0.509000 0.0640 0.578 120.423
## 5 0.1710 2.53e-02 0.000000 0.0698 0.525 167.953
## 6 0.0372 2.29e-02 0.086900 0.2980 0.966 120.264
## Duration_ms Url_youtube
## 1 222640 https://www.youtube.com/watch?v=HyHNuVaZJ-k
## 2 200173 https://www.youtube.com/watch?v=yYDmaexVHic
## 3 215150 https://www.youtube.com/watch?v=qJa-VFwPpYA
## 4 233867 https://www.youtube.com/watch?v=04mfKJWDSzI
## 5 340920 https://www.youtube.com/watch?v=1V_xRb0x9aw
## 6 245000 https://www.youtube.com/watch?v=uAOR6ib95kQ
## Title
## 1 Gorillaz - Feel Good Inc. (Official Video)
## 2 Gorillaz - Rhinestone Eyes [Storyboard Film] (Official Music Video)
## 3 Gorillaz - New Gold ft. Tame Impala & Bootie Brown (Official Visualiser)
## 4 Gorillaz - On Melancholy Hill (Official Video)
## 5 Gorillaz - Clint Eastwood (Official Video)
## 6 Gorillaz - DARE (Official Video)
## Channel Views Likes Comments
## 1 Gorillaz 693555221 6220896 169907
## 2 Gorillaz 72011645 1079128 31003
## 3 Gorillaz 8435055 282142 7399
## 4 Gorillaz 211754952 1788577 55229
## 5 Gorillaz 618480958 6197318 155930
## 6 Gorillaz 259021161 1844658 72008
## Description
## 1 Official HD Video for Gorillaz' fantastic track Feel Good
Inc.\n\nFollow Gorillaz
online:\nhhttp://gorillaz.com\nhhttp://facebook.com/Gorillaz\nhhttp://twitter.co

```

m/GorillazBand\nhttp://instagram/Gorillaz\n\nFor more information on Gorillaz don't forget to check out the official website at <http://www.gorillaz.com>

2 The official video for Gorillaz - Rhinestone Eyes\n\nRhinestone Eyes is taken from the 2010 album Plastic Beach including the singles Rhinestone Eyes, Stylo, Superfast Jellyfish and On Melancholy Hill.\n\nFollow Gorillaz online:\n<https://instagram.com/gorillaz> \n<https://tiktok.com/@gorillaz> \n<https://twitter.com/gorillaz> \n<https://facebook.com/gorillaz> \n<https://gorillaz.com>\n\n#Gorillaz #RhinestoneEyes #PlasticBeach

3 Gorillaz - New Gold ft. Tame Impala & Bootie Brown (Official Visualiser)\n\nPre-order Cracker Island album: <https://gorillaz.com> \nListen to New Gold: <https://gorill.az/newgold>\nJoin The Last Cult: <https://thelastcult.org>\nListen to Gorillaz: <https://gorill.az/listen>\nShop Gorillaz: <https://store.gorillaz.com/>\nTour gorillaz.com/tour\n\nFollow Gorillaz: \n<https://instagram.com/gorillaz> \n<https://tiktok.com/@gorillaz> \n<https://twitter.com/gorillaz> \n<https://facebook.com/gorillaz> \n<https://gorillaz.com>\n\nDirector: Jamie Hewlett\nCo-Direction: Swear Studio, Steve Gallagher\nExec Producers: Jamie Hewlett & Damon Albarn\nProducer: Alexa Pearson\nVideo Design: Catherine Woodhouse\nVideo Design: SHOP\n\n#newgold #crackerisland #gorillaz #newmusic #newalbum

4 Follow Gorillaz online:\n<http://gorillaz.com> \n<http://facebook.com/Gorillaz> \n<http://twitter.com/GorillazBand> \n<https://instagram.com/gorillaz>\n\nMusic video by Gorillaz performing On Melancholy Hill. (P) 2010 The copyright in this audiovisual recording is owned by Parlophone Records

5 The official music video for Gorillaz - Clint Eastwood\n\nTaken from Gorillaz debut Studio Album 'Gorillaz' released in 2001, which features the singles Clint Eastwood, 19-2000, Rock The House & Tomorrow Comes Today.\n\nPre-order Cracker Island album: <https://gorillaz.com> \n\nSubscribe to the Gorillaz channel for all the best and latest official music videos, behind the scenes and live performances here - <https://bit.ly/subscribeGORILLAZ>\n\nListen to more from the album 'Gorillaz' here\nhttps://www.youtube.com/playlist?list=OLAK5uy_nCZqWcwZb_UQ01mFgYUHiyWAlYKVBP_uA\n\nSee more official videos from Gorillaz here: \n<https://www.youtube.com/playlist?list=PLtKoi37ubAW01MrQNue2ekVg5k9jr1Llyo>\n\nFollow Gorillaz:\n<https://instagram.com/gorillaz> \n<https://tiktok.com/@gorillaz> \n<https://twitter.com/gorillaz> \n<https://facebook.com/gorillaz> \n<https://gorillaz.com>\n\nLYRICS:\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nThe future is coming on\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nThe future is coming on\nIt's coming on, it's coming on\nIt's coming on, it's coming on\n\nFinally someone let me out of my cage\nNow time for me is nothin', 'cause I'm countin' no age\nNow I couldn't be there, now you shouldn't be scared\nI'm good at repairs, and I'm under each snare\n\nIntangible (ah y'all), bet you didn't think\nSo I command you to, panoramic view (you)\nLook, I'll make it all manageable\nPick and choose, sit and lose all you different crews\nChicks and dudes, who you think is really kicking tunes?\n\nPicture you getting down in a picture tube\nLike you lit the fuse, you think it's fictional? Mystical? Maybe\nSpiritual hero who appears on you to clear your view\nWhen you're too

crazy\n\nLifeless to those the definition for what life is\nPriceless to you because I put you on the hype shift\nDid you like it? Gut smokin' righteous with one toke\nYou're psychic among knows possess you with one go\n\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nThe future is coming on\n\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nThe future (that's right) is coming on\nIt's coming on, it's coming on\nIt's coming on, it's coming on\n\nThe essence, the basics without did you make it?\nAllow me to make this childlike in nature\nRhythm, you have it or you don't\nThat's a fallacy, I'm in them\nEvery sprouting tree, every child of peace\nEvery cloud and sea, you see with your eyes\n\nI see destruction and demise\nCorruption in the skies (that's right)\nFrom this fucking enterprise, now I'm sucked into your lies\nThrough Russel, not his muscles\nBut percussion he provides\n\nFor me as a guide, y'all can see me now\n'Cause you don't see with your eye\nYou perceive with your mind, that's the inner (fuck 'em)\nSo I'mma stick around with Russ and be a mentor\n\nBust a few rhymes so motherfuckers remember\nWhere the thought is, I brought all this\nSo you can survive when law is lawless (right here)\nFeeling sensations that you thought was dead\nNo squealing and remember that it's all in your head\n\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nThe future is coming on\n\nI ain't happy, I'm feeling glad\nI got sunshine in a bag\nI'm useless but not for long\nMy future is coming on\nIt's coming on, it's coming on\nIt's coming on, it's coming on\n\nMy future\nIt's coming on, it's coming on, it's coming on\nIt's coming on, it's coming on, my future\nIt's coming on, it's coming on, it's coming on\n\nMy future\nIt's coming on, it's coming on, it's coming on\nMy future\nIt's coming on, it's coming on, it's coming on\nMy future\n\nAbout Gorillaz:\nVirtual band Gorillaz is singer 2D, bassist Murdoc Niccals, guitarist Noodle and drummer Russel Hobbs. Created by Damon Albarn and Jamie Hewlett, their acclaimed eponymous debut album was released in 2001. The BRIT and Grammy Award winning band's subsequent albums are Demon Days (2005), Plastic Beach (2010), The Fall (2011), Humanz (2017) and The Now Now (2018). A truly global phenomenon, Gorillaz have achieved success in entirely groundbreaking ways, touring the world from San Diego to Syria, winning numerous awards including the coveted Jim Henson Creativity Honor.\n\n* The band are recognised by The Guinness Book Of World Records as the planet's Most Successful Virtual Act.\n\n#Gorillaz #ClintEastwood

6 Follow Gorillaz online:\n<http://gorillaz.com>
<http://facebook.com/Gorillaz>\n<http://twitter.com/GorillazBand>\n<https://instagram.com/gorillaz>\n\nStream Gorillaz on Spotify:
<http://bit.ly/1kvIyPR>\n\nMusic video by Gorillaz performing DARE. (P) 2005
The copyright in this sound recording is owned by Parlophone Records

##	Licensed official_video	Stream
## 1	True	True 1040234854
## 2	True	True 310083733
## 3	True	True 63063467
## 4	True	True 434663559
## 5	True	True 617259738
## 6	True	True 323850327

Prilikom rada sa ovim podacima, odlučili smo da ne odaberemo odmah jednu ciljnu promenljivu, kako u ovom skupu podataka imamo 2 promenljive koje bi bile zanimljive za analizu onog što nas zanima, odnosno popularnosti pesama, a to su Views i Stream. Ideja je da odluku o ciljnoj promenljivoj koju predviđamo donesemo tek nakon detaljnijih analiza samih podataka.

Da bismo lakše radili sa podacima, i da bi sami nazivi kolona bili usklađeni, postavitićemo nazive kolona tako svaki ima početno veliko slovo.

```
names(data) = paste0(
  toupper(substr(names(data), 1, 1)),
  substr(names(data), 2, nchar(names(data)))
)

names(data)
```

## [1] "X"	"Artist"	"Url_spotify"	"Track"
## [5] "Album"	"Album_type"	"Uri"	
"Danceability"			
## [9] "Energy"	"Key"	"Loudness"	
"Speechiness"			
## [13] "Acousticness"	"Instrumentalness"	"Liveness"	"Valence"
## [17] "Tempo"	"Duration_ms"	"Url_youtube"	"Title"
## [21] "Channel"	"Views"	"Likes"	"Comments"
## [25] "Description"	"Licensed"	"Official_video"	"Stream"

Nakon ove transformacije sve kolone imaju konzistentan naziv.

Eksplorativna analiza podataka (EDA)

Eksplorativna analiza podataka (Exploratory Data Analysis - EDA) predstavlja korak u analizi podataka u okviru koga istražujemo i vizualizujemo podatke, kako bismo saznali njihovu strukturu, uočili šablone u podacima i proverili međusobne veze između promenljivih kako bismo kasnije kreirali adekvatne modele.

Jedna od odluka koju moramo doneti zarad kasnijeg kreiranja modela jeste koju bi promenljivu naši modeli trebalo da prediktuju, jer kao što smo već napomenuli ovaj skup ima 2 kolone koje imaju potencijal da budu ciljne, a to su kolone Stream i Views. Podaci kolone Stream vezani su za broj stream-ova na platformi Spotify, dok podaci Views oslikavaju broj pregleda na spotu date pesme. Kako bismo adekvatno odredili ciljnu promenljivu, konkretno bivarijatnu analizu nad obe kolone.

U nastavku se nalazi pomoćna funkcija koja se koristi u univarijatnoj i bivarijatnoj analizi. Služi da proverava normalnost neke promenljive unutar svake kategorije jedne kategorijske promenljive, koristeći Shapiro-Wilk test sa bootstrap pristupom, za višestruko uzorkovanje, jer je Shapiro-Wilk previše osetljiv na velike uzorke da bi se pouzdano primenio direktno na ceo skup.

```

shapiro_boot = function(data, group_col, value_col) {
  categories = unique(data[[group_col]])
  categories = categories[!is.na(categories)]

  lapply(categories, function(cat) {
    values = na.omit(data[[value_col]][data[[group_col]] == cat])

    if (length(values) > 3) {
      p_values = replicate(500, shapiro.test(sample(values, min(3000,
length(values))))$p.value)
      p_mean = mean(p_values)

      if (p_mean > 0.05) {
        sprintf("Varijabla %s po %s kategoriji ima normalnu raspodelu.",
value_col, as.character(cat))
      } else {
        sprintf("Varijabla %s po %s kategoriji nema normalnu raspodelu.",
value_col, as.character(cat))
      }
    } else {
      sprintf("Varijabla %s po %s kategoriji ima manje od 3 observacije.",
value_col, as.character(cat))
    }
  })
}

```

Univarijantna analiza

Da bismo adekvatno analizirali uticaj različitih promenljivih na potencijalne ciljne promenljive, trebalo bi odraditi i pojedinačnu odnosno univarijantnu analizu kolona kako bismo mogli da uočimo potencijalne obrasce u podacima, anomalije i ekstremne vrednosti. Promenljive koje ćemo analizirati u okviru univarijantne analize su:

- **Potencijalne ciljne promenljive** - Stream (broj stream-ova na Spotify-u) i Views (broj pregleda na Youtube spotu pesme)
- **Audio karakteristike pesme** - kolone koje opisuju samu pesmu i njene zvučne karakteristike u obliku brojnih vrednosti - Danceability, Energy, Loudness, Speechiness, Acousticness, Instrumentalness, Liveness, Valence, Tempo, Duration_ms
- **Reakcije publike na YouTube-u** - kolone koje prikazuju samu reakciju publike na pesmu na YouTube-u - Likes, Comments
- **Kategorije sadržaja** - podaci o tome da li je sadržaj licenciran i objavljen na zvaničnom YouTube kanalu - Licensed, Official_video, da li je pesma objavljena u okviru albuma ili singla (Album)

- **Izvođač** - podaci o tome ko je izvođač pesme (Artist), kao i na kom Youtube kanalu je objavljena određena pesma (Channel), kao i o tome na kom albumu je objavljena pesma (Album)

Stream

Stream je potencijalna ciljna promenljiva, i prvo ćemo proveriti njenu raspodelu.

```
summary(data$Stream)

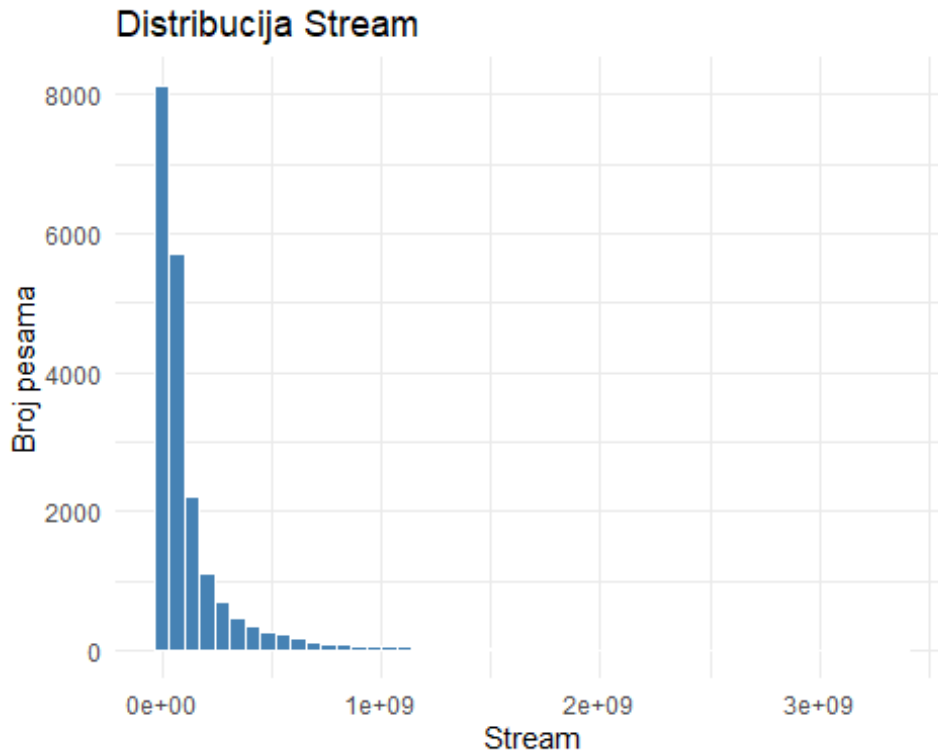
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.     NA's 
## 6.574e+03 1.767e+07 4.968e+07 1.359e+08 1.384e+08 3.387e+09    576 

sd(data$Stream, na.rm = TRUE)

## [1] 244132078
```

Iz ove analize promenljive možemo videti da se u okviru kolone stream nalazi preko 500 nedostajućih vrednosti, kao i da je standardna devijacija izuzetno visoka, pa već možemo da naslutimo da će ova kolona imati neki oblik asimetrije.

```
ggplot(data, aes(x = Stream)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  labs(title = "Distribucija Stream", x = "Stream", y = "Broj pesama") +
  theme_minimal()
```



Analizom histograma može se uočiti izražena desna asimetrija u okviru promenljive Stream, što znači da velika količina pesama ima relativno mali ili srednji broj stream-ova, dok određeni broj pesama dostiže vrednosti od preko milijardu stream-ova. Iako je ova pojava veoma izražena u okviru dijagrama, da bismo bili sigurni u asimetričnu distribuciju promenljive, pokušaćemo i numerički da potvrdimo desnu asimetričnost.

Da bismo sa sigurnošću potvrdili asimetričnost, koristimo meru asimetrije (skew) i meru spljoštenosti ili ekcesa (kurtosis). **Mera asimetrije** nam ukazuje u kojoj meri je neki raspored asimetričan, pri čemu ako je:

- **skew = 0** -> raspored je simetričan
- **skew > 0** -> raspored je asimetričan udesno (pozitivna asimetrija)
- **skew < 0** -> raspored je asimetričan ulevo (negativna asimetrija)

Ukoliko je apsolutna vrednost mere asimetrije veća od 0.5 zaključuje se jaka asimetrija.

Koeficijent spljoštenosti nam pokazuje u kojoj meri je neka raspodela spljoštena u odnosu na normalnu, pri čemu važi:

- **kurtosis = 0** -> raspored je normalno spljošten
- **kurtosis > 0** -> raspored je više izdužen u odnosu na normalni raspored
- **kurtosis < 0** -> raspored je više spljošten u odnosu na normalni raspored

Izvor: Ekonomski fakultet Novi Sad - Deskriptivna statistika ¹

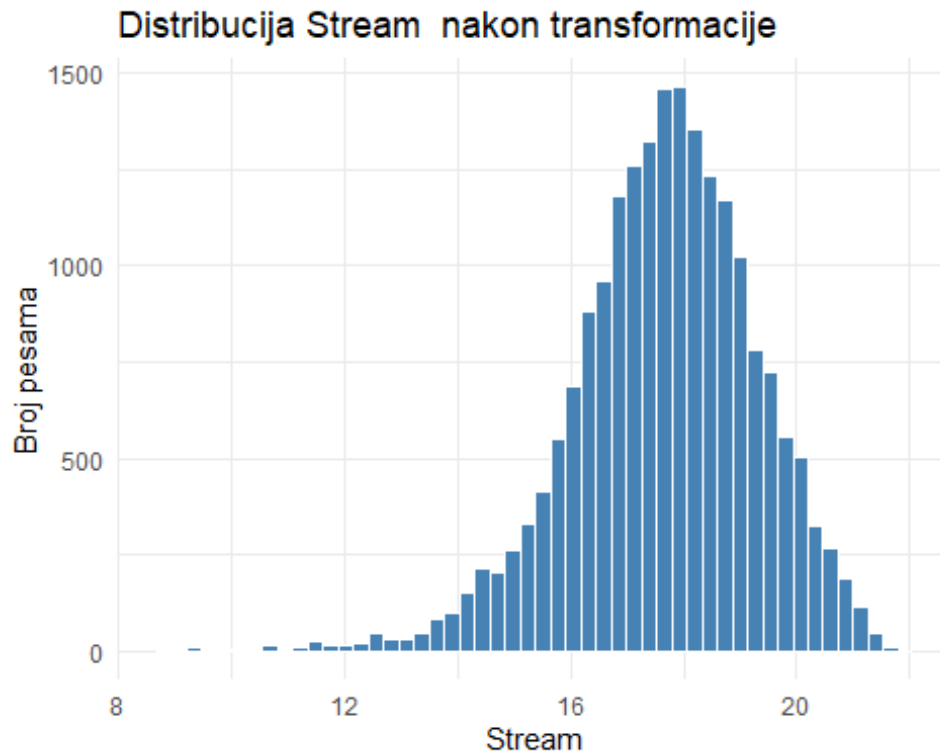
```
data %>% summarise(
  mean = mean(Stream, na.rm = TRUE),
  median = median(Stream, na.rm = TRUE),
  std = sd(Stream, na.rm = TRUE),
  skew = skewness(Stream, na.rm = TRUE),
  kurtosis = kurtosis(Stream, na.rm = TRUE)
)

##           mean    median      std      skew kurtosis
## 1 135942190 49682982 244132078 4.112336 26.15088
```

Na osnovu dobijenih vrednosti iz funkcije, pri čemu je skew znatno veći od 0.5 i kurtosis je veći od 20, možemo zaključiti da imamo prisustvo jake desne asimetrije. Da bismo mogli da izvršimo stabilnu analizu ove kolone, neophodno je da izvršimo logaritamsku transformaciju, koja će ublažiti uticaj ekstremnih vrednosti, kako one u ovom slučaju ne predstavljaju grešku u podacima, već jednostavno realan slučaj prisustva pesama koje su mnogo popularnije od većine.

¹ <https://www.ef.uns.ac.rs/predmeti/oas/statistika/2020-01-10-des-deskriptivna-statistika.pdf>

```
ggplot(data, aes(x = log(Stream))) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  labs(title = "Distribucija Stream nakon transformacije", x = "Stream", y =
"Broj pesama") + theme_minimal()
```



Nakon izvršene logaritamske transformacije, distribucija stream-ova postala je više simetrična i bliža je normalnoj raspodeli, što nam omogućava stabilniju analizu odnosa Stream-a i ostalih promenljivih.

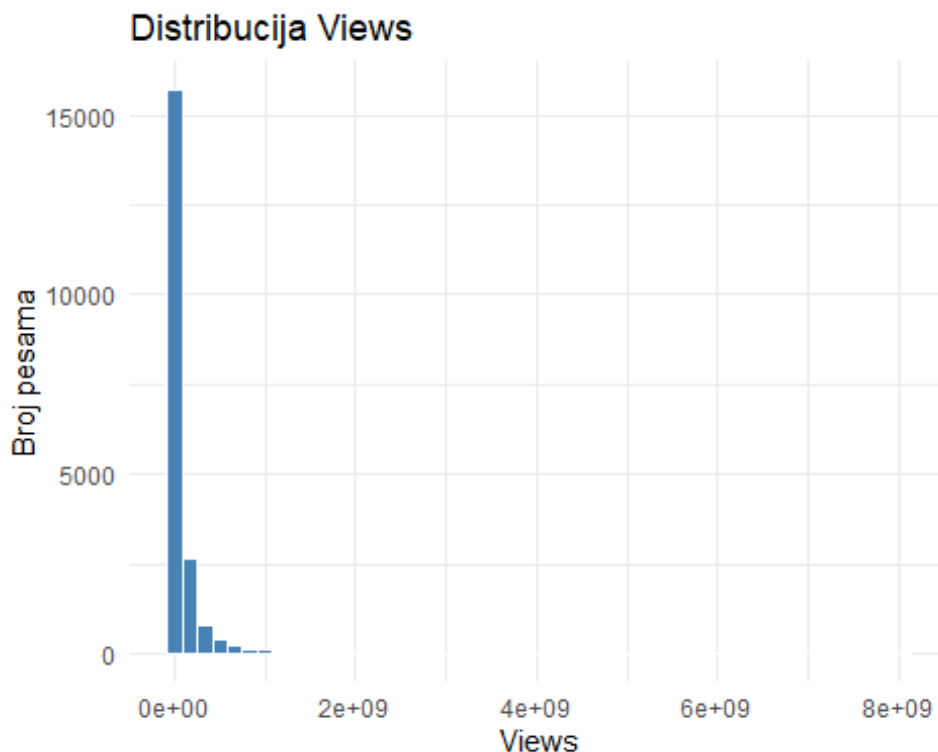
Views

Views je takođe potencijalna ciljna promenljiva, i prvo ćemo proveriti njenu raspodelu, jer i kod nje postoji veliki rizik od desne asimetričnosti.

```
summary(data$Views)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.     NA's
## 0.000e+00 1.826e+06 1.450e+07 9.394e+07 7.040e+07 8.080e+09    470
```

```
ggplot(data, aes(x = Views)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  labs(title = "Distribucija Views", x = "Views", y = "Broj pesama") +
  theme_minimal()
```

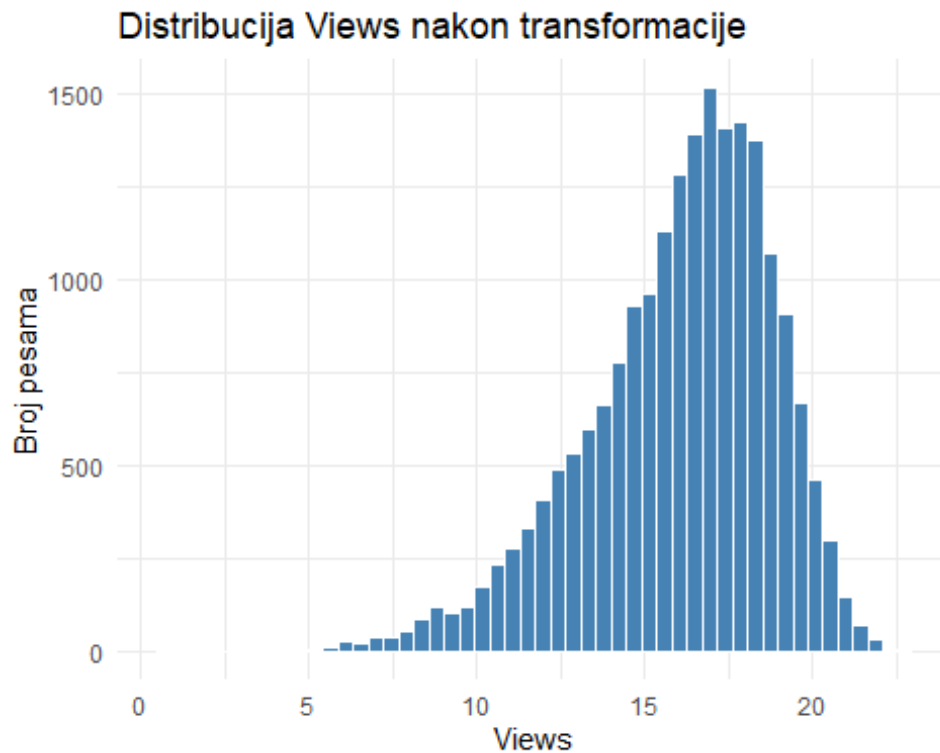
Na osnovu histograma možemo zaključiti da i kod kolone Views postoji ista problematika kao i kod Stream, odnosno desna asimetrija. Da bismo ovu kolonu učinili lakšom za analizu i upoređivanje sa drugim kolonama, korist ćemo istu metodologiju za rešavanje, odnosno logaritamsku transformaciju. Kako bismo sa sigurnošću potvrdili asimetriju, ponavljamo test.

```
data %>%
  summarise(
    mean = mean(Views, na.rm = TRUE),
    median = median(Views, na.rm = TRUE),
    std = sd(Views, na.rm = TRUE),
    skew = skewness(Views, na.rm = TRUE),
    kurtosis = kurtosis(Views, na.rm = TRUE)
  )

##           mean    median      std      skew kurtosis
## 1 93937821 14501095 274644322 9.237149 151.7561
```

Kao što možemo videti, ovde je situacija identična kao kod Stream, jako visoka vrednost skew i pozitivna vrednost kurtosis-a ukazuje nam na jaku desnu asimetričnost.

```
ggplot(data, aes(x = log(Views))) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  labs(title = "Distribucija Views nakon transformacije", x = "Views", y =
"Broj pesama") +
  theme_minimal()
```



Nakon logaritamske transformacije, kao i kod Stream promenljive, distribucija je bliža normalnoj.

Najpopularnije pesme po Stream

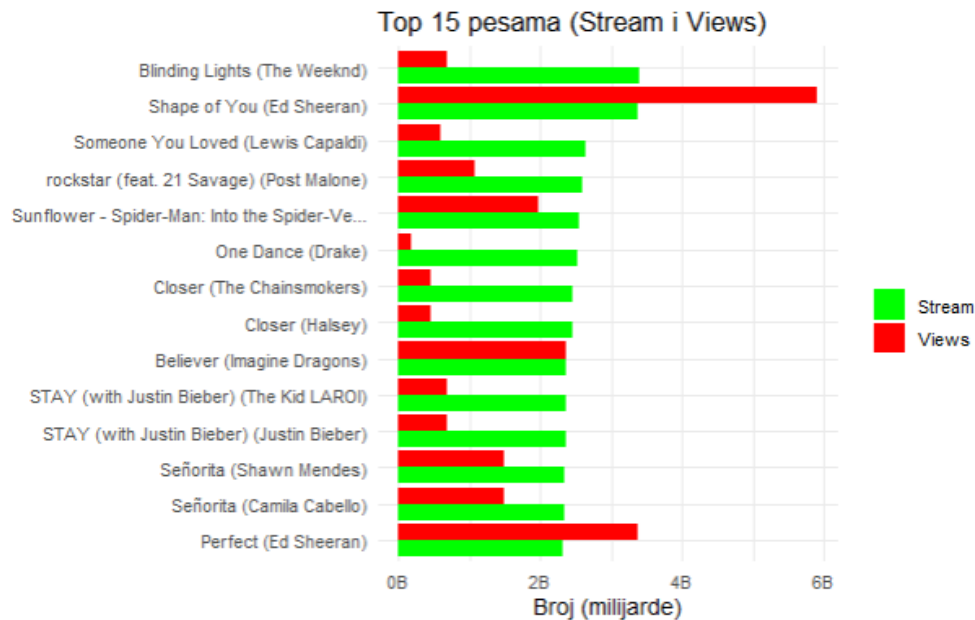
Korisno je pogledati koje su konkretno pesme na vrhu liste po Stream-u, kako bismo sagledali neke osnovne osobine najslušanijih pesama, kao i kako ove pesme stoje sa vrednostima za Views, što ćemo izvršiti pregledom top 15 pesama na listi.

```
top_songs = data %>% filter(!is.na(Stream)) %>% arrange(desc(Stream)) %>%
select(Track, Artist, Stream, Views) %>% slice_head(n = 15)

top_songs_long = top_songs %>%
  mutate(Label = str_trunc(paste0(Track, " (", Artist, ")"), 45)) %>%
  mutate(Label = fct_reorder(Label, Stream)) %>%
  pivot_longer(cols = c(Stream, Views), names_to = "Metrika", values_to =
"Vrednost")

ggplot(top_songs_long, aes(x = Label, y = Vrednost, fill = Metrika)) +
  geom_col(position = "dodge") +
  coord_flip() +
  scale_y_continuous(labels = scales::label_number(scale = 1e-9, suffix =
"B")) +
  scale_fill_manual(values = c("Stream" = "green", "Views" = "red")) +
  theme_minimal(base_size = 12) +
```

```
labs(title = "Top 15 pesama po broju stream-ova (Stream naspram Views)", x
= "", y = "Broj (milijarde)", fill = "")
```



Vrednosti Views u top 15 pesama generalno prate visok Stream, uz izuzetak jedne pesme ("One Dance"), koja ima relativno nizak broj pregleda u poređenju sa ostalim pesmama na listi. Takođe je bitno primetiti da imamo i neke ponovljene pesme, kako u skupu podataka postoji više pojavljivanja jedne pesme za različite umetnike (Senorita, STAY).

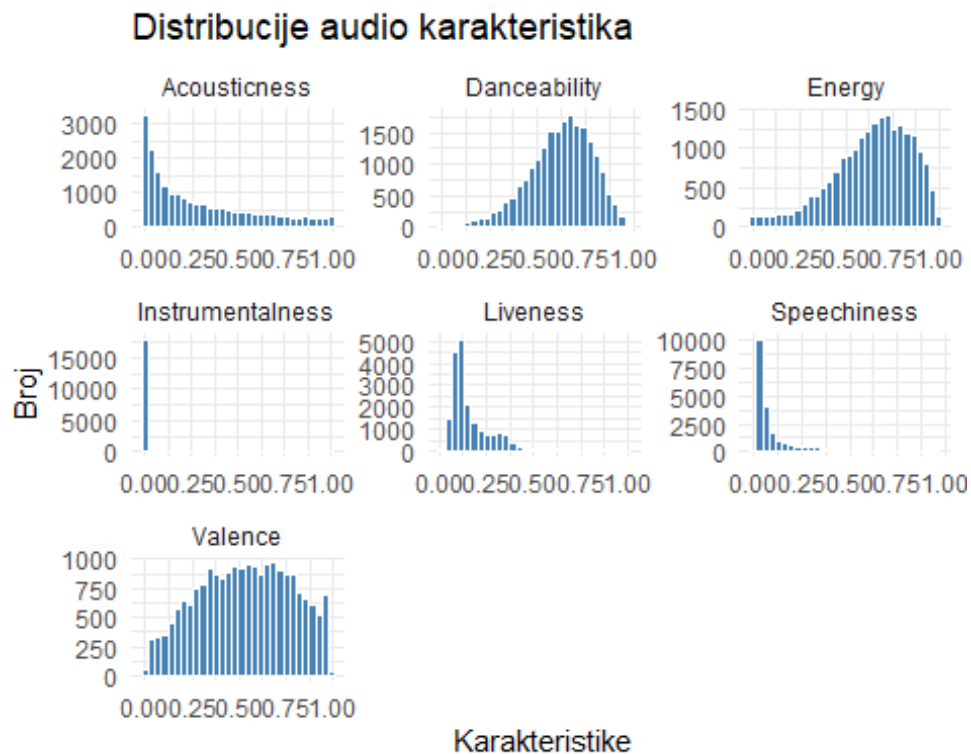
Audio karakteristike

Kako bismo lakše pregledali karakteristike kolona koje predstavljaju audio karakteristike, pregledaćemo ih kolektivno, jer ove kolone (**Danceability**, **Energy**, **Loudness**, **Speechiness**, **Acousticness**, **Instrumentalness**, **Liveness**, **Valence**, **Tempo**, **Duration_ms**) predstavljaju numeričke promenljive slične prirode jer opisuju zvučne karakteristike pesme. Na istom dijagramu ćemo pregledati kolone koje imaju vrednosti u opsegu 0-1, a zatim ćemo posebno pregledati Duration_ms, Loudness i Tempo, jer su u drugačijem opsegu.

```
audio_cols = c("Danceability", "Energy", "Speechiness", "Acousticness",
               "Instrumentalness", "Liveness", "Valence")

data %>%
  select(all_of(audio_cols)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~variable, scales = "free") +
```

```
theme_minimal() +
labs(title = "Distribucije audio karakteristika", x = 'Karakteristike', y =
'Broj')
```

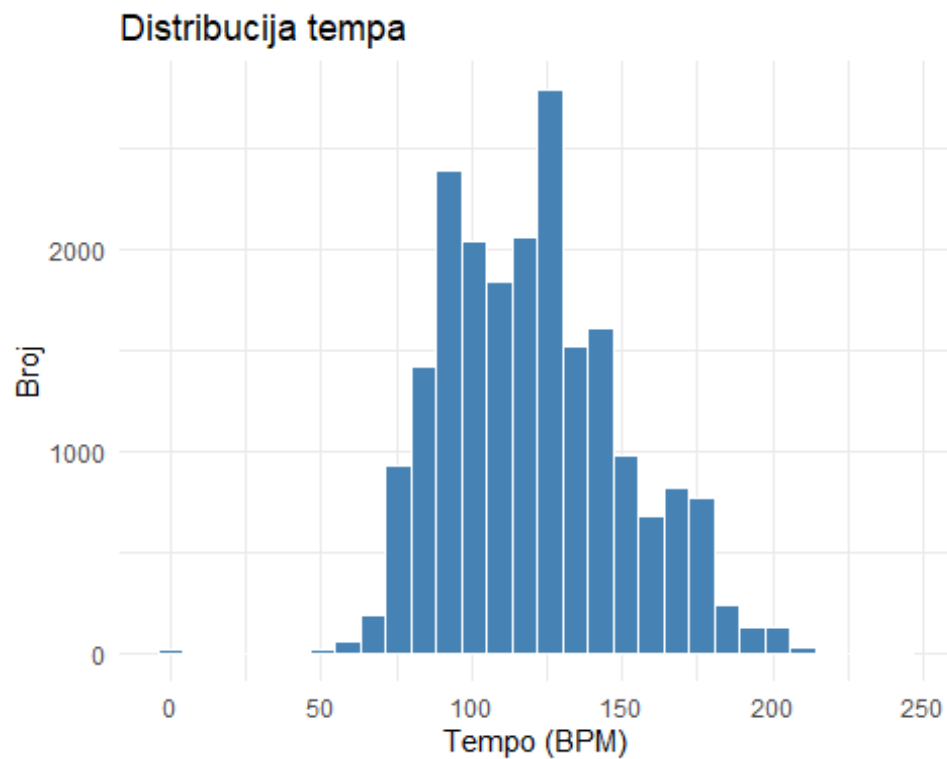


Kolone **Acousticness**, **Instrumentalness**, **Speechiness** i **Liveness** pokazuju izraženu desnu asimetričnost, sa velikom koncentracijom vrednosti u blizini nule, što je očekivano u odnosu na domensko znanje koje imamo o ovim karakteristikama, kako popularne pesme često nisu snimljene uživo (nizak Liveness), nisu samo instrumentalne, već imaju i neki oblik pevanja preko (nizak Instrumentalness) i slično.

Za kolone čiji se opseg kreće od 0 do 1, nema preteranog smisla raditi logaritamsku transformaciju, tako da bi eventualno imale smisla transformacije feature engineering-om kao što je uvođenje binarne promenljive koja predstavlja da li pesma ima ili nema određenu osobinu.

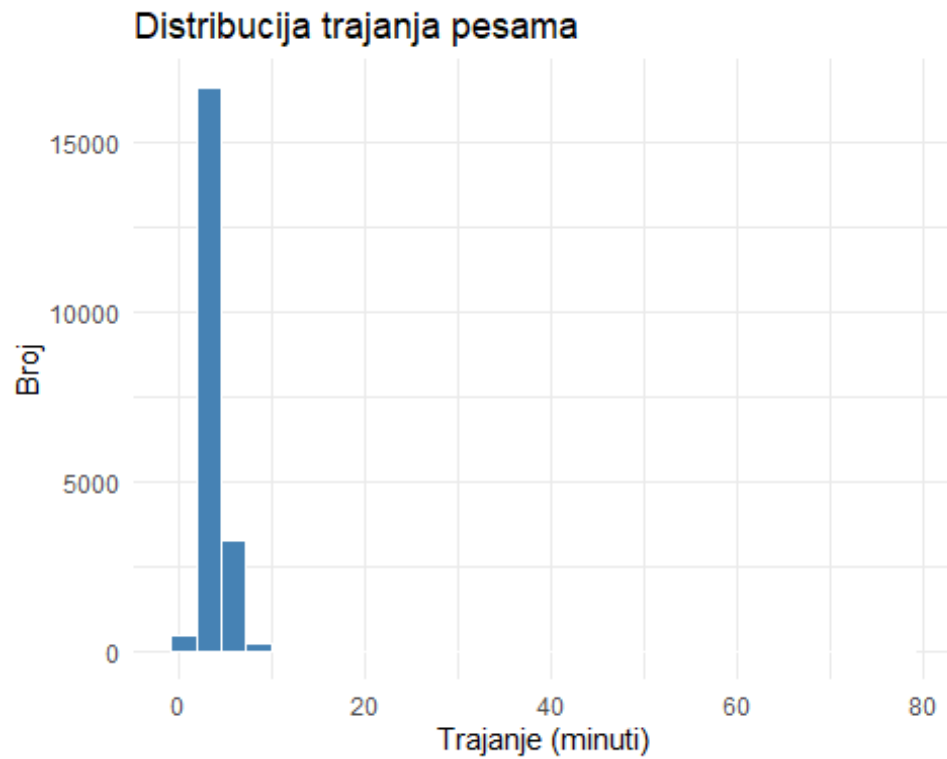
Kolone **Danceability**, **Energy**, **Valence** imaju raspodele koje su najbliže normalnoj.

```
data %>%
  ggplot(aes(x = Tempo)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribucija tempa", x = "Tempo (BPM)", y = "Broj")
```



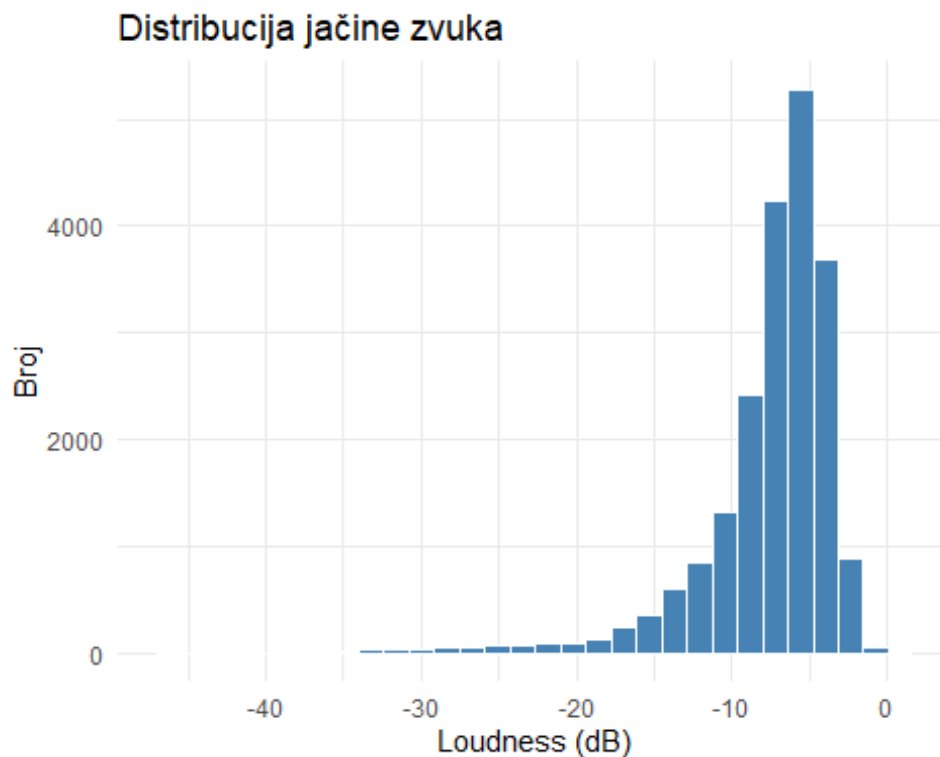
Kolone Tempo se meri u BPM (Beats per Minute) i ima raspodelu koja je blizu normalnoj, bez nekih izraženih neravnomernosti.

```
data %>%  
  mutate(Duration_min = Duration_ms / 60000) %>%  
  ggplot(aes(x = Duration_min)) +  
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +  
  theme_minimal() +  
  labs(title = "Distribucija trajanja pesama", x = "Trajanje (minuti)", y =  
"Broj")
```



Kolona **Duration_ms** pokazuje izraženu desnu asimetriju, pa bi ovde bi log transformacija imala smisla.

```
data %>%  
  ggplot(aes(x = Loudness)) +  
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +  
  theme_minimal() +  
  labs(title = "Distribucija jačine zvuka", x = "Loudness (dB)", y = "Broj")
```

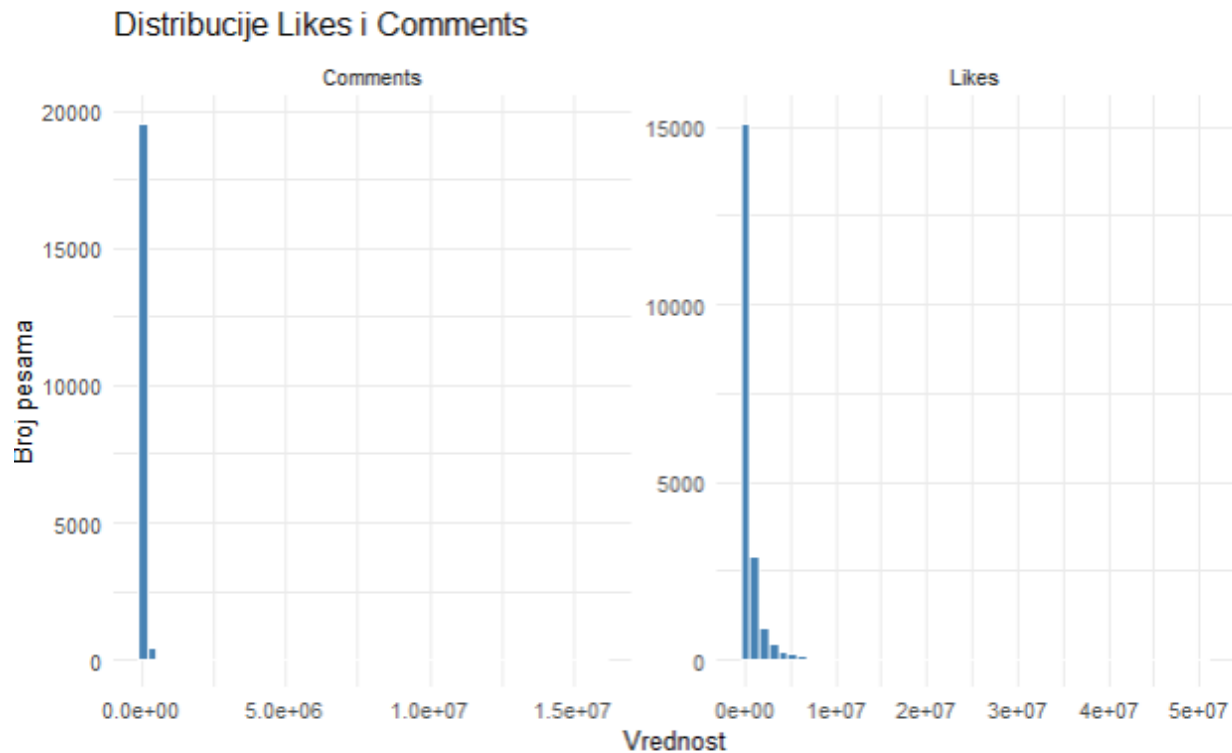


Kolona **Loudness** pokazuje levu asimetriju, sa koncentracijom vrednosti blizu maksimuma, koji iznosi 0db, pa bi ovde potencijalno imati smisla odraditi logaritamsku transformaciju, ukoliko kolonu koristimo kao prediktor.

Reakcije publike na YouTube-u

Pored pregleda na Youtube-u, pregledaćemo i broj komentara i lajkova na videu.

```
data %>%
  select(Likes, Comments) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  facet_wrap(~variable, scales = "free") +
  theme_minimal() +
  labs(title = "Distribucije Likes i Comments", x = "Vrednost", y = "Broj
pesama")
```

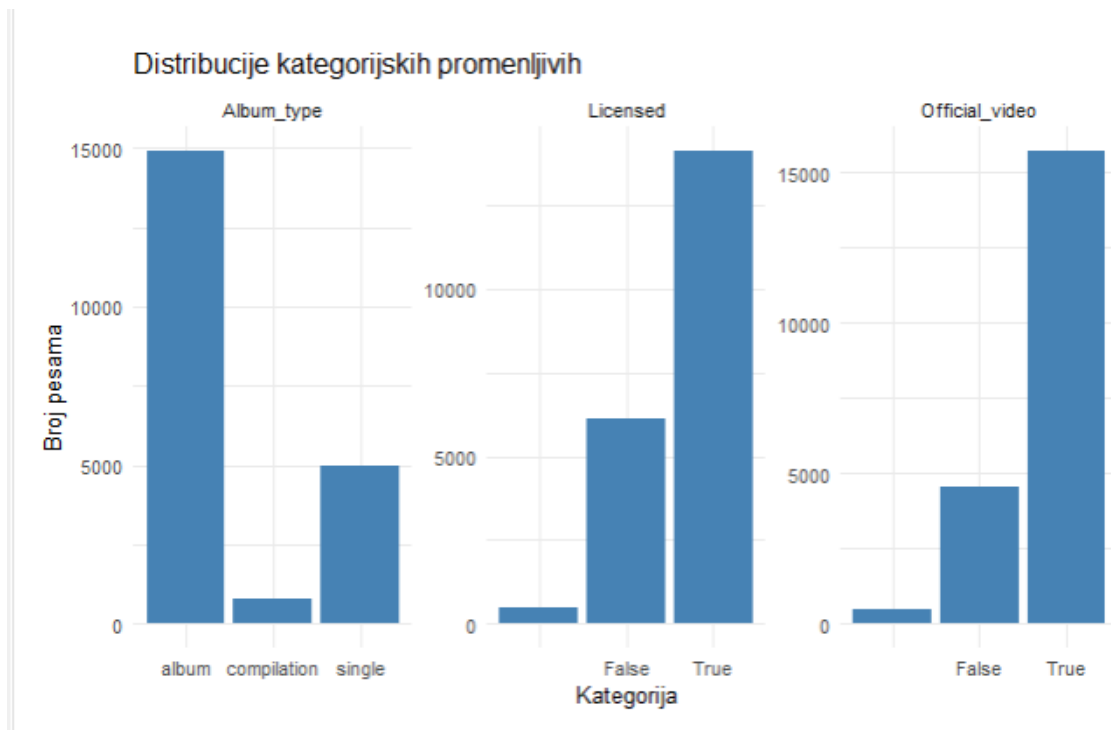


Kolone Likes i Comments predstavljaju mere reakcije publike na YouTube video spot. Slično kao i kod Views, obe promenljive pokazuju izraženu desnu asimetriju, što znači da mali broj izuzetno popularnih video spotova prikuplja veliki broj lajkova i komentara. Zbog ove osobine, kasnije bi bilo korisno izvršiti logaritamsku transformaciju i ovih kolona.

Kategorijske promenljive (Album_type, Licensed, Official_video)

Nakon numeričkih kolona, pregledaćemo kako su organizovane kategorijske promenljive i koliko su zastupljene različite kategorije u njima.

```
data %>%
  select(Licensed, Official_video, Album_type) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_bar(fill = "steelblue") +
  facet_wrap(~variable, scales = "free") +
  theme_minimal() +
  labs(title = "Distribucije kategorijskih promenljivih", x = "Kategorija", y
= "Broj pesama")
```

U okviru kategorijskih promenljivih, uočava se određena neravnomerna raspodela kategorija:

- **Album_type** - većina pesama objavljena je u okviru albuma, dok je manji broj objavljen kao singl, a najmanji broj pesama je objavljen kao kompilacija.
- **Licensed** - većina pesama je objavljena na Youtube-u kao licenciran sadržaj.
- **Official_video** - većina pesama ima spot koji je zvaničan.

Za Licensed i Official_video postoji dodatan stubić jer su to prazni stringovi, koji verovatno označavaju nedostajanje te vrednosti.

Izvođač i kanal

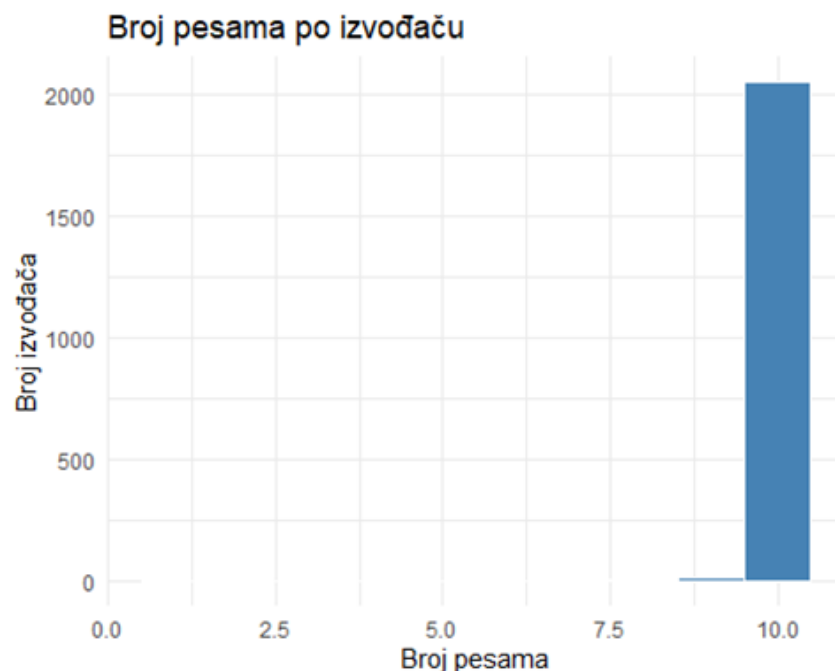
Pored numeričkih i kategorijskih promenljivih, skup podataka sadrži i dve promenljive sa velikim brojem različitih nivoa, a to su Artist (izvođač pesme) i Channel (YouTube kanal na kome je pesma objavljena).

```
data %>%
  summarise(
    unique_artists = n_distinct(Artist),
    unique_channels = n_distinct(Channel)
  )

##   unique_artists unique_channels
## 1             2079             6715
```

Izvođača ima preko 2000, dok kanala ima preko 6000, što je u potpunosti razumljivo ukoliko uzmemo u obzir da pesme istih izvođača mogu biti objavljene na više različitih kanala i slično.

```
data %>%  
  count(Artist, name = "broj_pesama") %>%  
  count(broj_pesama, name = "broj_izvodjaca") %>%  
  arrange(broj_pesama)  
  
##   broj_pesama broj_izvodjaca  
## 1           1              2  
## 2           3              1  
## 3           6              3  
## 4           7              5  
## 5           8              1  
## 6           9             18  
## 7          10            2049  
  
data %>%  
  count(Artist) %>%  
  ggplot(aes(x = n)) +  
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +  
  theme_minimal() +  
  labs(title = "Broj pesama po izvođaču", x = "Broj pesama", y = "Broj izvođača")
```

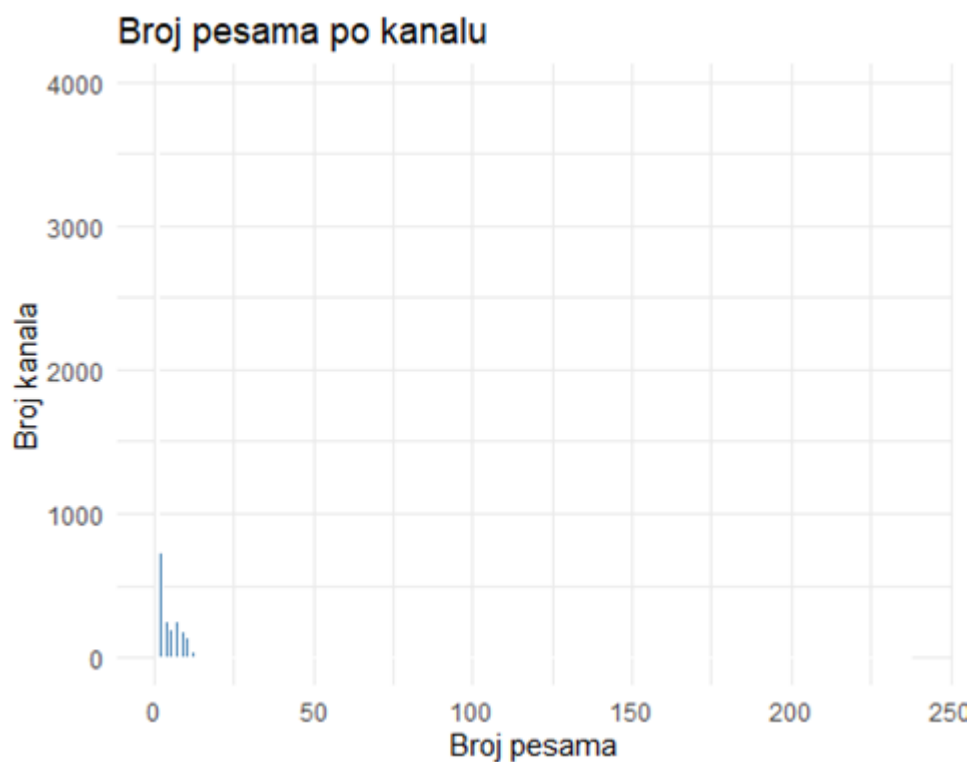


U tabelarnom prikazu se može videti da 2049 izvođača ima 10 pesama, što je u skladu sa opisom skupa. Preostalih 30 izvođača ima manje od 10 pesama, što može biti znak da za

neke izvođače nije bilo dovoljno pesama za kreiranje uzorka, što je potvrđeno i na histogramu.

Nakon toga proverićemo i broj pesama na kanalu.

```
data %>%  
  filter(!is.na(Channel), Channel != "") %>%  
  count(Channel) %>%  
  ggplot(aes(x = n)) +  
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +  
  theme_minimal() +  
  labs(title = "Broj pesama po kanalu", x = "Broj pesama", y = "Broj kanala")
```



Distribucija broja pesama po kanalu je jako desno asimetrična, i vidi se da većina kanala ima jednu pesmu. Međutim, ovaj histogram je jako razvučen i nije reprezentativan za zaključivanje, tako da ćemo probati da napravimo kategorije za broj pesama, odnosno da ih grupišemo, kako bismo mogli da ih prikazemo na barplot-u.

```
data %>%  
  filter(!is.na(Channel), Channel != "") %>%  
  count(Channel, name = "broj_pesama") %>%  
  mutate(grupa = cut(broj_pesama,  
    breaks = c(0, 1, 2, 3, 5, 10, 20, 50, Inf),  
    labels = c("1", "2", "3", "4-5", "6-10",
```

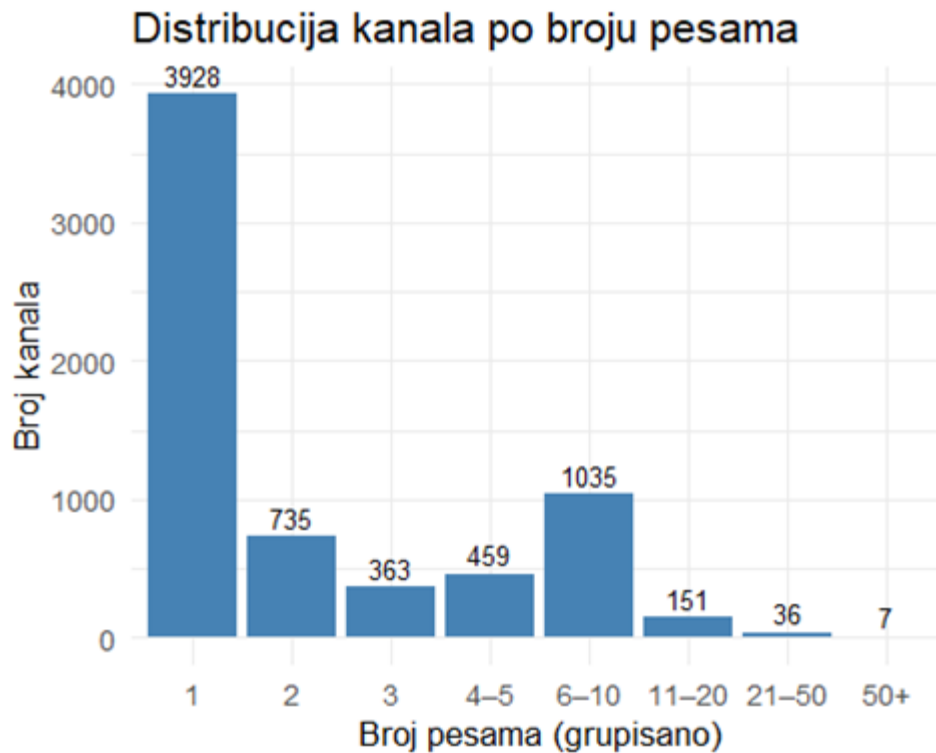
```

    "11-20", "21-50", "50+")) %>%
count(grupa, name = "broj_kanala")

##   grupa broj_kanala
## 1     1         3928
## 2     2          735
## 3     3          363
## 4  4-5          459
## 5  6-10         1035
## 6 11-20          151
## 7 21-50           36
## 8   50+           7

data %>%
  filter(!is.na(Channel), Channel != "") %>%
  count(Channel, name = "broj_pesama") %>%
  mutate(grupa = cut(broj_pesama, breaks = c(0, 1, 2, 3, 5, 10, 20, 50, Inf), labels = c("1",
"2", "3", "4-5", "6-10", "11-20", "21-50", "50+"))) %>%
  count(grupa, name = "broj_kanala") %>%
  ggplot(aes(x = grupa, y = broj_kanala)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = broj_kanala), vjust = -0.4, size = 3.5) +
  theme_minimal(base_size = 13) +
  labs(title = "Distribucija kanala po broju pesama",
    x = "Broj pesama (grupisano)",
    y = "Broj kanala")

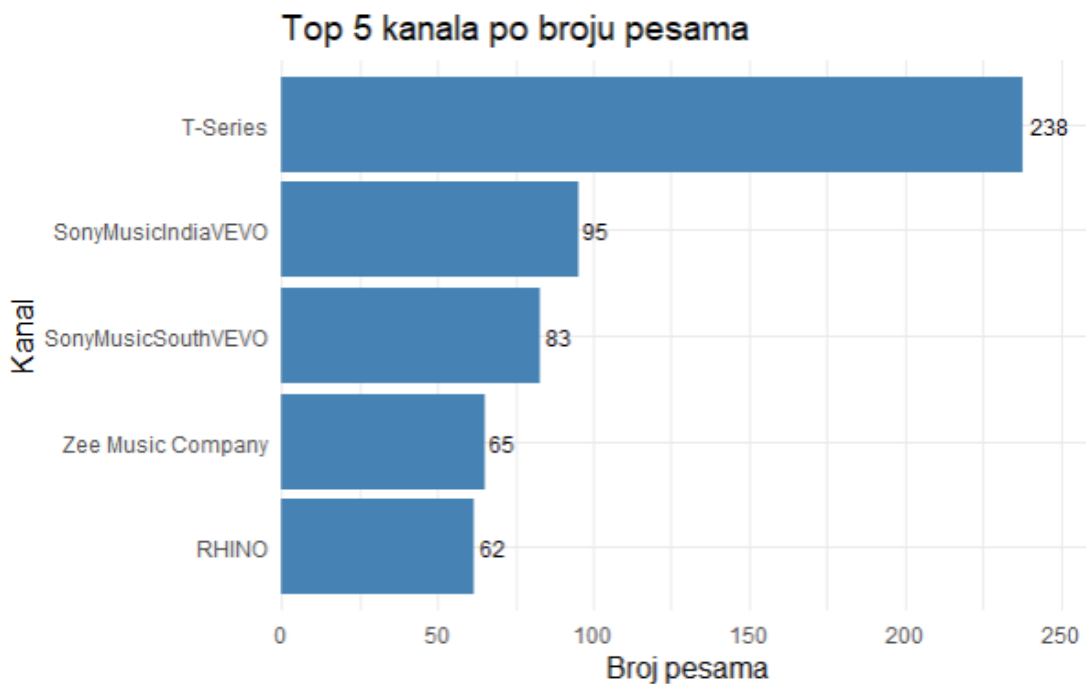
```



U prikazu na barplot-u se vidi da većina kanala ima samo jednu (više od 50%), što je potvrđeno i na histogramu, verovatno jer neki kanali pripadaju isključivo jednom umetniku, a neki kanali su veći distributeri pa imaju mnogo više objavljenih spotova. Ovo možemo proveriti ukoliko pogledamo top 5 kanala sa najviše pesama.

```
top_channels = data %>%
  filter(!is.na(Channel), Channel != "") %>%
  count(Channel) %>%
  arrange(desc(n)) %>%
  slice_head(n = 5)

ggplot(top_channels, aes(x = fct_reorder(Channel, n), y = n)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = n), hjust = -0.2, size = 3.5) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(title = "Top 5 kanala po broju pesama", x = "Kanal", y = "Broj pesama")
```



Na barplot-u je naša inicijalna tvrdnja i potvrđena, kako su ovi kanali neke veće kompanije, a ne pojedinačni izvođači.

```
channel_sizes = data %>%
  filter(!is.na(Channel), Channel != "") %>%
  count(Channel)

summary(channel_sizes$n)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.000  1.000   1.000   3.016  4.000 238.000

mean(channel_sizes$n == 1)

## [1] 0.5850462
```

Na osnovu činjenice da više od 50% kanala imaju samo jednu pesmu, možemo otprilike da zaključimo da ova kolona neće biti pogodna za transformaciju i korišćenje u modelima, čak ni ukoliko koristimo target ili frequency encoding, ali to možemo proveriti u odnosu na Views.

Bivarijantna analiza - Stream

Bivarijantna analiza predstavlja ispitivanje odnosa između dve promenljive sa ciljem utvrđivanja postojanja, jačine i pravca njihove povezanosti. Kako bismo odlučili koju

promenljivu predviđamo neophodno je da odradimo bivarijantnu analizu, odnosno odnos različitih promenljivih i potencijalnih ciljnih promenljivih, pri čemu će se u okviru ovog dela sagledati neki numeričke kolone pregledane u univariјantnoj analizi, kao i određene kategorijske promenljive, poput izvođača pesme, u odnosu na kolonu *Stream*.

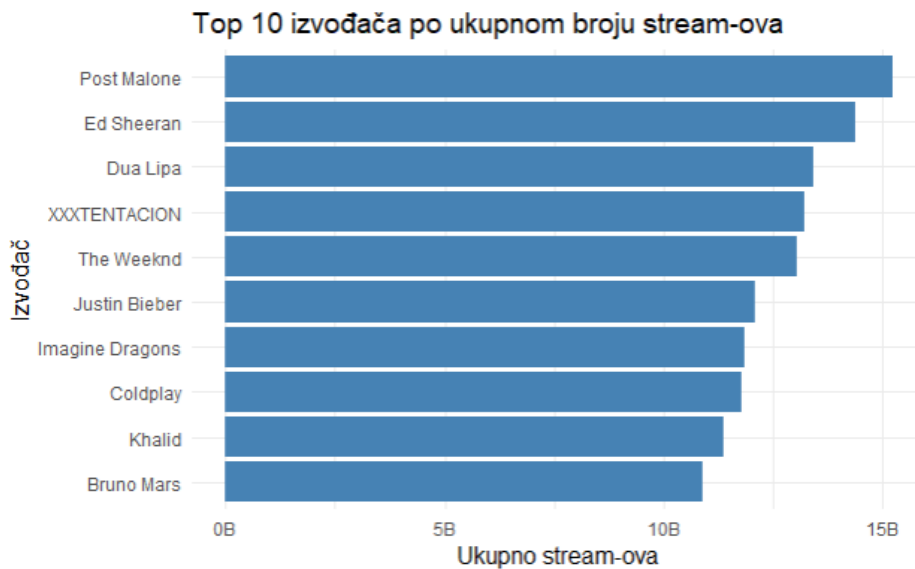
Artist

Artist je kategorijska promenljiva koja predstavlja izvođača same pesme, pri čemu postoji veliki broj različitih umetnika.

```
data %>% summarise(unique_artists = n_distinct(Artist))  
##   unique_artists  
## 1                2079
```

Kako broj jedinstvenih umetnika prelazi 2000, nije moguće niti korisno direktno porediti sve izvođače, pa ćemo pogledati top 10 izvođača po broju pesama zastupljenih u skupu.

```
top_artists_stream = data %>%  
  group_by(Artist) %>%  
  summarise(ukupno_stream = sum(Stream, na.rm = TRUE), .groups = "drop") %>%  
  arrange(desc(ukupno_stream)) %>%  
  slice_head(n = 10)  
  
ggplot(top_artists_stream, aes(x = fct_reorder(Artist, ukupno_stream), y =  
  ukupno_stream)) + geom_col(fill = "steelblue") + scale_y_continuous(labels =  
  label_number(scale = 1e-9, suffix = "B")) + coord_flip() +  
  theme_minimal(base_size = 13) + labs(title = "Top 10 izvođača po ukupnom  
  broju stream-ova", x = "Izvođač", y = "Ukupno stream-ova")
```



Na vrhu se po broju stream-ova nalaze određeni popularni pevači, što je i očekivano. Zbog prirode skupa gde svaki izvođač ima po 10 pesama, možemo da gledamo zbir stream-ova na ovaj način, u suprotnom bismo morali da gledamo izvođače po proseku stream-ova, kako ne bi na vrhu bio izvođač sa najvećim brojem pesama. Kako vizuelno ne možemo da proverimo ovu količinu različitih klasa i uticaj na stream, možemo odmah da uradimo statistički test i da saznamo uticaj, iako postoji veliki rizik da će efekat biti precenjen jer imamo puno grupa koje sadrže mali broj pesama.

```
kruskal.test(Stream ~ factor(Artist), data = data)

##
##  Kruskal-Wallis rank sum test
##
## data:  Stream by factor(Artist)
## Kruskal-Wallis chi-squared = 10210, df = 2056, p-value < 2.2e-16

kruskal_effsize(data, Stream ~ factor(Artist))

## # A tibble: 1 × 5
##   .y.      n effsize method  magnitude
## * <chr> <int>   <dbl> <chr>   <ord>
## 1 Stream 20142  0.451 eta2[H] large
```

Statistički test je pokazao veliki uticaj na stream, međutim neophodno je izvršiti dodatne provere nakon feature engineering-a, sa nekim novokreiranim kolonama, zbog problema koji je ranije spomenut sa velikim brojem različitih kategorija.

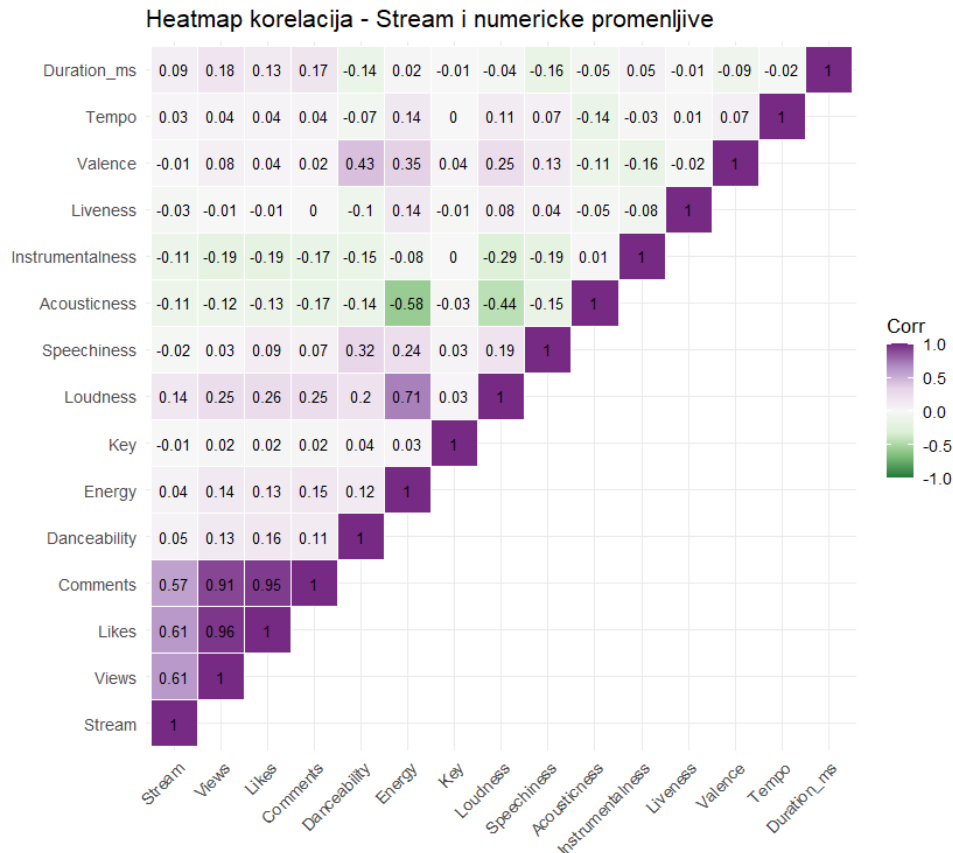
Numeričke promenljive

Kako je u okviru univarijantne analize zaključeno da numeričke promenljive (Stream, Views, Likes, Comments) nisu normalno raspodeljene, koristićemo Spearman koeficijent korelacije, zato što je on otporan na ekstremne vrednosti.

```
numeric_cols = c("Stream", "Views", "Likes", "Comments", "Danceability",
                 "Energy", "Key",
                 "Loudness", "Speechiness", "Acousticness",
                 "Instrumentalness",
                 "Liveness", "Valence", "Tempo", "Duration_ms")
corr = cor(data[, numeric_cols], use = "complete.obs", method = "spearman")

corr[lower.tri(corr)] = NA
corr.melt = melt(corr, na.rm = TRUE)

ggplot(corr.melt, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(value, 2)), size = 3.5) +
  scale_fill_distiller(palette = "PRGn", type = "div", limits = c(-1, 1)) +
  theme_minimal(base_size = 13) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  coord_fixed() +
  labs(title = "Heatmap korelacija - Stream i numericke promenljive", x = "",
       y = "", fill = "Corr")
```



Na osnovu matrice korelacija, možemo videti da su promenljive i **Views** i **Likes** i **Comments** u umereno jakoj korelaciji sa stream-om. Ono što je zajedničko za ove tri promenljive je da sve predstavljaju neku meru popularnosti na Youtube-u, imaju koeficijent korelacije veći od 0.9 međusobno, što znači da ćemo prilikom kreiranja modela morati da rešimo ovaj problem odabirom određenih kolona koje ćemo koristiti za predikciju.

Audio karakteristike su znatno slabije korelisane sa Stream-om, pri čemu nijedna od kolona koja pripada ovoj kategoriji ne prelazi vrednost od 0.14, što bi značilo da to kako pesma zvuči ima mali uticaj na njenu popularnost. Da bi se vrednost ovih promenljivih ipak nekako iskoristila, možemo isprobati feature engineering i kreiranje neke kombinacije i kategorizacije ovih kolona kako bi se eventualno postigla veća korelacija.

Album_type

Prvo ispitujemo da li se broj stream-ova razlikuje u zavisnosti od tipa albuma na kome se pesma nalazi (album, single, compilation).

```
data %>%
  group_by(Album_type) %>%
  summarise(
    Count = n(),
    Mean_Stream = mean(Stream, na.rm = TRUE),
```

```

Median_Stream = median(Stream, na.rm = TRUE),
Std_Stream = sd(Stream, na.rm = TRUE),
IQR_Stream = IQR(Stream, na.rm = TRUE)
) %>%
arrange(desc(Median_Stream))

## # A tibble: 3 × 6
##   Album_type Count Mean_Stream Median_Stream Std_Stream IQR_Stream
##   <chr>      <int>      <dbl>         <dbl>      <dbl>      <dbl>
## 1 album      14926  149978199.    56691188. 260506213. 132001036.
## 2 compilation  788    84007846.    43244696. 119642211.  76068446.
## 3 single     5004   101670765.    33307874. 198721268.  90672683.

```

Opisna statistika pokazuje da pesme objavljene u okviru albuma (**album**) imaju najveću medijanu broja stream-ova.

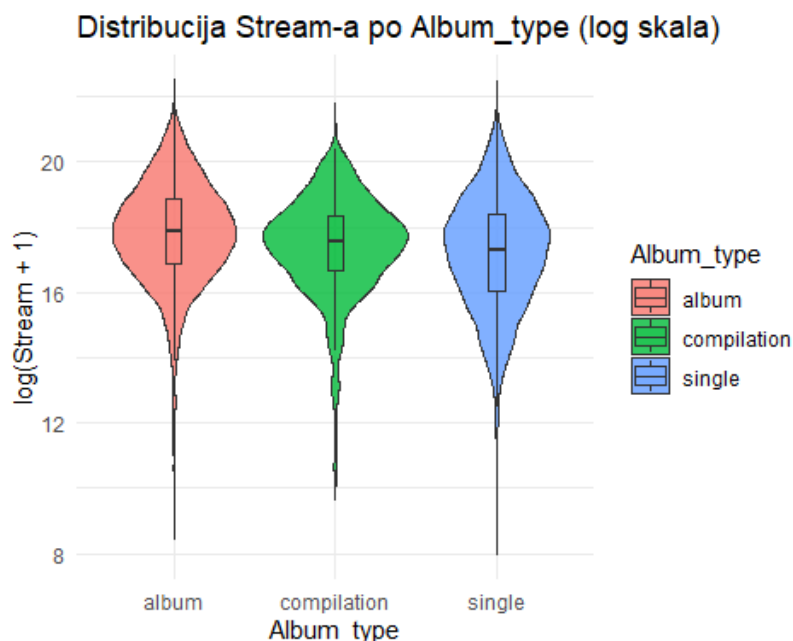
```

ggplot(data, aes(x = Album_type, y = log1p(Stream), fill = Album_type)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
  labs(title = "Distribucija Stream-a po Album_type (log skala)", x =
"Album_type", y = "log(Stream + 1)")

## Warning: Removed 576 rows containing non-finite outside the scale range
## (`stat_ydensity()`).

## Warning: Removed 576 rows containing non-finite outside the scale range
## (`stat_boxplot()`).

```



Violin plot dodatno oslikava ove razlike, sve tri distribucije su slične širine, ali se centar kod kategorije album vizuelno nalazi nešto više u odnosu na single.

```

set.seed(123)
shapiro_boot(data, "Album_type", "Stream")

## [[1]]
## [1] "Varijabla Stream po album kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Stream po single kategoriji nema normalnu raspodelu."
##
## [[3]]
## [1] "Varijabla Stream po compilation kategoriji nema normalnu raspodelu."

```

Kao i što smo mogli da očekujemo, raspodela stream-a nije normalna ni u jednoj kategoriji.

```

kruskal.test(Stream ~ Album_type, data = data)

##
## Kruskal-Wallis rank sum test
##
## data: Stream by Album_type
## Kruskal-Wallis chi-squared = 465.32, df = 2, p-value < 2.2e-16

kruskal_effsize(data, Stream ~ Album_type)

## # A tibble: 1 × 5
## .y.          n effsize method magnitude
## * <chr>    <int>    <dbl> <chr>    <ord>
## 1 Stream 20142  0.0230 eta2[H] small

```

Kruskal-Wallis test pokazuje statistički značajnu razliku u broju stream-ova između tri kategorije Album_type. Međutim, effect size spada u kategoriju **malog** efekta prema rstatix klasifikaciji. Ono što je važno je činjenica da kako u skupu imamo veliki broj opservacija, čak i male razlike između grupa lako prelaze prag statističke značajnosti, pa je problematično to što je effect size mali.

Licensed

Pre nego što pređemo na analizu, važno je napomenuti da su **Licensed** i **Official_video** promenljive koje opisuju YouTube video, a ne pesmu na Spotify-u. Uprkos tome, vredi analizirati ove kolone u odnosu na stream, kako se može dogoditi da će se status videa na jednoj platformi na neki način odraziti na drugu platformu, na primer zbog veće vidljivosti videa jer je oficijelan.

```

data_licensed = data %>% filter(Licensed != "")

data_licensed %>%
  group_by(Licensed) %>%
  summarise(
    Count = n(),
    Mean_Stream = mean(Stream, na.rm = TRUE),

```

```

    Median_Stream = median(Stream, na.rm = TRUE),
    Std_Stream = sd(Stream, na.rm = TRUE),
    IQR_Stream = IQR(Stream, na.rm = TRUE)
  ) %>%
  arrange(desc(Median_Stream))

## # A tibble: 2 × 6
##   Licensed Count Mean_Stream Median_Stream Std_Stream IQR_Stream
##   <chr>    <int>      <dbl>         <dbl>      <dbl>      <dbl>
## 1 True      14140  153848132.    55700784.  267685103.  137948921.
## 2 False     6108   97454338.    39692237  179730119.   85799431.

```

Opisna statistika pokazuje da pesme koje su licensed imaju veću medijanu broja stream-ova u odnosu na one koje nisu, razlika je od oko 1.4 puta. Kao i kod Album_type, standardna devijacija je nekoliko puta veća od medijane u obe grupe, što potvrđuje desnu asimetriju unutar svake kategorije.

Vredi primetiti i da je uzorak neujednačen, jer je grupa licenciranih videa više nego duplo veća od druge grupe.

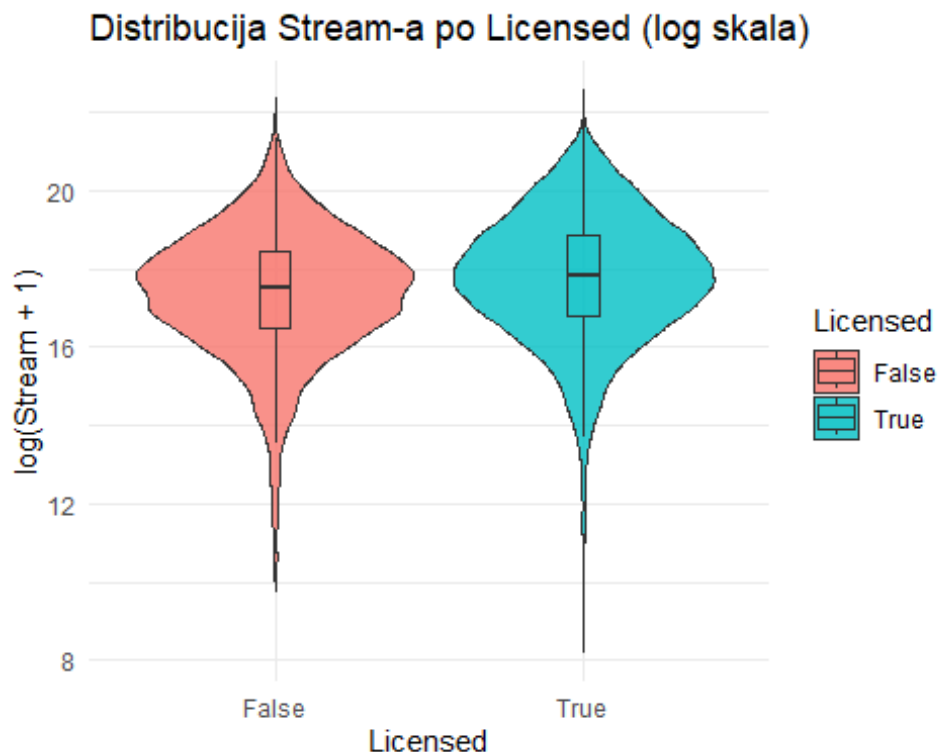
```

ggplot(data_licensed, aes(x = Licensed, y = log1p(Stream), fill = Licensed))
+
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
  labs(title = "Distribucija Stream-a po Licensed (log skala)", x =
"Licensed", y = "log(Stream + 1)")

## Warning: Removed 556 rows containing non-finite outside the scale range
## (`stat_ydensity()`).

## Warning: Removed 556 rows containing non-finite outside the scale range
## (`stat_boxplot()`).

```



Violin plot potvrđuje razliku uočenu u opisnoj statistici, ali pokazuje da je ona umerena, medijana kod licenciranih je malo pomerena naviše u odnosu na nelicencirane, a oblici raspodele su vrlo slični.

```
set.seed(123)
shapiro_boot(data_licensed, "Licensed", "Stream")

## [[1]]
## [1] "Varijabla Stream po True kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Stream po False kategoriji nema normalnu raspodelu."
```

Rezultati bootstrap Shapiro-Wilk testa potvrđuju da Stream nema normalnu raspodelu ni u jednoj kategoriji, pa ćemo koristiti neparametarski test.

```
wilcox.test(Stream ~ Licensed, data = data_licensed)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Stream by Licensed
## W = 34999289, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0

wilcox_effsize(Stream ~ Licensed, data = data_licensed)
```

```
## # A tibble: 1 × 7
##   .y.      group1 group2 effsize    n1    n2 magnitude
## * <chr>  <chr>  <chr>    <dbl> <int> <int> <ord>
## 1 Stream False   True    0.111  5908 13784 small
```

Mann-Whitney test pokazuje statistički značajnu razliku između grupa, dok effect size prema rstatix klasifikaciji spada u kategoriju **malog** (*small*) efekta. Ovo dodatno potvrđuje da, iako licenciran sadržaj u proseku ima nešto veći broj stream-ova, ta veza je praktično slaba i Licensed status verovatno neće biti naročito koristan prediktor za Stream.

Official_video

Kao kod kolone Licensed, proverićemo raspodela Stream-a po grupa koje se nalaze u okviru ove promenljive (True ili False).

```
data_video = data %>% filter(Official_video != "")

data_video %>%
  group_by(Official_video) %>%
  summarise(
    Count = n(),
    Mean_Stream = mean(Stream, na.rm = TRUE),
    Median_Stream = median(Stream, na.rm = TRUE),
    Std_Stream = sd(Stream, na.rm = TRUE),
    IQR_Stream = IQR(Stream, na.rm = TRUE)
  ) %>%
  arrange(desc(Median_Stream))

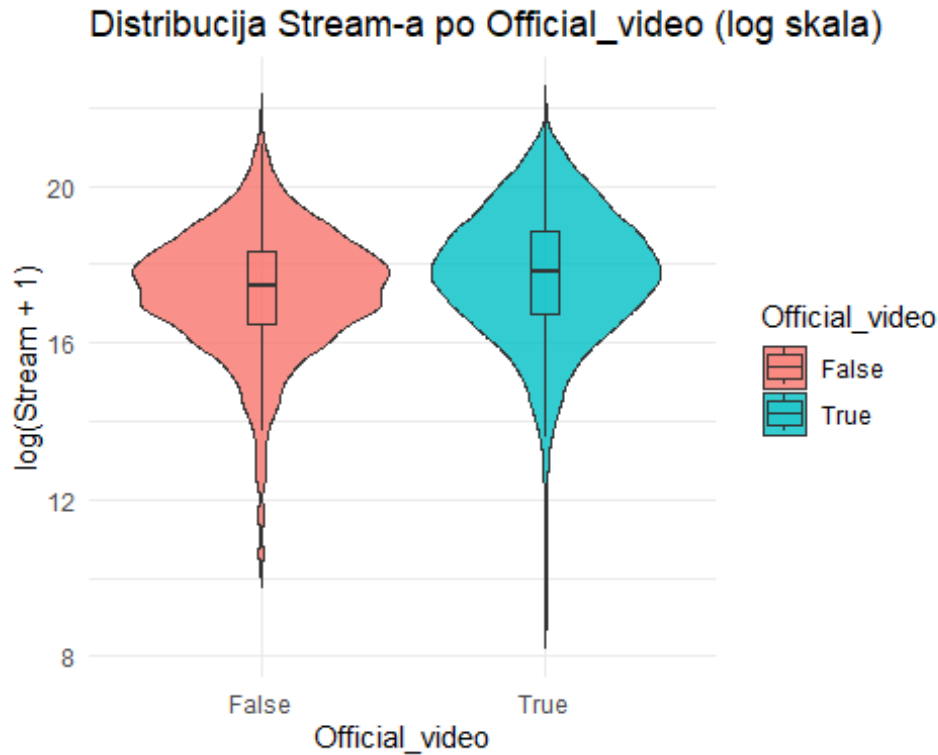
## # A tibble: 2 × 6
##   Official_video Count Mean_Stream Median_Stream Std_Stream IQR_Stream
##   <chr>          <int>    <dbl>        <dbl>    <dbl>    <dbl>
## 1 True          15723 150583264.    54559070. 262898823. 136064108.
## 2 False         4525  88969435.    38047584. 165398575.  77231817.
```

Pesme sa zvaničnim video spotom imaju veću medijalnu vrednost broja stream-ova u odnosu na one bez njega.

```
ggplot(data_video, aes(x = Official_video, y = log1p(Stream), fill =
Official_video)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
  labs(title = "Distribucija Stream-a po Official_video (log skala)", x =
"Official_video", y = "log(Stream + 1)")

## Warning: Removed 556 rows containing non-finite outside the scale range
## (`stat_ydensity()`).

## Warning: Removed 556 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Violin plot pokazuje razdvojenost grupa koja je vizuelno umerena, nema neke preterane vizuelne razlike.

```
set.seed(123)
shapiro_boot(data_video, "Official_video", "Stream")

## [[1]]
## [1] "Varijabla Stream po True kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Stream po False kategoriji nema normalnu raspodelu."
```

Rezultati bootstrap Shapiro-Wilk testa potvrđuju da Stream nema normalnu raspodelu ni u jednoj kategoriji, pa ćemo koristiti neparametarski test.

```
wilcox.test(Stream ~ Official_video, data = data_video)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Stream by Official_video
## W = 28488375, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0

wilcox_effsize(Stream ~ Official_video, data = data_video)

## # A tibble: 1 × 7
## .y. group1 group2 effsize n1 n2 magnitude
```



```
## * <chr> <chr> <chr> <dbl> <int> <int> <ord>
## 1 Stream False True 0.107 4364 15328 small
```

Mann-Whitney test pokazuje statistički značajnu razliku između grupa, ali rstatix klasifikacija nam pokazuje da je efekat mali (*small*).

Zaključak bivarijantne analize nad kolonom Stream

Nakon bivarijantne analize možemo da zaključimo koje kolone mogu biti korisni prediktorni za broj stream-ova.

Izvođač - kolona Artist ima jak uticaj na Stream, ali zbog velikog broja jedinstvenih vrednosti, Artist nije pogodan za direktno korišćenje u modelu bez enkodiranja, ili nekog oblika transformacije.

Mere popularnosti sa YouTube-a. Views, Likes i Comments pokazuju umerenu do jaku pozitivnu korelaciju sa Stream-om, što je očekivano obzirom na to da sve promenljive mere popularnost iste pesme.

Audio karakteristike. Nijedna audio karakteristika ne prelazi korelaciju od |0.14| sa Stream-om, što ukazuje da to kako pesma zvuči ima slab uticaj na njenu popularnost, odnosno da je komercijalni uspeh pesme verovatno pretežno određen faktorima izvan produkcije.

Kategorijske promenljive. Album_type, Licensed i Official_video pokazuju statistički značajne, ali praktično male razlike u broju stream-ova.

Od svih razmatranih promenljivih, mere popularnosti sa YouTube-a i identitet izvođača pokazuju najizraženiju vezu sa brojem stream-ova, dok audio karakteristike i kategorijske promenljive vezane za YouTube video imaju manji uticaj.

Bivarijantna analiza - Views

Cilj ove sekcije je da ispitamo odnos između promenljive **Views** i ostalih promenljivih u skupu podataka, kako bismo utvrdili koji atributi imaju potencijal da budu značajni prediktori, i kako bismo ovu analizu mogli da uporedimo sa bivarijantnom analizom nad Stream-om iz prethodnog dela. Analizu sprovodimo u tri dela:

- Views naspram numeričkih promenljivih (audio karakteristike, Duration_ms, Likes, Comments, Stream)
- Views naspram kategorijskih promenljivih (Album_type, Licensed, Official_video, Artist, Channel)

Artist

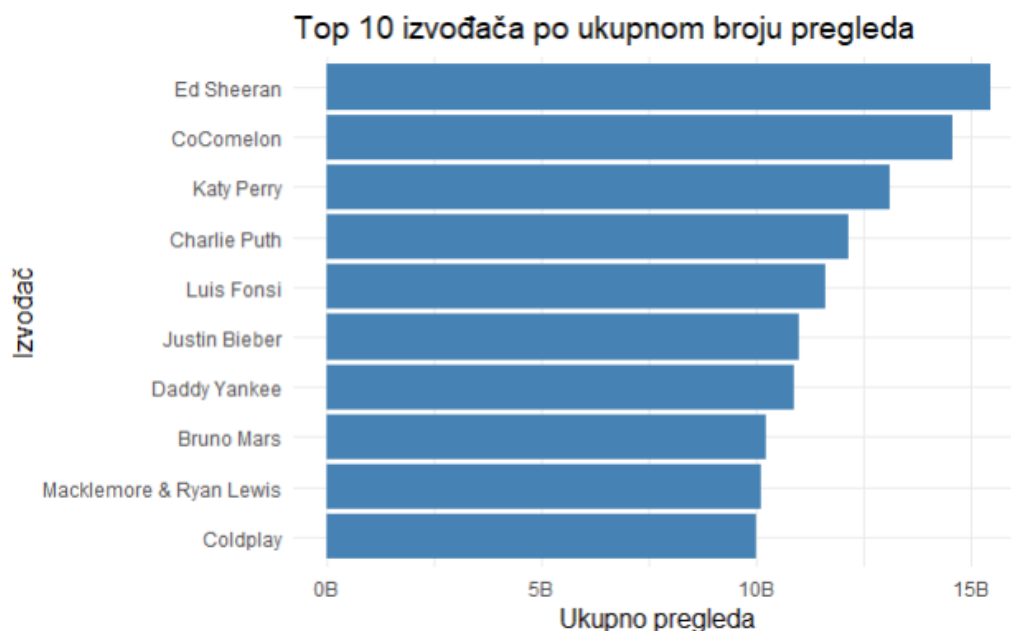
Kao i kod Stream-a, prvo proveravamo da li izvođač ima sistematski uticaj na broj pregleda, pri čemu već znamo da jedinstvenih izvođača u skupu ima preko 2000.

```

top_artists_views = data %>%
  group_by(Artist) %>%
  summarise(ukupno_views = sum(Views, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(ukupno_views)) %>%
  slice_head(n = 10)

ggplot(top_artists_views, aes(x = fct_reorder(Artist, ukupno_views), y =
ukupno_views)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = label_number(scale = 1e-9, suffix = "B")) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(title = "Top 10 izvođača po ukupnom broju pregleda", x = "Izvođač", y =
= "Ukupno pregleda")

```



Top 10 lista uglavnom sadrži poznate izvođače što je očekivano, s tim što se izdvaja CoComelon, jer nije klasičan muzički izvođač, nego dečji kanal, ali to ima smisla jer deca generalno gledaju isti video više puta, i neke dečje pesme imaju najviše pregleda na Youtube-u.

```

kruskal.test(Views ~ factor(Artist), data = data)

##
## Kruskal-Wallis rank sum test
##
## data: Views by factor(Artist)
## Kruskal-Wallis chi-squared = 8771.8, df = 2062, p-value < 2.2e-16

```

```
kruskal_effsize(data, Views ~ factor(Artist))
```

```
## # A tibble: 1 × 5  
##   .y.      n effsize method  magnitude  
## * <chr> <int>   <dbl> <chr>   <ord>  
## 1 Views 20248    0.369 eta2[H] large
```

Kruskal-Wallis effect size spada u kategoriju velikog efekta prema rstatix klasifikaciji, isto kao i kod Stream-a, s tim što važi isti problem sa velikim brojem malih grupa kao kod Stream-a.

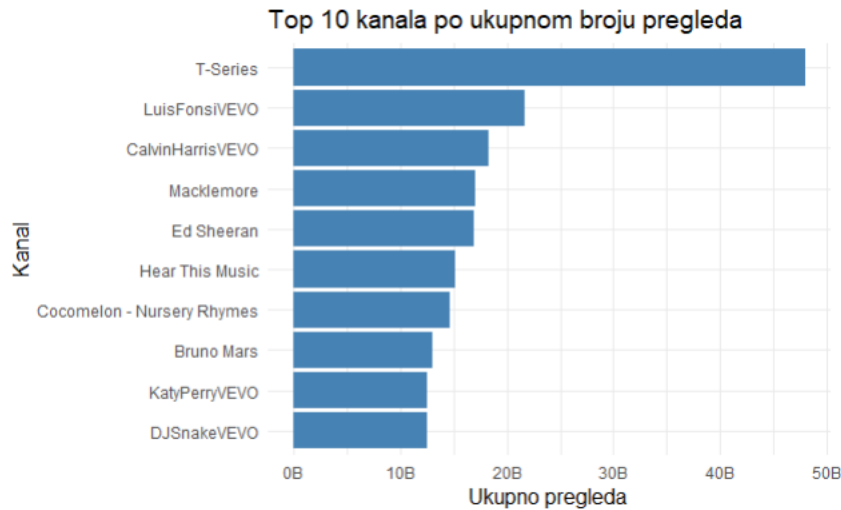
Channel

Za razliku od Stream-a, kod Views-a ima smisla proveriti i YouTube kanal na kome je video objavljen, nezavisno od izvođača, kako isti izvođač može imati objavljene pesme na više kanala, a ti kanali se mogu značajno razlikovati po broju pratilaca, što direktno utiče na broj pregleda nezavisno od same pesme.

```
data %>%  
  filter(!is.na(Channel), Channel != "") %>%  
  summarise(unique_channels = n_distinct(Channel))  
  
##   unique_channels  
## 1              6714
```

U skupu ima više kanala nego izvođača, preko 6000.

```
top_channels_views = data %>%  
  filter(!is.na(Channel), Channel != "") %>%  
  group_by(Channel) %>%  
  summarise(ukupno_views = sum(Views, na.rm = TRUE), broj_pesama = n(),  
    .groups = "drop") %>%  
  arrange(desc(ukupno_views)) %>%  
  slice_head(n = 10)  
  
ggplot(top_channels_views, aes(x = fct_reorder(Channel, ukupno_views), y =  
  ukupno_views)) +  
  geom_col(fill = "steelblue") +  
  scale_y_continuous(labels = label_number(scale = 1e-9, suffix = "B")) +  
  coord_flip() +  
  theme_minimal(base_size = 13) +  
  labs(title = "Top 10 kanala po ukupnom broju pregleda", x = "Kanal", y =  
    "Ukupno pregleda")
```



T-Series dominira po ukupnom broju pregleda i ima ih duplo više od drugog na listi, što ima smisla jer je T-Series ogroman bolivijski kanal, jedan od najgledanijih kanala na YouTube-u uopšte. Ostatak liste sadrži kanale vezane za pojedinačne izvođače i Cocomelon.

```
data_channel = data %>% filter(!is.na(Channel), Channel != "")
kruskal.test(Views ~ factor(Channel), data = data_channel)

##
## Kruskal-Wallis rank sum test
##
## data: Views by factor(Channel)
## Kruskal-Wallis chi-squared = 14644, df = 6713, p-value < 2.2e-16

kruskal_effsize(data_channel, Views ~ factor(Channel))

## # A tibble: 1 × 5
##   .y.      n effsize method magnitude
## * <chr> <int>   <dbl> <chr>   <ord>
## 1 Views 20248  0.586 eta2[H] large
```

Effect size za Channel je veći nego za Artist, ali sa preko 6000 grupa i prosečno 3 pesme po kanalu, ovaj broj je veoma nepouzdan.

Numeričke promenljive

Koristimo Spearman koeficijent korelacije umesto Pearson-ovog, obzirom na to da smo u univarijantnoj analizi utvrdili da numeričke promenljive nisu normalno raspodeljene, a Spearman je otporan na ekstremne vrednosti.

```
numeric_cols = c("Views", "Danceability", "Energy", "Key", "Loudness",
                 "Speechiness", "Acousticness", "Instrumentalness",
                 "Liveness", "Valence", "Tempo", "Duration_ms",
                 "Likes", "Comments", "Stream")
```

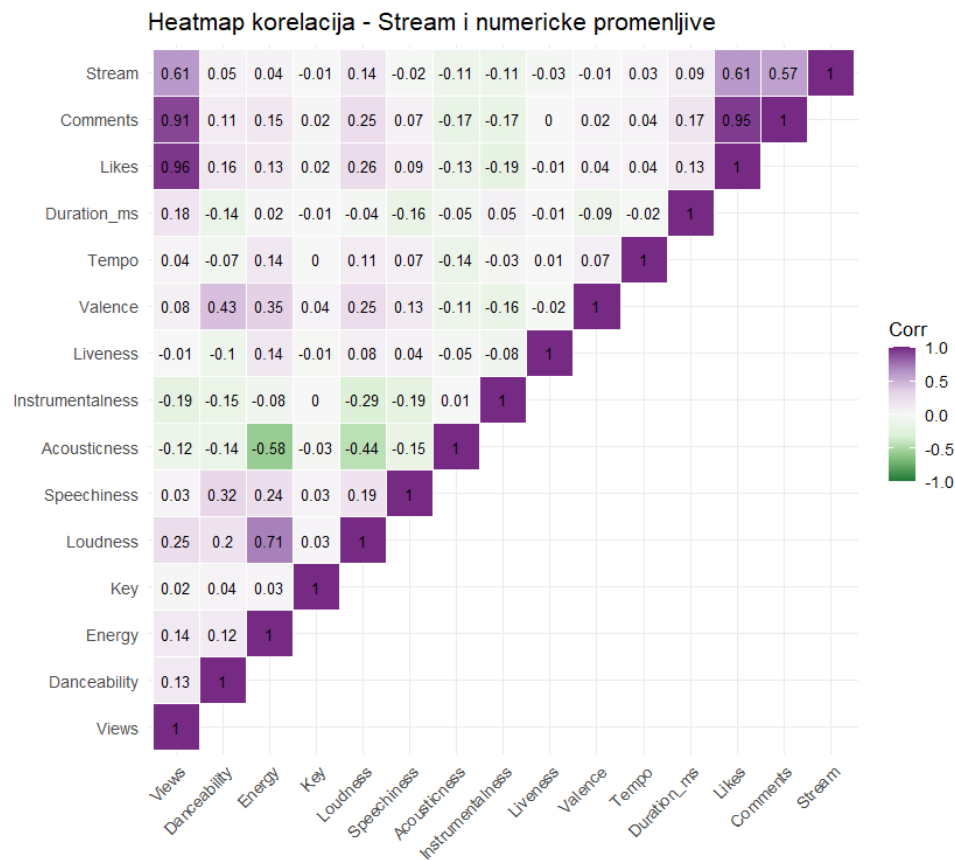
```

corr = cor(data[, numeric_cols], use = "complete.obs", method = "spearman")

corr[lower.tri(corr)] = NA
corr.melt = melt(corr, na.rm = TRUE)

ggplot(corr.melt, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(value, 2)), size = 3.5) +
  scale_fill_distiller(palette = "PRGn", type = "div", limits = c(-1, 1)) +
  theme_minimal(base_size = 13) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  coord_fixed() +
  labs(title = "Heatmap korelacija - Stream i numericke promenljive", x = "",
y = "", fill = "Corr")

```



```

sort(corr["Views", ], decreasing = TRUE)

```

	Views	Likes	Comments	Stream
##	1.000000000	0.961463715	0.910569375	0.608819905
##	Loudness	Duration_ms	Energy	Danceability
##	0.248383896	0.178198534	0.136712648	0.132697484
##	Valence	Tempo	Speechiness	Key
##	0.079501531	0.039813474	0.031264637	0.018004654

##	Liveness	Acousticness	Instrumentalness
##	-0.009276683	-0.116852827	-0.186646682

Analizom heatmape uočavamo da su ubedljivo najjače povezane promenljive sa Views upravo **Likes** i **Comments**, što je i očekivano obzirom na to da su to mere reakcije publike izmerene na istom YouTube videu. Nešto slabija, ali i dalje jaka veza postoji sa **Stream**, što ukazuje da je popularnost pesme donekle povezana kroz obe platforme.

Sa druge strane, audio karakteristike pesme pokazuju samo slabe veze sa brojem pregleda: najizraženije su **Loudness** i **Duration_ms**, dok su **Energy**, **Danceability** i **Valence** ispod 0.15, a **Instrumentalness** i **Acousticness** pokazuju blagu negativnu vezu. Ovo sugerše da same zvučne karakteristike pesme imaju veoma ograničen uticaj na broj pregleda, popularnost videa verovatno je više određena faktorima koji nisu direktno obuhvaćeni audio atributima. Korelacija sa **Key** je zanemarljiva, što je i očekivano jer Key predstavlja nominalnu (kategorijsku) promenljivu kodiranu brojevima, pa ovakav oblik poređenja sa njom suštinski nema smisla.

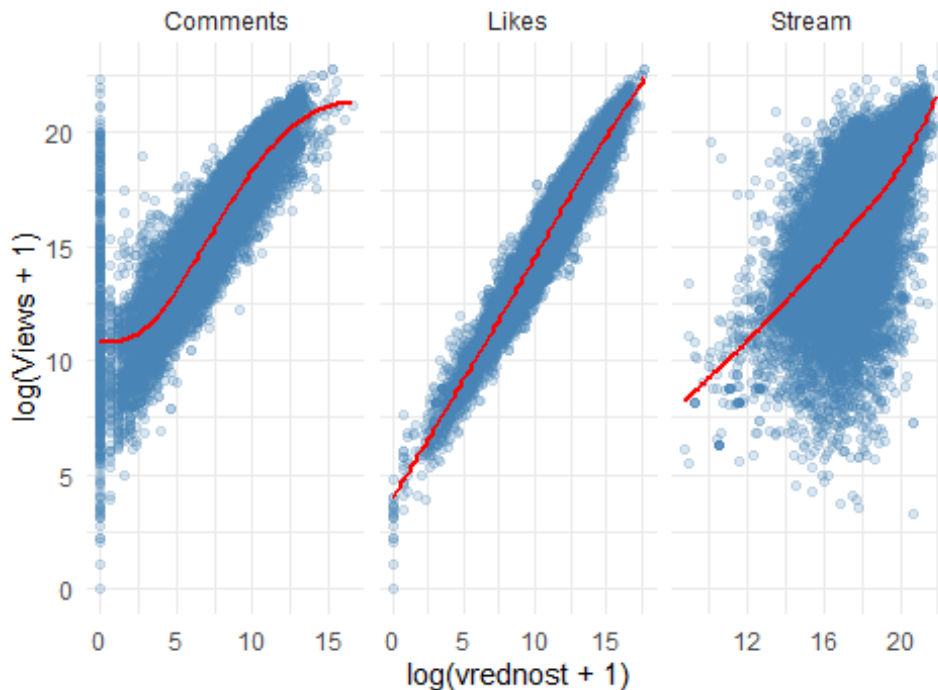
Važno je napomenuti da **Likes**, **Comments** i **Stream** bi trebalo oprezno koristiti kao prediktore ukoliko Views postane ciljna promenljiva, reč je o varijablama koje se prikupljaju u istom trenutku i istom kontekstu kao i sam broj pregleda, a ne o karakteristikama poznatim unapred, pa postoji rizik od curenja podataka.

```
data %>%
  select(Views, Likes, Comments, Stream) %>%
  pivot_longer(cols = c(Likes, Comments, Stream), names_to = "variable",
values_to = "value") %>%
  ggplot(aes(x = log1p(value), y = log1p(Views))) +
  geom_point(alpha = 0.2, color = "steelblue") +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  facet_wrap(~variable, scales = "free_x") +
  theme_minimal() +
  labs(title = "Views naspram Likes, Comments i Stream (log skala)", x =
"log(vrednost + 1)", y = "log(Views + 1)")

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 2136 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

Views naspram Likes, Comments i Stream (log skala)



Scatter plotovi potvrđuju razlike u jačini veze koje smo uočili u korelacionoj matrici, ali otkrivaju i dodatne obrasce koje sama korelaciona vrednost ne pokazuje:

- **Likes** pokazuje najčistiju, gotovo linearnu vezu sa Views na log-skali kroz ceo opseg vrednosti, jer oblak tačke tesno prate regresionu liniju, što je u skladu sa najvišom izračunatom korelacijom.
- **Comments** pokazuje S-oblik krive, jer za pesme sa nula komentara, broj pregleda i dalje značajno varira, što znači da odsustvo komentara samo po sebi ne govori mnogo o popularnosti videa.
- **Stream** pokazuje raspršeniju raspodelu tačaka, u skladu sa slabijom korelacijom. Posebno se izdvaja grupa pesama sa visokim brojem stream-ova na Spotify-u, ali neočekivano niskim brojem YouTube pregleda, što nam pokazuje da popularnost na dve platforme ne prati uvek isti obrazac.

Album_type

```
data %>%
  group_by(Album_type) %>%
  summarise(
    Count = n(),
    Mean_Views = mean(Views, na.rm = TRUE),
    Median_Views = median(Views, na.rm = TRUE),
    Std_Views = sd(Views, na.rm = TRUE),
    IQR_Views = IQR(Views, na.rm = TRUE)
  ) %>%
  arrange(desc(Median_Views))
```

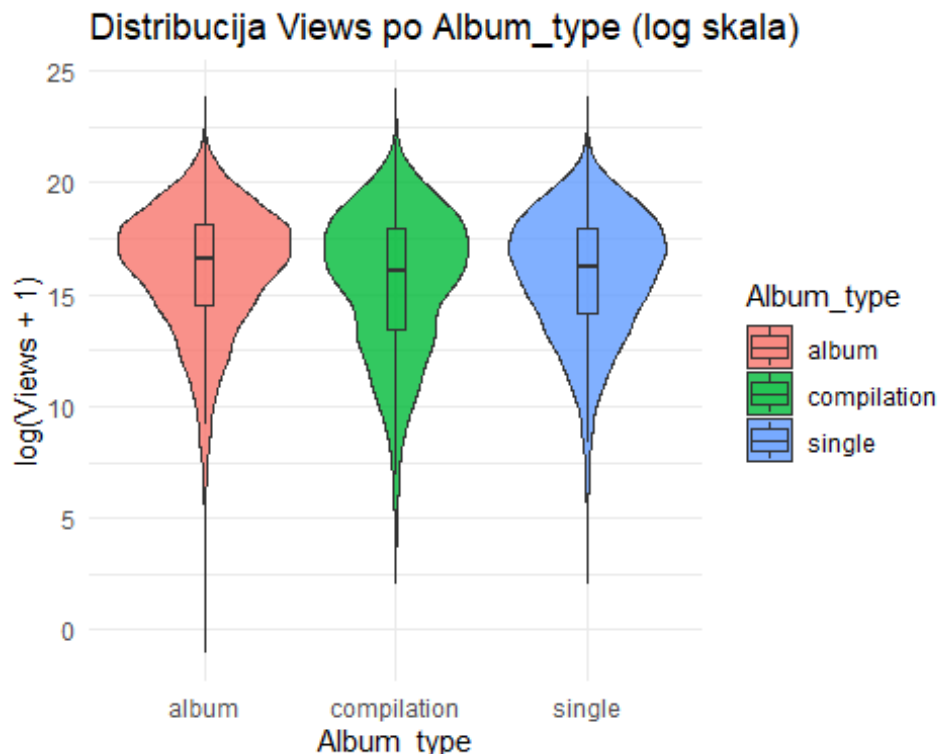
```
## # A tibble: 3 × 6
##   Album_type Count Mean_Views Median_Views Std_Views IQR_Views
##   <chr>      <int>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 album      14926  98427393.    15781710 285893290. 70590170.
## 2 single      5004   82698859.    11324830 246102326. 61911540.
## 3 compilation  788   79618318.    9569629 219674148. 62468954
```

Opisna statistika pokazuje da pesme sa albuma (**album**) imaju najveću medijanu za Views, zatim slede **single** i **compilation**. Takođe se uočava da je standardna devijacija u sve tri grupe ogromna u odnosu na medijanu, zbog desne asimetrije Views-a.

```
ggplot(data, aes(x = Album_type, y = log1p(Views), fill = Album_type)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
  labs(title = "Distribucija Views po Album_type (log skala)", x =
"Album_type", y = "log(Views + 1)")
```

```
## Warning: Removed 470 rows containing non-finite outside the scale range
## (`stat_ydensity()`).
```

```
## Warning: Removed 470 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Violin plot pokazuje da su oblici raspodele Views veoma slični u sve tri kategorije, slične širine, sa dugim donjim repom u svakoj grupi. Medijane su vizuelno blizu jedna drugoj, jer log transformacija kompresuje razlike koje smo uočili na originalnoj skali, iako razlika ne

izgleda drastično na grafiku, kod ovako velikog broja opservacija i mala razlika može biti statistički značajna.

Iako je malo verovatno da je Views normalno raspodeljena unutar bilo koje kategorije Album_type, proverićemo ovo formalno, umesto da se oslonimo samo na pretpostavku.

```
set.seed(123)
shapiro_boot(data, "Album_type", "Views")

## [[1]]
## [1] "Varijabla Views po album kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Views po single kategoriji nema normalnu raspodelu."
##
## [[3]]
## [1] "Varijabla Views po compilation kategoriji nema normalnu raspodelu."
```

Kao što smo i pretpostavili, Views nema normalnu raspodelu ni u jednoj od tri kategorije Album_type (album, single, compilation), što potvrđuje opravdanost korišćenja neparametarskog Kruskal-Wallis testa.

Da bismo utvrdili da li je razlika i statistički i praktično značajna, sprovodimo Kruskal-Wallis test uz effect size.

```
kruskal.test(Views ~ Album_type, data = data)

##
## Kruskal-Wallis rank sum test
##
## data: Views by Album_type
## Kruskal-Wallis chi-squared = 48.647, df = 2, p-value = 2.731e-11

kruskal_effsize(Views ~ Album_type, data = data)

## # A tibble: 1 × 5
## .y.      n effsize method magnitude
## * <chr> <int> <dbl> <chr> <ord>
## 1 Views 20248 0.00230 eta2[H] small
```

Iako Kruskal-Wallis test pokazuje statistički veoma značajnu razliku, effect size je jako mali. Ovo znači da, iako postoji statistički merljiva razlika u broju pregleda između različitih tipova albuma, ta razlika je toliko mala da nema praktičnog značaja, pa Album_type verovatno neće biti koristan prediktor.

Licensed

Pretpostavka je da je licenciran sadržaj verovatno zvanično distribuiran od strane izdavačke kuće ili label-a, pa bi mogao imati veću vidljivost i samim tim više pregleda.

Kolona Licensed sadrži, pored True i False, prazan string "", što su verovatno opservacije kod kojih nedostaju i ostali YouTube podaci. Kako Mann-Whitney test zahteva tačno dve grupe, za potrebe ove analize ih izbacujemo.

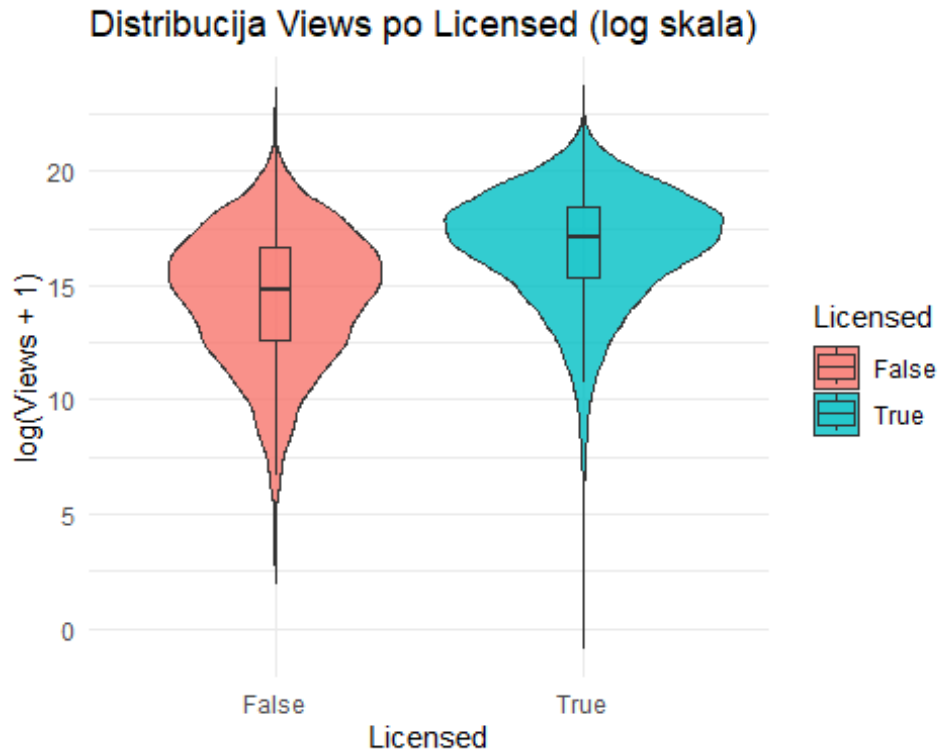
```
data_licensed = data %>% filter(Licensed != "")

data_licensed %>%
  group_by(Licensed) %>%
  summarise(
    Count = n(),
    Mean_Views = mean(Views, na.rm = TRUE),
    Median_Views = median(Views, na.rm = TRUE),
    Std_Views = sd(Views, na.rm = TRUE),
    IQR_Views = IQR(Views, na.rm = TRUE)
  ) %>%
  arrange(desc(Median_Views))

## # A tibble: 2 × 6
##   Licensed Count Mean_Views Median_Views Std_Views IQR_Views
##   <chr>    <int>    <dbl>      <dbl>    <dbl>    <dbl>
## 1 True      14140 121161134.    25815653 312882264. 95308319
## 2 False     6108  30915941.    2818800. 133193071. 16725729
```

Za razliku od Album_type, ovde vidimo mnogo izraženiju razliku, pesme koje su licensed imaju dosta veću medijalnu vrednost Views od onih koje nisu licensed, što je u skladu pretpostavkom da licenciran sadržaj ima veću vidljivost.

```
ggplot(data_licensed, aes(x = Licensed, y = log1p(Views), fill = Licensed)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
  labs(title = "Distribucija Views po Licensed (log skala)", x = "Licensed",
y = "log(Views + 1)")
```



Violin plot potvrđuje ovu razliku vizuelno, distribucija za licensed je pomeren na više vrednosti u odnosu na pesme koje nisu licensed, dok oba oblika raspodele ostaju slična. Ovo je znatno izraženija razlika nego što smo videli kod Album_type, što sugerise da bi Licensed mogao biti relevantniji prediktor za Views.

Iako je malo verovatno da je Views normalno raspodeljena unutar bilo koje kategorije Licensed, proverićemo to formalno Shapiro-Wilk testom sa bootstrap pristupom.

```
set.seed(123)
shapiro_boot(data_licensed, "Licensed", "Views")

## [[1]]
## [1] "Varijabla Views po True kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Views po False kategoriji nema normalnu raspodelu."
```

Kao što smo i pretpostavili, Views nema normalnu raspodelu ni u jednoj od dve kategorije Licensed (True, False), što opravdava korišćenje Mann-Whitney U testa.

```
wilcox.test(Views ~ Licensed, data = data_licensed)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Views by Licensed
```

```
## W = 24044285, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0

wilcox_effsize(Views ~ Licensed, data = data_licensed)

## # A tibble: 1 × 7
##   .y.    group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr>  <chr>    <dbl> <int> <int> <ord>
## 1 Views False   True     0.352  6108 14140 moderate
```

Mann-Whitney test pokazuje i statističku i praktičnu značajnost, effect size spada u kategoriju umerene (*moderate*) razlike. Ovo potvrđuje da je Licensed relevantna promenljiva, jer pesme koje su licencirane imaju znatno veći broj pregleda od onih koje nisu, i ova razlika nije samo posledica velikog uzorka, već predstavlja stvarnu, praktično uočljivu razliku. Licensed bi mogao biti koristan prediktor za Views.

Official_video

Postoji mogućnost da pesme sa zvaničnim video spotom su vizuelno bolje i više promovisane (u odnosu na lyrics video ili generičku sliku), pa bismo očekivali veći broj pregleda.

I ovde izbacujemo prazan string iz analize, jer Mann-Whitney test očekuje 2 kategorije.

```
data_video = data %>% filter(Official_video != "")

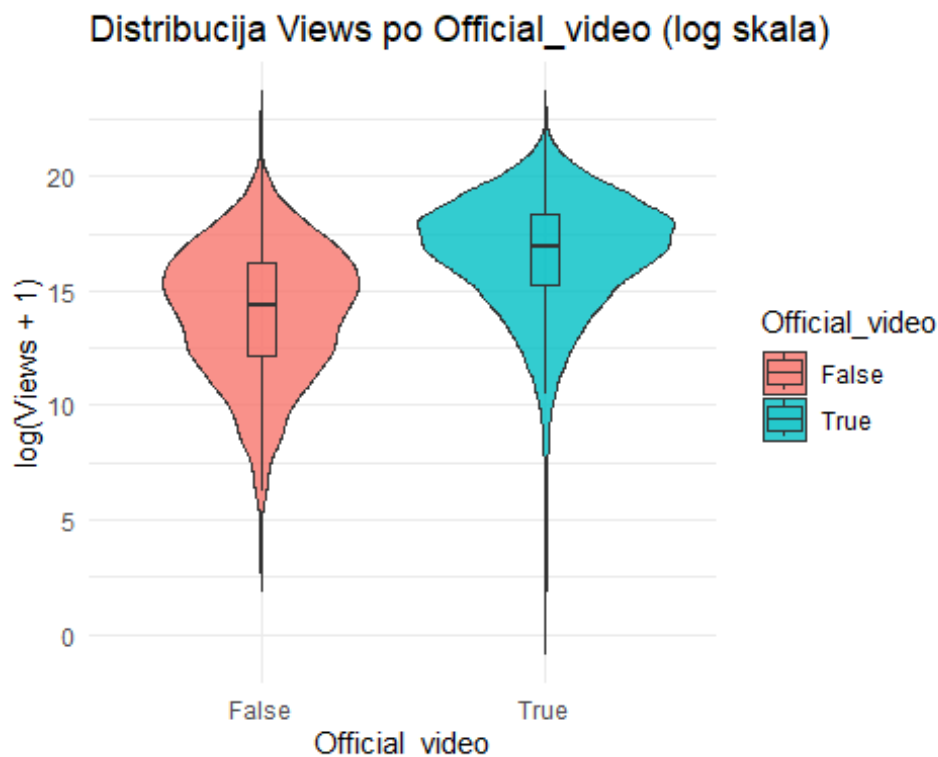
data_video %>%
  group_by(Official_video) %>%
  summarise(
    Count = n(),
    Mean_Views = mean(Views, na.rm = TRUE),
    Median_Views = median(Views, na.rm = TRUE),
    Std_Views = sd(Views, na.rm = TRUE),
    IQR_Views = IQR(Views, na.rm = TRUE)
  ) %>%
  arrange(desc(Median_Views))

## # A tibble: 2 × 6
##   Official_video Count Mean_Views Median_Views Std_Views IQR_Views
##   <chr>          <int>    <dbl>      <dbl>    <dbl>    <dbl>
## 1 True          15723 114494757.    23329150 300852401. 89245871
## 2 False         4525  22508714.    1787395 128289987. 10717072
```

Pesme sa zvaničnim spotom imaju medijalnu vrednost skoro 13 puta veću medijanu za Views.

```
ggplot(data_video, aes(x = Official_video, y = log1p(Views), fill =
Official_video)) +
  geom_violin(trim = FALSE, alpha = 0.8) +
  geom_boxplot(width = 0.1, outlier.shape = NA, alpha = 0.3) +
  theme_minimal() +
```

```
labs(title = "Distribucija Views po Official_video (log skala)", x =
"Official_video", y = "log(Views + 1)")
```



Violin plot potvrđuje vizuelno ono što smo videli u opisnoj statistici, istribucija za Official_video = True je jasno pomerena na više vrednosti u odnosu na False, uz nešto užu i koncentrisaniju raspodelu kod True.

Iako je malo verovatno da je Views normalno raspodeljena unutar bilo koje kategorije Official_video, proverićemo to formalno Shapiro-Wilk testom sa bootstrap pristupom.

```
set.seed(123)
shapiro_boot(data_video, "Official_video", "Views")

## [[1]]
## [1] "Varijabla Views po True kategoriji nema normalnu raspodelu."
##
## [[2]]
## [1] "Varijabla Views po False kategoriji nema normalnu raspodelu."
```

Kao što smo i pretpostavili, Views nema normalnu raspodelu ni u jednoj od dve kategorije Official_video (True, False), što opravdava korišćenje Mann-Whitney U testa.

```
wilcox.test(Views ~ Official_video, data = data_video)

##
## Wilcoxon rank sum test with continuity correction
##
```

```
## data: Views by Official_video
## W = 17638189, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0

wilcox_effsize(Views ~ Official_video, data = data_video)

## # A tibble: 1 × 7
##   .y.   group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr>  <chr>    <dbl> <int> <int> <ord>
## 1 Views False   True     0.364  4525 15723 moderate
```

Mann-Whitney test potvrđuje statistički veoma značajnu razliku, a effect size takođe spada u kategoriju umerene razlike. Ovo potvrđuje da Official_video ima određenu značajnost, pesme sa zvaničnim video spotom imaju značajno veći broj pregleda.

Zaključak - kategorijske promenljive

- **Channel** i **Artist** imaju značajan uticaj na Views, s tim što će nam biti neophodne određene transformacije da bismo mogli da probamo da iskoristimo ove kolone kao prediktore, pored toga što se sastoji iz velikih grupa.
- **Album_type** pokazuje statistički značajnu, ali praktično zanemarljivu razliku, verovatno neće biti koristan prediktor za Views.
- **Licensed** i **Official_video** pokazuju umerenu praktičnu značajnost, što ih čini potencijalno korisnim prediktorima za Views.

Zanimljivo je što Licensed i Official_video imaju gotovo identičnu jačinu efekta - ovo je moguće zato što su međusobno povezane promenljive (pesme sa zvaničnim video spotom su verovatno i češće licencirane). Ovu vezu bi vredelo proveriti u nekom od narednih koraka, jer bi mogla ukazivati na redundantnost (obe promenljive nose sličnu informaciju) koju treba uzeti u obzir prilikom odabira prediktora za model.

Views naspram Stream

U ovom sekciji ćemo uporediti Views i Stream kao dva kandidata za ciljnu promenljivu, pa je korisno da vidimo koliko su međusobno povezani i koliko je koji od njih kompletan (broj nedostajućih vrednosti), pre nego što donesemo konačnu odluku o ciljnoj promenljivoj.

```
sum(is.na(data$Views))
## [1] 470

sum(is.na(data$Stream))
## [1] 576
```

Views ima nešto manje nedostajućih vrednosti u odnosu na Stream.

```
ggplot(data, aes(x = log1p(Stream), y = log1p(Views))) +
  geom_point(alpha = 0.2, color = "steelblue") +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
```

```

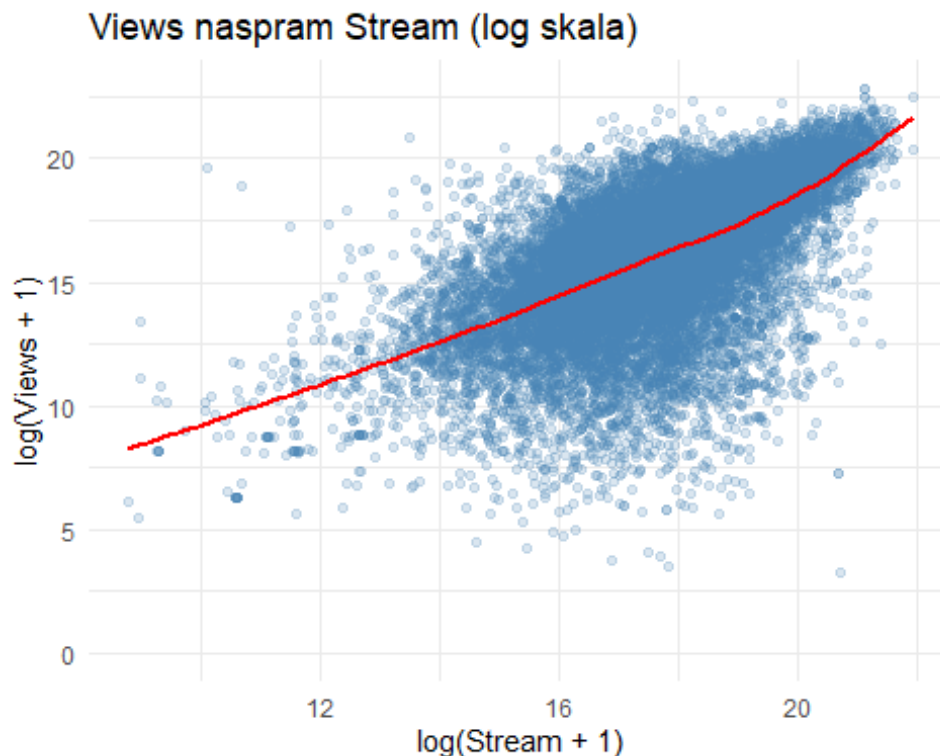
theme_minimal() +
labs(title = "Views naspram Stream (log skala)", x = "log(Stream + 1)", y =
"log(Views + 1)")

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 1026 rows containing non-finite outside the scale range
## (`stat_smooth()`).

## Warning: Removed 1026 rows containing missing values or values outside the
scale range
## (`geom_point()`).

```



Scatter plot pokazuje pozitivnu vezu, sa vidljivo većim šumom u odnosu na Likes i Comments, u skladu sa umerenom korelacijom. Vidi se grupa pesama sa srednje-visokim brojem stream-ova, ali neočekivano niskim brojem pregleda, što nam ukazuje da popularnost na dve platforme ne prati uvek isti obrazac.

```

cor.test(data$Views, data$Stream, method = "spearman")

## Warning in cor.test.default(data$Views, data$Stream, method = "spearman"):
## Cannot compute exact p-value with ties

##
## Spearman's rank correlation rho
##
## data: data$Views and data$Stream
## S = 4.9935e+11, p-value < 2.2e-16

```

```
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##          rho
## 0.6076394
```

Formalni test potvrđuje umerenu do jaku pozitivnu korelaciju, u skladu sa vrednošću iz početne korelacione matrice. Ovo pokazuje da su Views i Stream povezani, ali ne toliko snažno da bi bili međusobno zamenljivi - svaka promenljiva nosi i deo jedinstvene informacije o popularnosti pesme na svojoj platformi, pa izbor ciljne promenljive nije trivijalan.

Ukupan zaključak bivarijantne analize nad kolonom Views

- **Numeričke promenljive:** Likes i Comments su najjače povezane sa Views, ali predstavljaju potencijalni rizik od data leakage-a jer se mere istovremeno sa samim pregledima, audio karakteristike pokazuju samo slabe veze, stream pokazuje umerenu vezu.
- **Kategorijske promenljive:** Album_type nema praktičnog značaja, dok Licensed i Official_video pokazuju umerenu praktičnu značajnost i mogu biti korisni prediktori, a Artist i Channel mogu eventualno biti uz transformacije.
- **Views vs Stream:** umereno do jako korelisani, ali ne identični, svaka promenljiva nosi deo jedinstvene informacije, a Views je neznatno kompletnija varijabla (manje NA vrednosti).

Izbor ciljne promenljive - Stream ili Views

Nakon univarijantne i bivarijantnih analiza, možemo da donesemo odluku o tome koju od dve kandidata za ciljnu promenljivu, **Stream ili Views**, ćemo koristiti kao ciljnu promenljivu u nastavku rada.

Obe promenljive dele iste osnovne osobine, što su izrazito desno asimetrična raspodela koja zahteva logaritamsku transformaciju i međusobno su umereno do jako korelisane.

Ono što je presudno u donošenju odluke jeste kako izgledaju mogući prediktori za moguće ciljne promenljive. Kod Views-a, dva ubedljivo najjača kandidata su Likes i Comments, koje u suštini mere istu stvar kao i sam Views. Isti problem, samo blaži, važi i za Licensed i Official_video, jer njihov efekat na Views je umeren i jači nego kod Stream-a, ali to nije stvarna prednost, jer obe promenljive opisuju status istog tog YouTube videa čiji broj pregleda pokušavamo da objasnimo, ne pesmu kao takvu. Kad se izuzmu ove tri kolone, ono što preostaje za Views je prilično slabo, i ne izdvaja se znatno u odnosu na streamove.

Kod Stream-a je situacija drugačija upravo zato što je Artist jače korelisani sa Stream-om nego sa Views-om. To nam otvara pitanje koje kod Views-a suštinski ne postoji, koliko YouTube osobine podataka dodaju objašnjenju Stream-a preko onoga što već nose audio karakteristike i identitet izvođača? Koliko Spotify strana dodaje objašnjenju Views-a, nema smisla postaviti na isti način, jer je Views već skoro u potpunosti zavistan unutar sopstvene

platforme. Audio karakteristike su slabo povezane i sa Stream-om, ali to svakako ne menja situaciju u poređenju sa Views-om.

Pored toga, skup podataka je pre svega organizovan oko pesama na Spotify-u, dok su YouTube podaci dodatak spojen preko postojanja zvaničnog spota. Stream je izvorna mera uspeha pesme na platformi oko koje je skup organizovan, dok je Views mera uspeha konkretnog video zapisa koji prati pesmu, a ne nužno i same pesme, pa taj video može biti lyrics video, remix, cover, ili uopšte ne postojati.

Na osnovu ovih argumenata, odlučili smo da koristimo **Stream kao ciljnu promenljivu**, dok ćemo Views koristiti kao prediktor u budućem radu.

Čišćenje podataka

Duplikati

Prvo ćemo proveriti da li postoje duplikati među redovima, tj. redovi kod kojih se sve kolone međusobno poklapaju.

```
data %>% filter(duplicated(.)) %>% nrow()

## [1] 0
```

Kao što možemo videti, nema u potpunosti identičnih redova u dataset-u. Prilikom univarijatne analize primetili smo da postoji više unosa istih pesama sa različitim autorima, što je sasvim validno ponašanje kako je moguće da postoje pesme sa više autora i ovakve redove nećemo uklanjati, kako su relevantni za analizu i predikciju. Međutim, neophodno je da proverimo da li postoje redovi gde nije reč o koautorskim pesmama, već je više puta unesena ista pesma sa istim autorom, tako što ćemo prebrojati broj pojavljivanja pesme i broj različitih autora.

```
duplicate_pesme = data %>%
  group_by(Track) %>%
  summarise(broj_razlicitih_izvodjaca = n_distinct(Artist),
    broj_pojavljivanja = n()) %>%
  filter(broj_razlicitih_izvodjaca > 1) %>%
  arrange(desc(broj_pojavljivanja))
duplicate_pesme

## # A tibble: 2,065 × 3
##   Track                                broj_razlicitih_izvo...1 broj_pojavljivanja
##   <chr>                                <int>                <int>
## 1 El Ultimo Adiós - Varios Artistas    24                    24
## 2 Color Esperanza 2020                  19                    19
## 3 Resistiré                             14                    14
## 4 52 Non Stop Dilbar Dilbar Remix(R.    9                     9
## 5 Heaven                                9                     9
## 6 Roll Me Up and Smoke Me When I D..    9                     9
```

```
## 7 Valentine's Mashup 2019          9          9
## 8 Y, ¿Si Fuera Ella? - + Es +      9          9
## 9 Angel                            7          7
## 10 Ay Haiti!                       7          7
## # i 2,055 more rows
## # i abbreviated name: ^broj_razlicitih_izvodjaca
```

Da bismo videli da li negde postoje problematične vrednosti, u smislu da je jedan autor naveden više puta za istu pesmu, proverićemo i ostale faktora osim naziva, kao što su album_type, kako se može dogoditi da je jedna pesma objavljena u više izdanja, na više albuma i slično.

```
data %>%
  filter(Track %in% duplicate_pesme$Track) %>%
  group_by(Artist, Track, Album, Album_type) %>%
  filter(n() > 1) %>%
  ungroup() %>%
  arrange(Artist, Track)

## # A tibble: 0 × 28
## # i 28 variables: X <int>, Artist <chr>, Url_spotify <chr>, Track <chr>,
## #   Album <chr>, Album_type <chr>, Uri <chr>, Danceability <dbl>, Energy
## #   <dbl>,
## #   Key <dbl>, Loudness <dbl>, Speechiness <dbl>, Acousticness <dbl>,
## #   Instrumentalness <dbl>, Liveness <dbl>, Valence <dbl>, Tempo <dbl>,
## #   Duration_ms <dbl>, Url_youtube <chr>, Title <chr>, Channel <chr>,
## #   Views <dbl>, Likes <dbl>, Comments <dbl>, Description <chr>,
## #   Licensed <chr>, Official_video <chr>, Stream <dbl>
```

Kao što možemo videti, nema pesama koje se ponavljaju više puta za više umetnika, već se pesme ponavljaju u različitim izdanjima, što nama neće uticati na modele.

Anomalije

Pored duplikata i nedostajućih vrednosti, neophodno je proveriti i da li skup sadrži vrednosti koje nisu nedostajuće (nisu NA), ali su nelogične u kontekstu same promenljive.

Likes, Views, Comments, Streams

U univariјantnoj analizi zaključili smo da je svaka od ovih kolona ima ekstremne vrednosti, kako su sve desno asimetrične, međutim do ovih vrednosti nije došlo greškom već zbog postojanja pesama koje imaju jako visoku metriku u odnosu na druge. Broj lajkova ili komentara ne može biti veći od broja pregleda videa, da bi neko reagovao na video mora ga prvo pogledati.

```
data %>%
  filter(!is.na(Url_youtube)) %>%
  filter(Likes > Views | Comments > Views) %>%
  select(Artist, Track, Views, Likes, Comments)
```

```
## [1] Artist    Track    Views    Likes    Comments
## <0 rows> (or 0-length row.names)
```

Takođe, ove kolone ne bi trebalo da imaju vrednosti koje su negativne.

```
data %>%
  filter(Views < 0 | Likes < 0 | Comments < 0 | Stream < 0) %>%
  select(Artist, Track, Views, Likes, Comments, Stream)

## [1] Artist    Track    Views    Likes    Comments Stream
## <0 rows> (or 0-length row.names)
```

Loudness

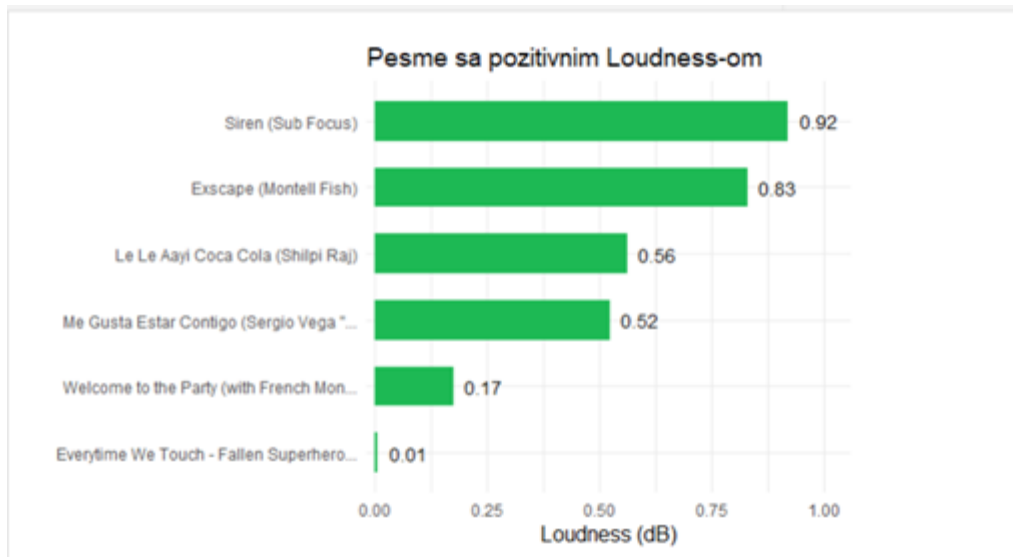
Loudness se meri u decibelima (dB) u odnosu na referentnu tačku od 0 dB, koja predstavlja maksimalnu moguću jačinu zvuka bez izobličenja, zbog čega se očekuje da je Loudness gotovo uvek negativan broj, s tim što pozitivne vrednosti nisu nemoguća pojava.

```
data %>% filter(Loudness > 0) %>% summarise(broj_pozitivnih_loudness = n())

##   broj_pozitivnih_loudness
## 1                        6
```

Obzirom da postoje pesme koje imaju ovakav Loudness, pogledaćemo koje su to pesme na barplot-u.

```
data %>%
  filter(Loudness > 0) %>%
  select(Artist, Track, Loudness) %>%
  mutate(Label = str_trunc(paste0(Track, " (", Artist, ")"), 40)) %>%
  mutate(Label = fct_reorder(Label, Loudness)) %>%
  ggplot(aes(x = Label, y = Loudness)) +
  geom_col(fill = "#1DB954", width = 0.6) +
  geom_text(aes(label = round(Loudness, 2)), hjust = -0.3, size = 4) +
  coord_flip() +
  scale_y_continuous(expand = expansion(mult = c(0.02, 0.15))) +
  theme_minimal(base_size = 13) +
  labs(title = "Pesme sa pozitivnim Loudness-om", x = "", y = "Loudness (dB)")
```



Kod nekih veoma masterizovanih numera loudness može da prelazi 0db, što ovde vidimo kod 6 pesama, kojima je loudness svakako ispod 1dB, pa ih zadržavamo.

Duration_ms

Neophodno je proveriti i da li postoje pesme koje su predugačke ili prekratke, odnosno imaju jako veliki ili jako mali duration. Pošto imamo više pojavljivanja jedne pesme u skupu, gledaćemo po jedan primerak te pesme u skupu.

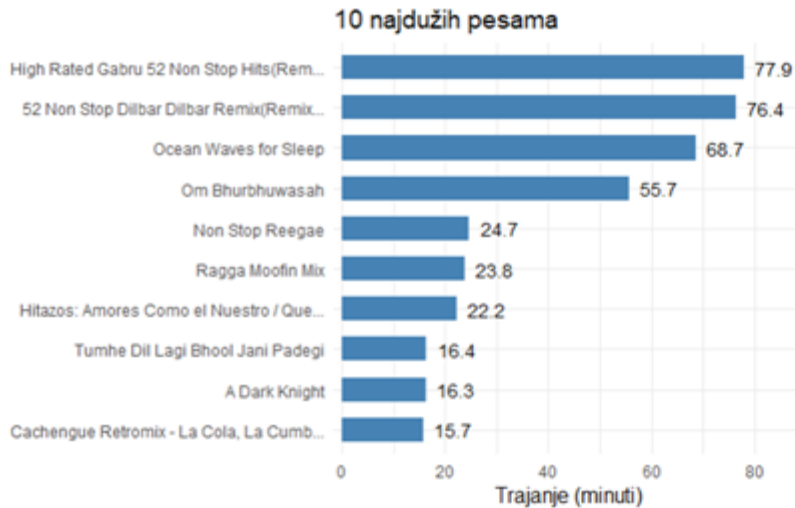
Najkraće pesme:

```
data %>%
  arrange(Duration_ms) %>%
  distinct(Track, .keep_all = TRUE) %>%
  select(Artist, Track, Duration_ms) %>%
  head(10) %>%
  mutate(Duration_min = Duration_ms / 60000) %>%
  mutate(Label = str_trunc(paste0(Track), 40)) %>%
  mutate(Label = fct_reorder(Label, Duration_min, .desc = TRUE)) %>%
  ggplot(aes(x = Label, y = Duration_min)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = round(Duration_min, 2)), hjust = -0.3, size = 4) +
  coord_flip() +
  scale_y_continuous(expand = expansion(mult = c(0.02, 0.15))) +
  theme_minimal(base_size = 13) +
  labs(title = "10 najkraćih pesama (jedinствене)", x = "", y = "Trajanje (minuti)")
```



Najduže pesme:

```
data %>%
  arrange(desc(Duration_ms)) %>%
  distinct(Track, .keep_all = TRUE) %>%
  select(Artist, Track, Duration_ms) %>%
  head(10) %>%
  mutate(Duration_min = Duration_ms / 60000) %>%
  mutate(Label = str_trunc(paste0(Track), 40)) %>%
  mutate(Label = fct_reorder(Label, Duration_min)) %>%
  ggplot(aes(x = Label, y = Duration_min)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = round(Duration_min, 1)), hjust = -0.3, size = 4) +
  coord_flip() +
  scale_y_continuous(expand = expansion(mult = c(0.02, 0.15))) +
  theme_minimal(base_size = 13) +
  labs(title = "10 najdužih pesama", x = "", y = "Trajanje (minuti)")
```



Najkraće pesme u skupu uglavnom predstavljaju kratke pesme za uspavljivanje beba, i traju oko pola minuta, što je na osnovu domenskog znanja sasvim moguće. Najduže pesme su zapravo kompilacije i miksevi, pa ima smisla da imaju veći duration.

Audio karakteristike

Kolone poput Danceability, Energy, Valence, Acousticness, Instrumentalness, Liveness, Speechiness po definiciji bi trebalo da budu između 0 i 1, pa ćemo proveriti da li postoje vrednosti izvan tog opsega. Kolona Key bi trebalo da bude u opsegu između -1 i 11.

```
data %>%
  select(Danceability, Energy, Valence, Acousticness, Instrumentalness,
  Liveness, Speechiness) %>%
  summarise(across(everything(), list(min = ~min(., na.rm = TRUE), max =
  ~max(., na.rm = TRUE)))) %>%
  pivot_longer(everything(), names_to = c("karakteristika", "statistika"),
  names_sep = "_(?=min|max)") %>%
  pivot_wider(names_from = statistika, values_from = value) %>%
  mutate(u_opsegu = min >= 0 & max <= 1)
```

```
## # A tibble: 7 × 4
##   karakteristika      min    max u_opsegu
##   <chr>          <dbl> <dbl> <lgl>
## 1 Danceability    0      0.975 TRUE
## 2 Energy          0.0000203 1     TRUE
## 3 Valence         0      0.993 TRUE
## 4 Acousticness    0.00000111 0.996 TRUE
## 5 Instrumentalness 0      1     TRUE
## 6 Liveness       0.0145    1     TRUE
## 7 Speechiness     0      0.964 TRUE
```

```
data %>%
  filter(Key < -1 | Key > 11) %>%
  select(Artist, Track, Key)

## [1] Artist Track Key
## <0 rows> (or 0-length row.names)
```

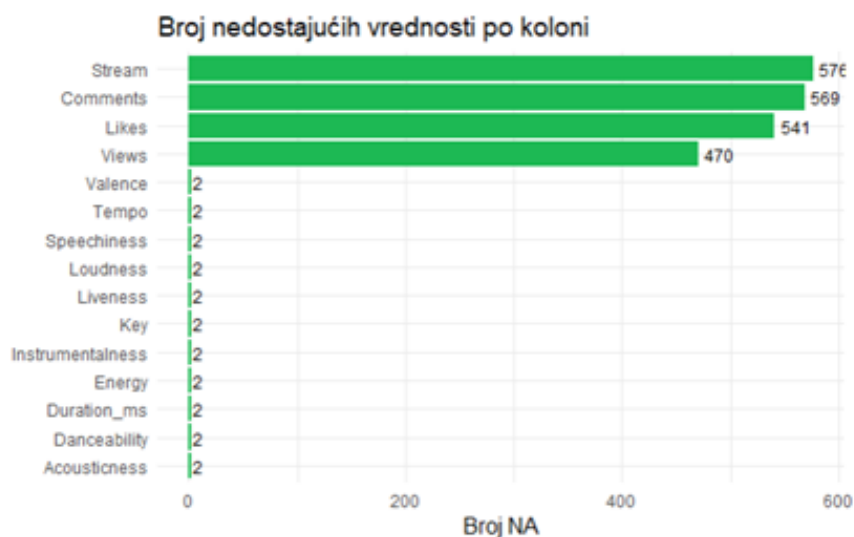
Sve audio karakteristike su u očekivanom opsegu.

Nedostajuće vrednosti

Da bismo uspešno kreirali modele, neophodno je i da rešimo problem nedostajućih vrednosti iz skupa, njihovim uklanjanjem ili imputacijom. Prvo ćemo proveriti koje kolone sadrže nedostajuće vrednosti.

```
na_summary = data %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  pivot_longer(cols = everything(), names_to = "Kolona", values_to =
"Broj_NA") %>%
  filter(Broj_NA > 0) %>%
  arrange(desc(Broj_NA))

ggplot(na_summary, aes(x = fct_reorder(Kolona, Broj_NA), y = Broj_NA)) +
  geom_col(fill = "#1DB954") +
  geom_text(aes(label = Broj_NA), hjust = -0.2, size = 3.5) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(title = "Broj nedostajućih vrednosti po koloni", x = "", y = "Broj
NA")
```



Ono što možemo videti je da imamo nedostajuće vrednosti u kolonama Stream, Comments, Likes, Views i po 2 nedostajuće vrednosti u svim kolonama vezanim za audio karakteristike.

Pored toga, trebalo bi da proverimo da li postoje tekstualne kolone koje imaju vrednost praznog stringa.

```
empty_string_columns = data %>%
  summarise(across(
    where(~ is.character(.) || is.factor(.)),
    ~ sum(trimws(as.character(.)) == "", na.rm = TRUE)
  )) %>%
  pivot_longer(cols = everything(), names_to = "kolona", values_to = "broj")
%>%
  filter(broj > 0) %>%
  arrange(desc(broj))

ggplot(empty_string_columns, aes(x = fct_reorder(kolona, broj), y = broj)) +
  geom_col(fill = "#1DB954") +
  geom_text(aes(label = broj), hjust = -0.2, size = 3.5) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(title = "Broj praznih stringova po koloni", x = "", y = "Broj praznih stringova")
```



Određene kolone vezane za Youtube kanal imaju prazan string u svojim vrednostima, što već sad možemo da povežemo sa tim da su to pesme koje jednostavno nemaju spot na Youtube-u. Da bismo kasnije lakše baratali ovih vrednostima, postavilićemo NA vrednosti umesto praznog stringa.

```
data = data %>%
  mutate(across(
```



```

where(~ is.character(.) || is.factor(.)),
~ ifelse(trimws(as.character(.)) == "", NA, as.character(.))
))

```

Audio karakteristike

Da bismo rešili problem nedostajućih vrednosti kod audio karakteristika, neophodno je da proverimo za koje pesme tačno nedostaju ove vrednosti i da procenimo koji je uzrok, tj. da li imamo MAR (Missing At Random), MCAR (Missing Completely At Random) ili MNAR (Missing Not At Random) nedostajuću vrednost.

```

data %>%
  filter(
    is.na(Danceability) | is.na(Energy) | is.na(Key) |
    is.na(Loudness) | is.na(Speechiness) |
    is.na(Acousticness) | is.na(Instrumentalness) |
    is.na(Liveness) | is.na(Valence) | is.na(Tempo) | is.na(Duration_ms)
  ) %>%
  select(Artist, Track, Album, Danceability, Energy, Loudness, Valence,
Stream, Views)

```

##	Artist	Track	Album	Danceability	
## 1	Natasha Bedingfield	These Words			
## 2	White Noise for Babies	Rain in the Early Morning			
##			Album	Danceability	
## 1			Unwritten	NA	
## 2	Soothing Rain for Background Sounds and Natural	White Noise		NA	
##	Energy	Loudness	Valence	Stream	Views
## 1	NA	NA	NA	110442210	21655597
## 2	NA	NA	NA	145339552	NA

Ono što možemo da primetimo je da postoje dve pesme kojima nedostaju sve audio karakteristike, ali jednoj od tih pesama su popunjene kolone Stream i Views, dok je drugoj popunjena samo kolona Stream. Kako u ovom slučaju, ove dve pesme imaju mnogo nedostajućih vrednosti, možemo da zaključimo da je nedostatak ove vrednosti verovatno nastao zbog prekida prilikom dohvaćanja podataka, a ne izazvan nečim drugim, kao na primer da za neku pesme nije merljiv Danceability, pa možemo da zaključimo da su ovo MCAR nedostajuće vrednosti. Pored toga, kako su u pitanju samo 2 reda, možemo da ih obrišemo, a da to ne utiče mnogo na naš dalji rad.

```

data = data %>% filter(!is.na(Danceability))

data %>%
  filter(
    is.na(Danceability) | is.na(Energy) | is.na(Key) |
    is.na(Loudness) | is.na(Speechiness) |
    is.na(Acousticness) | is.na(Instrumentalness) |
    is.na(Liveness) | is.na(Valence) | is.na(Tempo) | is.na(Duration_ms)
  )

```

```
) %>%
nrow()

## [1] 0
```

Nakon brisanja, ne postoje više kolone sa ovim problemima, pa možemo nastaviti dalje sa rešavanjem nedostajućih vrednosti.

```
na_summary = data %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(cols = everything(), names_to = "kolona", values_to =
"broj_NA") %>%
  filter(broj_NA > 0) %>%
  arrange(desc(broj_NA))

na_summary

## # A tibble: 10 × 2
##   kolona      broj_NA
##   <chr>      <int>
## 1 Description      989
## 2 Stream           576
## 3 Comments         568
## 4 Likes            540
## 5 Url_youtube      469
## 6 Title            469
## 7 Channel          469
## 8 Views            469
## 9 Licensed         469
## 10 Official_video  469
```

Views

Da bismo shvatili uzrok ovih nedostajućih vrednosti, proverićemo preklapanja, ali pored toga možemo da pretpostavimo, kako je naglašeno i ranije, da je uzrok ovih nedostajućih vrednosti verovatno nepostajanje Youtube spota za datu pesmu, na osnovu čega možemo zaključiti da su ovo podaci MNAR tipa.

```
yt_cols = c("Views", "Likes", "Comments", "Url_youtube", "Title", "Channel",
"Description", "Licensed", "Official_video")

data %>% filter(if_all(all_of(yt_cols), is.na)) %>% nrow()

## [1] 469
```

Nakon provere možemo da zaključimo da zaista postoji 469 redova gde nema nijednog podatka o Youtube videu.

Obzirom da su ovo podaci čija se prediktivna moć kasnije može koristiti, a nema smisla da popunjavamo sve ove kolone nekim vrednostima, dodaćemo indikatorsku kolonu koja će

ukazivati na to da li postoji Youtube video, što ćemo kasnije i moći da iskoristimo prilikom kreiranja nelinearnih modela, a u numeričke kolone upisaćemo 0. U kolone koje kategorijskog tipa, upisaćemo tekstualne vrednosti koje ukazuje na to da video ne postoji, kako ćemo svakako prilikom korišćenja ovakvih kolona koristiti i novu, **Has_youtube** kolonu.

```
data = data %>%
  mutate(Has_youtube = ifelse(is.na(Views), FALSE, TRUE))

data = data %>%
  mutate(
    Views = ifelse(!Has_youtube, 0, Views),
    Likes = ifelse(!Has_youtube, 0, Likes),
    Comments = ifelse(!Has_youtube, 0, Comments),
    Url_youtube = ifelse(!Has_youtube, "None", Url_youtube),
    Title = ifelse(!Has_youtube, "Unknown", Title),
    Channel = ifelse(!Has_youtube, "Unknown", Channel),
    Description = ifelse(!Has_youtube & is.na(Description), "No Description",
Description),
    Licensed = ifelse(!Has_youtube, "Unknown", as.character(Licensed)),
    Official_video = ifelse(!Has_youtube, "Unknown",
as.character(Official_video))
  )

data %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  pivot_longer(cols = everything(), names_to = "kolona", values_to =
"broj_NA") %>%
  filter(broj_NA > 0) %>%
  arrange(desc(broj_NA))

## # A tibble: 4 × 2
##   kolona      broj_NA
##   <chr>      <int>
## 1 Stream         576
## 2 Description    520
## 3 Comments       99
## 4 Likes         71
```

Nakon ovakvih transformacija sve kolone ,gde smo imali nedostajuće vrednosti zbog nedostatka Youtube spota za pesmu, su popunjene.

Description

Za opis videa ćemo samo popuniti NA vrednosti sa No Description indikatorom, kako svakako neće biti relevantna za kasnije kreiranje modela.

```
data = data %>%
  mutate(Description = ifelse(is.na(Description), "No Description",
Description))
```

Podela na trening i test skup

Pre nego što nastavimo sa popunjavanjem preostalih nedostajućih vrednosti i sa feature engineering-om, moramo prvo da podelimo podatke na trening i test skup. Razlog za ovo je sprečavanje curenja podataka, kako koraci koji slede računaju statistike preko grupa redova, odnosno po izvođaču, po opsegu Views-a i ako bismo te statistike računali na celom skupu pre podele, test skup bi delimično imao informaciju o samom sebi kroz ove statistike.

Prilikom podele na test i train, pobrinućemo se da se ne desi da se ista pesma pojavi i u test i u train skupu (kako postoje duplikati zbog koautorstva).

```
## izmenjeno - da se ne bi desilo da se ista pesma pojavi i u test i u train  
set.seed(123)
```

```
sve_pesme <- unique(data$Track)
```

```
test_pesme <- sample(sve_pesme, size = floor(0.2 * length(sve_pesme)))
```

```
test <- data[data$Track %in% test_pesme, ]
```

```
train <- data[!(data$Track %in% test_pesme), ]
```

```
nrow(test)
```

```
## [1] 4126
```

```
nrow(train)
```

```
## [1] 16590
```

Prebrojavanjem redova, utvrdili smo da su podaci uspešno podeljeni na test i train.

Likes i Comments

Da bismo popunili vrednost za Likes i Comments, prvo ćemo proveriti da li se ove nedostajuće vrednosti mahom javljaju na istim videima ili ne. Ovu i sve naredne provere radimo isključivo nad trening skupom.

```
train %>% filter(Has_youtube) %>%  
  summarise(  
    NA_Likes = sum(is.na(Likes)),  
    NA_Comments = sum(is.na(Comments)),  
    NA_Likes_i_Comments = sum(is.na(Likes) & is.na(Comments))  
  )
```

```
##   NA_Likes NA_Comments NA_Likes_i_Comments  
## 1         60          75                  17
```

Ove nedostajuće vrednosti se više javljaju na različitim yt videima, što se može desiti iz više razloga. Na primer, na Youtube videu kreator ima opciju da ne dozvoli da se komentari postavljaju, ili da sakrije broj lajkova za video, međutim, mi u ovom slučaju nemamo mogućnost da izvršimo proveru da li su komentari isključeni ili podaci jednostavno samo nisu prikupljeni. Ovo nije isti slučaj kao kod pesama koje nemaju Youtube video, kako smo tamo jasno iz nepostojanja URL-a i ostalih kolona mogli da zaključimo i nepostojanje Youtube videa, za šta ovde nemamo mogućnost. Pored toga, karakteristično je za videe sa puno pregleda da imaju puno komentara i lajkova, tako da ćemo popunjavanje ovih vrednosti izvršiti procenom vrednosti.

Umesto da unapred pretpostavimo koja metoda imputacije daje najtačnije rezultate, uzećemo da testiramo koja je metoda najbolja, tako što ćemo na delu trening podataka gde je prava vrednost poznata, vrednost sakriti, a zatim ćemo primeniti svaku metodu koja dolazi u obzir za predviđanje te vrednosti, a na kraju ćemo meriti grešku uz pomoć metrike MAE (Mean Absolute Error) u odnosu na pravu vrednost. Testiraćemo četiri kandidata - medijanu po grupama po Views-u, globalnu medijanu, linearnu regresiju na logaritamskoj transformaciji Views, i medijanu po izvođaču.

```
uporedi_metode_imputacije = function(data, target_col, seed = 123) {  
  set.seed(seed)  
  
  poznato = data %>% filter(Has_youtube, !is.na(.data[[target_col]]))  
  n_test = floor(0.2 * nrow(poznato))  
  test_idx = sample(seq_len(nrow(poznato)), n_test)  
  
  train_i = poznato[-test_idx, ]  
  test_i = poznato[test_idx, ]  
  actual = test_i[[target_col]]  
  
  granice = quantile(train_i$Views, probs = seq(0, 1, length.out = 11), na.rm  
= TRUE)  
  granice = unique(granice)  
  granice[1] = -Inf  
  granice[length(granice)] = Inf  
  medijane_grupa = train_i %>%  
    mutate(grupa = cut(Views, breaks = granice, labels = FALSE,  
include.lowest = TRUE)) %>%  
    group_by(grupa) %>%  
    summarise(medijana = median(.data[[target_col]]))  
  pred_decile = test_i %>%  
    mutate(grupa = cut(Views, breaks = granice, labels = FALSE,  
include.lowest = TRUE)) %>%  
    left_join(medijane_grupa, by = "grupa") %>%  
    pull(medijana)  
  
  pred_globalna = rep(median(train_i[[target_col]]), nrow(test_i))  
  
  formula_reg = as.formula(paste0("log1p(", target_col, ") ~ log1p(Views)"))
```

```

model_reg = lm(formula_reg, data = train_i)
pred_regresija = expm1(predict(model_reg, newdata = test_i))

medijana_izvodjac = train_i %>%
  group_by(Artist) %>%
  summarise(med_art = median(.data[[target_col]]))
pred_artist = test_i %>%
  left_join(medijana_izvodjac, by = "Artist") %>%
  mutate(pred = ifelse(is.na(med_art), median(train_i[[target_col]]),
med_art)) %>%
  pull(pred)

tibble(
  metod = c("Medijana po Views grupama", "Globalna medijana", "Regresija na
log(Views)", "Medijana po izvođaču"),
  MAE_log = c(
    mean(abs(log1p(pred_decile) - log1p(actual))),
    mean(abs(log1p(pred_globalna) - log1p(actual))),
    mean(abs(log1p(pred_regresija) - log1p(actual))),
    mean(abs(log1p(pred_artist) - log1p(actual)))
  )
) %>% arrange(MAE_log)
}

```

Nakon kreiranja funkcije za proveru najbolje metode, možemo da testiramo na kolonama.

```

uporedi_metode_imputacije(train, "Likes")

## # A tibble: 4 × 2
##   metod                MAE_log
##   <chr>                <dbl>
## 1 Regresija na log(Views)    0.492
## 2 Medijana po Views grupama  0.617
## 3 Medijana po izvođaču      1.61
## 4 Globalna medijana         1.95

uporedi_metode_imputacije(train, "Comments")

## # A tibble: 4 × 2
##   metod                MAE_log
##   <chr>                <dbl>
## 1 Regresija na log(Views)    0.807
## 2 Medijana po Views grupama  0.891
## 3 Medijana po izvođaču      1.72
## 4 Globalna medijana         2.15

```

Za obe kolone kao najbolja metoda pokazala se regresija na osnovu views, tako da ćemo koristiti tu metodu. Sada je bitno da model treniramo samo na trening skupu, a zatim isti model primenimo i na trening i na test skup.

```

fit_likes_comments_model = function(train_data, target_col) {
  poznato = train_data %>% filter(Has_youtube, !is.na(.data[[target_col]]))
  formula_reg = as.formula(paste0("log1p(", target_col, ") ~ log1p(Views)"))
  lm(formula_reg, data = poznato)
}

primeni_likes_comments_model = function(df, model, target_col) {
  idx = df$Has_youtube & is.na(df[[target_col]])
  if (any(idx)) {
    df[[target_col]][idx] = expm1(predict(model, newdata = df[idx, ]))
  }
  df
}

likes_model = fit_likes_comments_model(train, "Likes")
comments_model = fit_likes_comments_model(train, "Comments")

train = primeni_likes_comments_model(train, likes_model, "Likes")
train = primeni_likes_comments_model(train, comments_model, "Comments")
test = primeni_likes_comments_model(test, likes_model, "Likes")
test = primeni_likes_comments_model(test, comments_model, "Comments")

train %>% filter(Has_youtube) %>% summarise(NA_Likes = sum(is.na(Likes)),
NA_Comments = sum(is.na(Comments)))

##   NA_Likes NA_Comments
## 1         0           0

```

Nakon ovih procesa, popunili smo i sve redove za Likes i Comments za koje je postojao video, ali nije bilo vrednosti.

Stream

Za razliku od Likes i Comments, gde su nedostajuće vrednosti postojale isključivo kod redova sa YouTube videom, Stream je nezavisan od postojanja videa na YouTube-u. Kod redova bez videa, Views je ranije postavljen na 0, pa bi korišćenje Views kao prediktora tu bilo besmisleno. Zato možemo opet da uporedimo metode ali da podelimo podatke na grupe onih koje imaju i nemaju YouTube video. Pored toga je i dalje bitno da nastavimo da izračunavanja radimo na osnovu trening skupa, kako ne bi došlo do curenja podataka.

```

uporedi_metode = function(poznato, target_col, include_views = FALSE, seed =
123) {
  set.seed(seed)
  n_test = floor(0.2 * nrow(poznato))
  test_idx = sample(seq_len(nrow(poznato)), n_test)
  train_i = poznato[-test_idx, ]
  test_i = poznato[test_idx, ]
  actual = test_i[[target_col]]

```

```

rezultati = list()

rezultati[["Globalna medijana"]] = rep(median(train_i[[target_col]]),
nrow(test_i))

med_art = train_i %>% group_by(Artist) %>% summarise(m =
median(.data[[target_col]]))
rezultati[["Medijana po izvođaču"]] = test_i %>%
  left_join(med_art, by = "Artist") %>%
  mutate(p = ifelse(is.na(m), median(train_i[[target_col]]), m)) %>%
  pull(p)

if (include_views) {
  granice = quantile(train_i$Views, probs = seq(0, 1, length.out = 11),
na.rm = TRUE)
  granice = unique(granice)
  granice[1] = -Inf
  granice[length(granice)] = Inf
  med_dec = train_i %>%
    mutate(grupa = cut(Views, breaks = granice, labels = FALSE,
include.lowest = TRUE)) %>%
    group_by(grupa) %>% summarise(m = median(.data[[target_col]]))
  rezultati[["Medijana po Views"]] = test_i %>%
    mutate(grupa = cut(Views, breaks = granice, labels = FALSE,
include.lowest = TRUE)) %>%
    left_join(med_dec, by = "grupa") %>% pull(m)

  model = lm(as.formula(paste0("log1p(", target_col, ") ~ log1p(Views)")),
data = train_i)
  rezultati[["Regresija na log1p(Views)"]] = expm1(predict(model, newdata =
test_i))
}

tibble(
  metod = names(rezultati),
  MAE_log = sapply(rezultati, function(p) mean(abs(log1p(p) -
log1p(actual))))
) %>% arrange(MAE_log)
}

poznato_sa_yt = train %>% filter(Has_youtube, !is.na(Stream))
rezultati_stream = uporedi_metode(poznato_sa_yt, "Stream", include_views =
TRUE)
rezultati_stream

## # A tibble: 4 × 2
##   metod                MAE_log
##   <chr>                <dbl>
## 1 Medijana po izvođaču    0.974

```



```
## 2 Medijana po Views      0.994
## 3 Regresija na log1p(Views) 1.03
## 4 Globalna medijana      1.28

poznato_bez_yt = train %>% filter(!Has_youtube, !is.na(Stream))
rezultati_stream_bez_yt = uporedi_metode(poznato_bez_yt, "Stream",
include_views = FALSE)
rezultati_stream_bez_yt

## # A tibble: 2 × 2
##   metod      MAE_log
##   <chr>      <dbl>
## 1 Medijana po izvođaču  0.886
## 2 Globalna medijana    1.44
```

Za obe vrste podataka, najbolje je koristiti imputaciju na osnovu Artist-a, pa ćemo tako i popuniti podatke.

```
artist_median_stream_train = train %>%
  filter(!is.na(Stream)) %>%
  group_by(Artist, Has_youtube) %>%
  summarise(median_stream_artist = median(Stream), .groups = "drop")

globalna_medijana_yt_train = train %>% filter(Has_youtube, !is.na(Stream))
%>% summarise(m = median(Stream)) %>% pull(m)
globalna_medijana_bez_yt_train = train %>% filter(!Has_youtube,
!is.na(Stream)) %>% summarise(m = median(Stream)) %>% pull(m)

impute_stream = function(df) {
  df %>%
    left_join(artist_median_stream_train, by = c("Artist", "Has_youtube"))
  %>%
    mutate(
      globalna_medijana = ifelse(Has_youtube, globalna_medijana_yt_train,
globalna_medijana_bez_yt_train),
      Stream = case_when(
        !is.na(Stream) ~ Stream,
        !is.na(median_stream_artist) ~ median_stream_artist,
        TRUE ~ globalna_medijana
      )
    ) %>%
    select(-median_stream_artist, -globalna_medijana)
}

train = impute_stream(train)
test = impute_stream(test)
```

Nakon ovih imputacija, popunili smo i Streams kolonu i u trening i u test skupu, tako da smo popunili sve nedostajuće vrednosti, što bi trebalo da bude potvrđeno i sledećim kodom.

```

train %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(cols = everything(), names_to = "Kolona", values_to =
"Broj_NA") %>%
  filter(Broj_NA > 0) %>%
  arrange(desc(Broj_NA))

## # A tibble: 0 × 2
## #   i 2 variables: Kolona <chr>, Broj_NA <int>

test %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(cols = everything(), names_to = "Kolona", values_to =
"Broj_NA") %>%
  filter(Broj_NA > 0) %>%
  arrange(desc(Broj_NA))

## # A tibble: 0 × 2
## #   i 2 variables: Kolona <chr>, Broj_NA <int>

```

Čuvanje očišćenih podataka

Pre nego što nastavimo, sačuvaćemo skup podatke sada kada su podaci su u potpunosti očišćeni, ali još uvek sadrže sve originalne kolone, pre bilo kakvog enkodiranja ili kreiranja novih feature-a.

```

out_dir = "Podaci_za_model"
dir.create(out_dir, showWarnings = FALSE)

all_data = bind_rows(
  train %>% mutate(Split = "train"),
  test %>% mutate(Split = "test")
)

write_csv(all_data, file.path(out_dir, "SpotifyAndYoutubeClean.csv"))

```

Struktura podataka

U okviru ovog dela čišćenja podataka, pregledaćemo da li je negde došlo do neke greške u tipu podataka, tj da li su neki numerički podaci postali string i slično. Ovu i sve naredne transformacije primenjujemo identično i na trening i na test skup, kako bi obe tabele imale isti tip kolona.

```

str(train)

## 'data.frame':    16572 obs. of  29 variables:
## $ X              : int  18848 18896 2985 1841 3370 11637 4760 6745 16129
## $ Artist         : chr   "Ski Mask The Slump God" "NCT DREAM" "Chinmayi"
## $ ...            : chr   "Diddy" ...

```

```

## $ Url_spotify      : chr
"https://open.spotify.com/artist/2rhFzFmezpnW82MNqEKVry"
"https://open.spotify.com/artist/1gBUSTR3TyDdTVFIaQnc02"
"https://open.spotify.com/artist/5UJ2sH02ELrgW6aXeRLTQQ"
"https://open.spotify.com/artist/59wfkubONyhDMQGCljbUba" ...
## $ Track           : chr "Nuketown (feat. Juice WRLD)" "Glitch Mode"
"Main Rang Sharbaton Ka" "Bad Boy for Life" ...
## $ Album           : chr "STOKELEY" "Glitch Mode - The 2nd Album" "Phata
Poster Nikhla Hero (Original Motion Picture Soundtrack)" "The Saga
Continues..." ...
## $ Album_type      : chr "album" "album" "album" "album" ...
## $ Uri             : chr "spotify:track:74lnM5V6ecvoTPV0fvptx9"
"spotify:track:5b1PngLlxc7hj3fJXrE2Zm" "spotify:track:1yYM7dY6wSJFp5qmxPRLu1"
"spotify:track:2eOuL8Kess1TLQERQPu11D" ...
## $ Danceability    : num 0.808 0.634 0.631 0.669 0.737 0.751 0.714 0.598
0.502 0.323 ...
## $ Energy          : num 0.617 0.78 0.631 0.829 0.603 0.746 0.762 0.824
0.845 0.0114 ...
## $ Key             : num 10 0 6 1 10 2 5 11 0 4 ...
## $ Loudness        : num -9.32 -2.32 -6.93 -3.8 -7.49 ...
## $ Speechiness     : num 0.442 0.0963 0.0307 0.49 0.239 0.292 0.0429
0.131 0.138 0.0343 ...
## $ Acousticness    : num 0.113 0.0164 0.693 0.179 0.0606 0.188 0.341
0.00609 0.411 0.893 ...
## $ Instrumentalness: num 0.00 0.00 0.00 0.00 6.08e-06 0.00 5.88e-05 0.00
0.00 6.52e-01 ...
## $ Liveness        : num 0.734 0.141 0.133 0.241 0.118 0.0503 0.0468
0.0975 0.784 0.171 ...
## $ Valence         : num 0.632 0.694 0.697 0.61 0.492 0.889 0.963 0.416
0.649 0.134 ...
## $ Tempo           : num 150 167 90 119 76 ...
## $ Duration_ms     : num 166400 207053 263152 253067 281800 ...
## $ Url_youtube     : chr "https://www.youtube.com/watch?v=LjkDor2LenI"
"https://www.youtube.com/watch?v=oZP2h3WIZqk"
"https://www.youtube.com/watch?v=GnzZyGQi2ps"
"https://www.youtube.com/watch?v=GbJO59nWQRU" ...
## $ Title           : chr "Ski Mask The Slump God - Nuketown ft. Juice
WRLD (Directed by Cole Bennett)" "NCT DREAM 엔시티 드림 '버퍼링 (Glitch Mode)'
MV" "Main Rang Sharbaton Ka - Phata Poster Nikhla Hero I Shahid,Ileana | Atif
Aslam & Chinmayi | Pritam" "P. Diddy [feat. Black Rob & Mark Curry] - Bad Boy
4 Life (Official Music Video)" ...
## $ Channel         : chr "Lyrical Lemonade" "SMTOWN" "Tips Official" "Bad
Boy Entertainment" ...
## $ Views           : num 66031808 80870010 87878430 5174815 48682 ...
## $ Likes           : num 1216983 1847160 478253 67542 461 ...
## $ Comments        : num 63108 260399 12162 3536 20 ...
## $ Description     : chr "Lyrical Lemonade Presents:\n\nSki Mask The
Slump God - Nuketown [ft. Juice WRLD] (Official Music Video)\n\nIn L"|
__truncated__ "NCT DREAM's 2nd album \"Glitch Mode\" is out!\nListen and

```

```

download on your favorite platform: https://smarturl."| __truncated__ "Watch
Shahid Kapoor & Ileana D'Cruz in the song 'Main Rang Sharbaton Ka' from the
movie 'Phata Poster Nikhla He"| __truncated__ "Official Music Video for P.
Diddy [feat. Black Rob & Mark Curry] - \"Bad Boy 4 Life\" directed by Chris
Robinso"| __truncated__ ...
## $ Licensed      : chr  "False" "True" "True" "True" ...
## $ Official_video : chr  "False" "True" "True" "True" ...
## $ Stream        : num  3.70e+08 3.27e+07 4.17e+07 1.02e+08 6.17e+04 ...
## $ Has_youtube    : logi  TRUE TRUE TRUE TRUE TRUE TRUE ...

```

Na osnovu strukture podataka, trebalo bi da promenimo je Licensed i Official_video kolone, nad kojima bi trebalo izvršiti faktorizaciju, kako smo dodali novu vrednost za pesme koje nemaju video. Takođe, kako kolona Album_type ima fiksiran skup vrednosti, možemo da odradimo faktorizaciju i nad tom kolonom.

```

faktorizuj= function(df) {
  df %>% mutate(
    Licensed = factor(Licensed, levels = c("True", "False", "Unknown")),
    Official_video = factor(Official_video, levels = c("True", "False",
"Unknown")),
    Album_type = factor(Album_type)
  )
}

train = faktorizuj(train)
test = faktorizuj(test)

str(train)

## 'data.frame':    16572 obs. of  29 variables:
## $ X              : int  18848 18896 2985 1841 3370 11637 4760 6745 16129
2756 ...
## $ Artist         : chr  "Ski Mask The Slump God" "NCT DREAM" "Chinmayi"
"Diddy" ...
## $ Url_spotify     : chr
"https://open.spotify.com/artist/2rhFzFmezpnW82MNqEKVry"
"https://open.spotify.com/artist/1gBUSTR3TyDdTVFIaQnc02"
"https://open.spotify.com/artist/5UJ2sH02ELrgW6aXeRLTQQ"
"https://open.spotify.com/artist/59wfkuBoNyhDMQGCljbUBA" ...
## $ Track          : chr  "Nuketown (feat. Juice WRLD)" "Glitch Mode"
"Main Rang Sharbaton Ka" "Bad Boy for Life" ...
## $ Album          : chr  "STOKELEY" "Glitch Mode - The 2nd Album" "Phata
Poster Nikhla Hero (Original Motion Picture Soundtrack)" "The Saga
Continues..." ...
## $ Album_type     : Factor w/ 3 levels "album","compilation",...: 1 1 1 1
1 3 1 1 3 1 ...
## $ Uri            : chr  "spotify:track:74lnM5V6ecvoTPV0fvptx9"
"spotify:track:5b1PngLlxc7hj3fJXrE2Zm" "spotify:track:1yYM7dY6wSJFp5qmxPRLu1"
"spotify:track:2eOuL8KesslTLQERQPu11D" ...
## $ Danceability   : num  0.808 0.634 0.631 0.669 0.737 0.751 0.714 0.598

```

```

0.502 0.323 ...
## $ Energy          : num  0.617 0.78 0.631 0.829 0.603 0.746 0.762 0.824
0.845 0.0114 ...
## $ Key             : num  10 0 6 1 10 2 5 11 0 4 ...
## $ Loudness        : num  -9.32 -2.32 -6.93 -3.8 -7.49 ...
## $ Speechiness     : num  0.442 0.0963 0.0307 0.49 0.239 0.292 0.0429
0.131 0.138 0.0343 ...
## $ Acousticness    : num  0.113 0.0164 0.693 0.179 0.0606 0.188 0.341
0.00609 0.411 0.893 ...
## $ Instrumentalness: num  0.00 0.00 0.00 0.00 6.08e-06 0.00 5.88e-05 0.00
0.00 6.52e-01 ...
## $ Liveness        : num  0.734 0.141 0.133 0.241 0.118 0.0503 0.0468
0.0975 0.784 0.171 ...
## $ Valence         : num  0.632 0.694 0.697 0.61 0.492 0.889 0.963 0.416
0.649 0.134 ...
## $ Tempo           : num  150 167 90 119 76 ...
## $ Duration_ms     : num  166400 207053 263152 253067 281800 ...
## $ Url_youtube     : chr   "https://www.youtube.com/watch?v=LjkDor2LenI"
"https://www.youtube.com/watch?v=oZP2h3WIZqk"
"https://www.youtube.com/watch?v=GnzZyGQi2ps"
"https://www.youtube.com/watch?v=GbJO59nWQRU" ...
## $ Title           : chr   "Ski Mask The Slump God - Nuketown ft. Juice
WRLD (Directed by Cole Bennett)" "NCT DREAM 엔시티 드림 '버퍼링 (Glitch Mode)'
MV" "Main Rang Sharbaton Ka - Phata Poster Nikhla Hero I Shahid,Ileana | Atif
Aslam & Chinmayi | Pritam" "P. Diddy [feat. Black Rob & Mark Curry] - Bad Boy
4 Life (Official Music Video)" ...
## $ Channel         : chr   "Lyrical Lemonade" "SMTOWN" "Tips Official" "Bad
Boy Entertainment" ...
## $ Views           : num  66031808 80870010 87878430 5174815 48682 ...
## $ Likes           : num  1216983 1847160 478253 67542 461 ...
## $ Comments        : num  63108 260399 12162 3536 20 ...
## $ Description     : chr   "Lyrical Lemonade Presents:\n\nSki Mask The
Slump God - Nuketown [ft. Juice WRLD] (Official Music Video)\n\nIn L"|
__truncated__ "NCT DREAM's 2nd album \"Glitch Mode\" is out!\nListen and
download on your favorite platform: https://smarturl."| __truncated__ "Watch
Shahid Kapoor & Ileana D'Cruz in the song 'Main Rang Sharbaton Ka' from the
movie 'Phata Poster Nikhla He"| __truncated__ "Official Music Video for P.
Diddy [feat. Black Rob & Mark Curry] - \"Bad Boy 4 Life\" directed by Chris
Robinso"| __truncated__ ...
## $ Licensed        : Factor w/ 3 levels "True","False",...: 2 1 1 1 1 1 2 1
3 1 ...
## $ Official_video  : Factor w/ 3 levels "True","False",...: 2 1 1 1 1 1 2 1
3 1 ...
## $ Stream          : num  3.70e+08 3.27e+07 4.17e+07 1.02e+08 6.17e+04 ...
## $ Has_youtube     : logi   TRUE TRUE TRUE TRUE TRUE TRUE TRUE ...

```

Nakon ovih promena, strukturalna šema podataka izgleda u redu.

Feature engineering

Artist

Iz bivarijantne analize i generalnog domenskog znanja znamo da Artist ima uticaj na Stream, ali kolona ima previše vrednosti da bi mogla direktno da se koristi u modelu bez enkodiranja, ali nam bila jako značajan i zanimljiv prediktor za sagledavanje.

Prva pomisao je da iskoristimo prosečan Stream po izvođaču kao numerički podatak, međutim ovaj pristup nosi rizik od curenja podataka, jer ako u prosek uključimo i Stream same pesme, ta pesma ima jasan podatak o sopstvenoj ciljnoj vrednosti.

Imamo mogućnost i da iskoristimo frequency encoding, ali kako iz univarijantne analize znamo da skoro svaki izvođač ima tačno 10 pesama u skupu, ova kolona ne bi imala smisla jer bi za većinu pesama imala istu vrednost.

Zbog toga se opredeljujemo za **leave-one-out (LOO) median encoding**, pri čemu za svaku pesmu računamo medijanu stream-ova *ostalih* pesama istog izvođača, bez trenutne pesme. Uzimamo medijanu umesto srednje vrednosti jer je Stream izraženo desno asimetričan, pa bi srednja vrednost pokupila dosta uticaja od najpopularnijih pesama umetnika.

Sam princip izostavljanja jedne pesme rešava curenje na nivou pojedinačnog reda, ali ne i na nivou trening i test granice, ako bismo LOO medijanu računali na celom skupu, Stream test pesama bi i dalje ulazio u izračunavanje feature-a za trening pesme istog izvođača (i obrnuto), jer LOO samo izuzima trenutni red, ne i sve redove van trening skupa. Zato LOO medijanu računamo isključivo unutar trening skupa, dok test skup dobija medijanu Stream-ova tog izvođača, računat isključivo na trening podacima.

Pored osnovne logike imputacije, uveden je i fallback mehanizam, pa ukoliko za dati red ne postoji odgovarajuća medijana, koristi se globalna medijana izračunata na train skupu. Ovo obezbeđuje da nijedan red ne ostane bez vrednosti. Ovo naravno donosi i određene probleme, kako se može dogoditi da Artist ima samo jednu pesmu i da se nalazi u test skupu, ili da ima više pesama ali nijednu u trening skupu, međutim, trenutno nemamo način da razrešimo ovaj problem, jer je ovo ograničenje target encoding-a.

```
loo_medijana = function(x) {  
  n = length(x)  
  if (n <= 1) return(rep(NA_real_, n))  
  sapply(seq_along(x), function(i) median(x[-i], na.rm = TRUE))  
}  
  
train = train %>%  
  group_by(Artist) %>%  
  mutate(Artist_LOO_stream = loo_medijana(Stream)) %>%  
  ungroup()
```

```

globalna_medijana_train_stream = median(train$Stream, na.rm = TRUE)

train = train %>%
  mutate(Artist_LOO_stream = ifelse(is.na(Artist_LOO_stream),
    globalna_medijana_train_stream, Artist_LOO_stream))

artist_full_median_train = train %>%
  group_by(Artist) %>%
  summarise(Artist_median_stream = median(Stream, na.rm = TRUE), .groups = "drop")

test = test %>%
  left_join(artist_full_median_train, by = "Artist") %>%
  mutate(Artist_LOO_stream = ifelse(is.na(Artist_median_stream),
    globalna_medijana_train_stream, Artist_median_stream)) %>%
  select(-Artist_median_stream)

sum(is.na(train$Artist_LOO_stream))

## [1] 0

sum(is.na(test$Artist_LOO_stream))

## [1] 0

cor.test(train$Artist_LOO_stream, train$Stream, method = "spearman")

## Warning in cor.test.default(train$Artist_LOO_stream, train$Stream, method =
## "spearman"): Cannot compute exact p-value with ties

##
## Spearman's rank correlation rho
##
## data: train$Artist_LOO_stream and train$Stream
## S = 2.9143e+11, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## 0.6158035

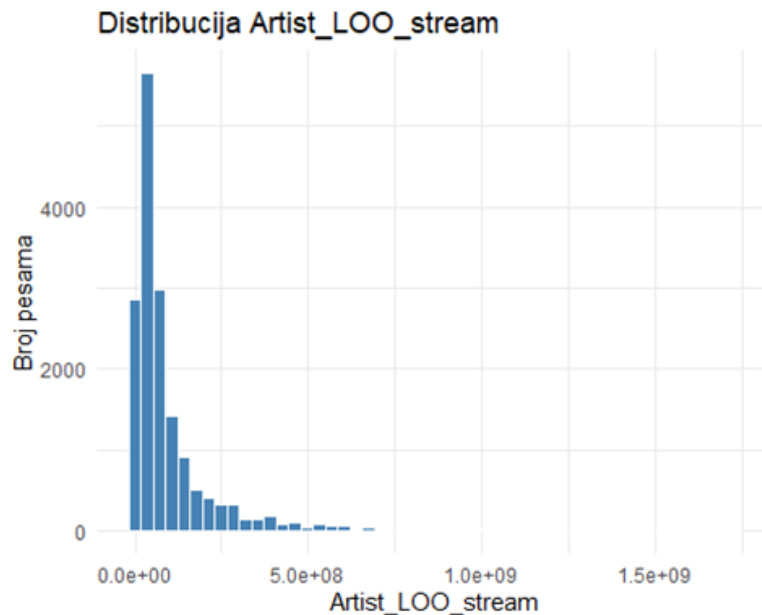
```

Nakon kreiranja kolone, pogledaćemo i njemu raspodelu.

```

ggplot(train, aes(x = Artist_LOO_stream)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
  labs(title = "Distribucija Artist_LOO_stream", x = "Artist_LOO_stream", y = "Broj pesama")
+
  theme_minimal()

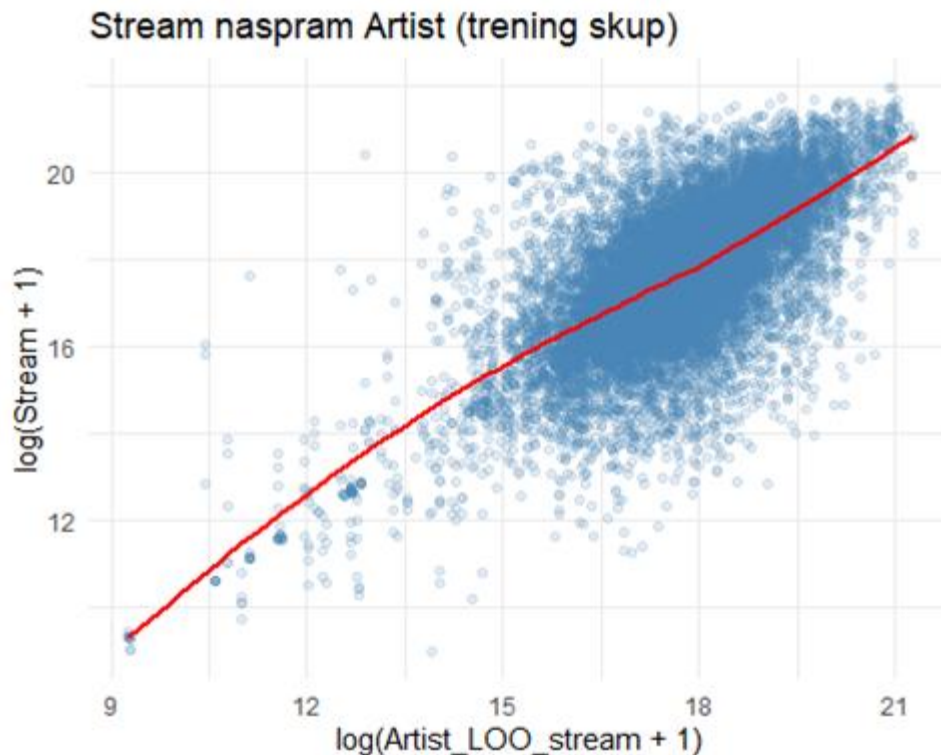
```



Kako smo i očekivali i Artist_LOO_stream je desno asimetrična, kako je i nastala od Stream-a koji je sam po sebi desno asimetričan, tako da je i za ovu kolonu neophodno koristiti logaritamsku transformaciju kada je koristimo u linearnim modelima.

Spearmanova korelacija, na trening skupu, potvrđuje umerenu do jaku pozitivnu vezu između dodate kolone i Stream-a, što znači da je ova kolona definitivno dobar kandidat za prediktor. Prikazaćemo ovu povezanost i u okviru scatter plot-a, pri čemu ćemo za Artist_LOO kao i za Stream koristiti logaritamsku transformaciju.

```
train %>%
  ggplot(aes(x = log1p(Artist_LOO_stream), y = log1p(Stream))) +
  geom_point(alpha = 0.15, color = "steelblue") +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  theme_minimal() +
  labs(title = "Stream naspram Artist (trening skup)", x = "log(Artist_LOO_stream + 1)", y =
"log(Stream + 1)")
## `geom_smooth()` using formula = 'y ~ x'
```

Grafik potvrđuje umerenu do jaku pozitivnu vezu između Artist_LOO_stream i Stream-a, uočenu i Spearman testom. Veza nije potpuno linearna, jer je kriva blago konkavna. Primetno je i znatno rasipanje tačaka oko krive, posebno u srednjem delu opsega, što pokazuje da istorijska popularnost izvođača objašnjava samo deo varijacije u broju stream-ova pojedinačne pesme.

Uprkos svim nedostacima koje ovakav način enkodiranja kategorije ima, ovakav način je jedini da nekako pretvorimo izvođača u mogući prediktor, jer je label encoding nepovoljan za linearnu regresiju, a one-hot encoding bi doveo do uvođenja previše kolona. U ovoj situaciji ćemo ovu kolonu koristiti kao prediktor, posebno jer je i iz domenskog znanja možemo da zaključimo da je ovo faktor koji će veoma uticati na Stream-ove, a to su pokazali i testovi, za razliku od ostatka kvantifikovanih podataka vezanih za samu pesmu, koji za sada nisu pokazali idealne rezultate.

Engagement rate (Likes, Comments)

Views, Likes i Comments su međusobno jako korelisani, pa to može izazvati problem multikolinearnosti. Views ćemo svakako zadržati kao glavnog pokazatelja YouTube popularnosti, dok od Likes i Comments možemo da napravimo nove feature-e, koji će pokazivati koliko je publika angažovana prilikom gledanja videa.

```
dodaj_engagement_rate = function(df) {
  df %>%
    mutate(
      Likes_rate = ifelse(Has_youtube & Views > 0, Likes / Views, 0),
```

```

    Comments_rate = ifelse(Has_youtube & Views > 0, Comments / Views, 0)
  )
}

```

```

train = dodaj_engagement_rate(train)
test = dodaj_engagement_rate(test)

```

```
summary(train$Likes_rate)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.000000 0.005533 0.008590 0.011973 0.014776 0.249204
```

```
summary(train$Comments_rate)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.0010418 0.0001234 0.0002340 0.0004488 0.0004559 0.0457184
```

Za pesme bez YouTube videa, Likes_rate i Comments_rate postavljamo na 0, u skladu sa ranije usvojenom logikom za same izvorne kolone.

```
cor.test(train$Likes_rate, train$Stream, method = "spearman")
```

```
## Warning in cor.test.default(train$Likes_rate, train$Stream, method =
## "spearman"): Cannot compute exact p-value with ties
```

```
##
## Spearman's rank correlation rho
##
## data: train$Likes_rate and train$Stream
## S = 9.0148e+11, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## -0.1884527
```

```
cor.test(train$Comments_rate, train$Stream, method = "spearman")
```

```
## Warning in cor.test.default(train$Comments_rate, train$Stream, method =
## "spearman"): Cannot compute exact p-value with ties
```

```
##
## Spearman's rank correlation rho
##
## data: train$Comments_rate and train$Stream
## S = 8.5833e+11, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## -0.1315684
```

Ove kolone pokazuju znatno manju korelaciju sa Stream-ovima od njihovih izvornih verzija, ali se svakako kasnije u nelinearnim modelima mogu pokazati kao korisni prediktori. Svakako je bitno i proveriti kolika je korelacija ovih kolona sa kolonom Views.

```
cor.test(train$Likes_rate, train$Views, method = "spearman")

## Warning in cor.test.default(train$Likes_rate, train$Views, method =
## "spearman"): Cannot compute exact p-value with ties

##
## Spearman's rank correlation rho
##
## data: train$Likes_rate and train$Views
## S = 1.0464e+12, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## -0.3794941

cor.test(train$Comments_rate, train$Views, method = "spearman")

## Warning in cor.test.default(train$Comments_rate, train$Views, method =
## "spearman"): Cannot compute exact p-value with ties

##
## Spearman's rank correlation rho
##
## data: train$Comments_rate and train$Views
## S = 9.2651e+11, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## -0.2214538
```

Korelacija sa Views-om je daleko manja, tako da se ove kolone, ukoliko bude bilo potrebe za tim, takođe mogu koristiti kao prediktori, bez rizika od kolinearnosti sa ovom kolonom.

PCA (Views, Likes, Comments)

Umesto izbacivanja Likes i Comments kolona u linearnoj regresiji, možemo probati i **PCA (Principal Component Analysis)** nad ove tri kolone, kako bismo probali da iskoristimo svu njihovu informaciju bez uvođenja multikolinearnosti u linearne modele. PCA je statistička metoda koja se koristi za smanjenje broja promenljivih u velikim skupovima podataka, uz maksimalno moguće očuvanje informacija i varijanse.

Pošto su Views, Likes i Comments izrazito desno asimetrični, prvo ćemo odraditi logaritamsku transformaciju, a zatim standardizovati podatke, jer je PCA osetljiva na razlike u varijansi između promenljivih.

Fitovanje PCA-a radimo samo nad pesmama koje imaju Youtube spot.

```

pca_cols_train = train %>%
  filter(Has_youtube) %>%
  transmute(
    log_Views = log1p(Views),
    log_Likes = log1p(Likes),
    log_Comments = log1p(Comments)
  )

pca_model = prcomp(pca_cols_train, center = TRUE, scale. = TRUE)

summary(pca_model)

## Importance of components:
##              PC1      PC2      PC3
## Standard deviation  1.6877 0.35546 0.15926
## Proportion of Variance 0.9494 0.04212 0.00846
## Cumulative Proportion 0.9494 0.99154 1.00000

```

Pogledaćemo i opterećenja, kako bismo razumeli šta svaka komponenta predstavlja.

```

pca_model$rotation

##              PC1      PC2      PC3
## log_Views   -0.5788803 -0.5328072 -0.6172634
## log_Likes   -0.5856019 -0.2551234  0.7694040
## log_Comments -0.5674223  0.8068634 -0.1643270

```

Sada primenjujemo istu PCA transformaciju i na trening i na test skup, uključujući i pesme bez YouTube spota, jer će za njih sve tri komponente ispasti identične, jer su ulazne vrednosti samo nule.

```

primeni_pca = function(df, pca_model) {
  ulaz = df %>%
    transmute(
      log_Views = log1p(Views),
      log_Likes = log1p(Likes),
      log_Comments = log1p(Comments)
    )

  scores = predict(pca_model, newdata = ulaz)

  df %>%
    mutate(
      PC1_popularnost = scores[, 1],
      PC2_popularnost = scores[, 2],
      PC3_popularnost = scores[, 3]
    )
}

```

```
train = primeni_pca(train, pca_model)
test = primeni_pca(test, pca_model)
```

Proverićemo korelaciju svake komponente sa Stream-om, kako bismo videli da li PC2 i PC3 nose dodatni, nezavisan signal iznad onoga što PC1, koja predstavlja opštu YouTube popularnost, već objašnjava.

```
cor.test(train$PC1_popularnost, train$Stream, method = "spearman")

##
## Spearman's rank correlation rho
##
## data: train$PC1_popularnost and train$Stream
## S = 1.1952e+12, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## -0.5756337

cor.test(train$PC2_popularnost, train$Stream, method = "spearman")

## Warning in cor.test.default(train$PC2_popularnost, train$Stream, method =
## "spearman"): Cannot compute exact p-value with ties

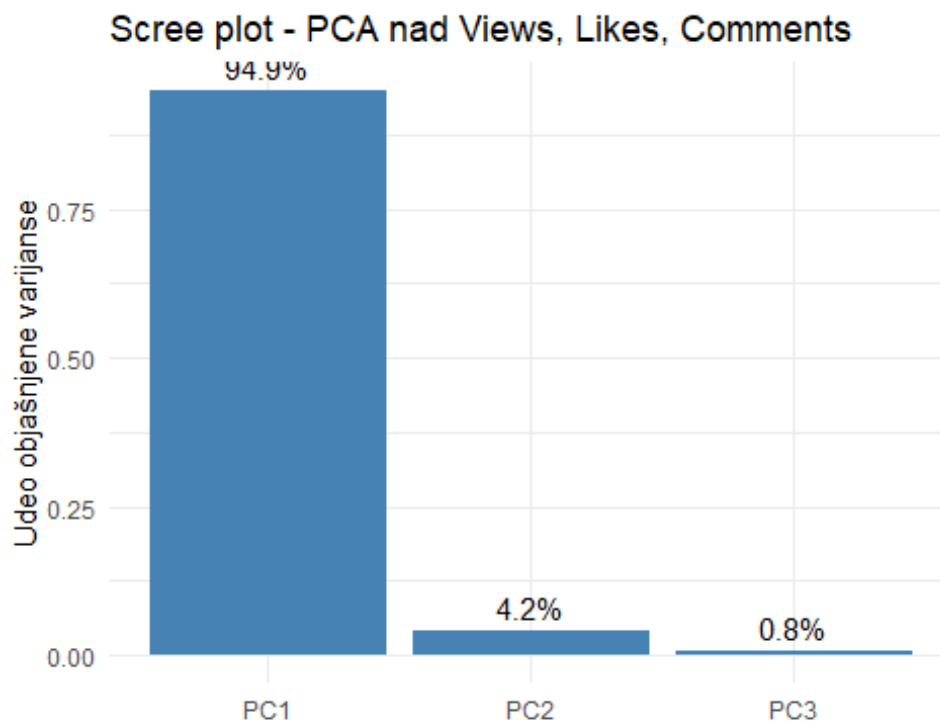
##
## Spearman's rank correlation rho
##
## data: train$PC2_popularnost and train$Stream
## S = 8.5037e+11, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## -0.1210795

cor.test(train$PC3_popularnost, train$Stream, method = "spearman")

##
## Spearman's rank correlation rho
##
## data: train$PC3_popularnost and train$Stream
## S = 7.693e+11, p-value = 0.06763
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## -0.01419637
```

Kako bismo dobili i vizuelni uvid u to koliko varijanse svaka komponenta objašnjava, iscrtaćemo scree plot, na kome se prikazuje udeo varijanse koju objašnjavaju kreirane komponente.

```
tibble(
  PC = factor(paste0("PC", 1:3), levels = paste0("PC", 1:3)),
  Varijansa = summary(pca_model)$importance[2, ]
) %>%
  ggplot(aes(x = PC, y = Varijansa)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = scales::percent(Varijansa, accuracy = 0.1)), vjust =
-0.5) +
  theme_minimal() +
  labs(title = "Scree plot - PCA nad Views, Likes, Comments", x = "", y =
"Udeo objašnjene varijanse")
```



Rezultati pokazuju da je PC1 skoro identičan po informativnoj vrednosti samom Views-u, što je i očekivano s obzirom na to da PC1 objašnjava ubedljivo najveći deo varijanse i predstavlja neku vrstu opšte YouTube popularnosti. PC2 nosi znatno slabiji, ali statistički značajan dodatni signal, dok je sa PC3 korelacija zanemarljiva.

Poređenja radi, PC2 ne donosi jači signal od Likes_rate i Comments_rate koje smo već ranije konstruisali, a ove kolone nose sličnu ideju, ali na direktniji i lakše interpretabilan način.

Na osnovu ovoga možemo da zaključimo da nam PCA neće doneti merljivu prednost u odnosu na pristupe s kojima smo se već susreli, odnosno korišćenje Likes_rate, Comments_rate i Views. Iako bi PCA komponente bile korisne u linearnim modelima zbog savršene ortogonalnosti, gubitak direktne interpretabilnosti nije opravdan obzirom na to

neki već kreirani feature-i nose isti ili jači signal uz razumljivo značenje. Zbog toga PCA komponente nećemo uvrstiti u finalni skup prediktora.

```
train = train %>% select(-PC1_popularnost, -PC2_popularnost, -PC3_popularnost)
test = test %>% select(-PC1_popularnost, -PC2_popularnost, -PC3_popularnost)
```

Mood kvadrant (Valence x Energy)

Nijedna audio karakteristika pojedinačno ne prelazi korelaciju od |0.14| sa Stream-om, što nam već ukazuje da to kako pesma zvuči ima slab uticaj na njenu popularnost. Međutim, umesto da posmatramo karakteristike izolovano, pokušaćemo da ih kombinujemo na način koji ima muzički smisao, za slučaj da kombinacija nosi jači signal od pojedinačnih kolona. Valence i Energy su dve karakteristike koje se koriste za opisivanje muzičkog raspoloženja. Energy predstavlja koliko pesma deluje mirno ili intenzivno, dok Valence opisuje koliko deluje pozitivno ili negativno. Njihovim kombinovanjem mogu se formirati četiri kvadranta raspoloženja: visoka vrednost Energy i visoka Valence odgovaraju srećnom i uzbudljivom raspoloženju, visoka Energy i niska Valence napetom ili ljutom, niska Energy i visoka Valence mirnom i zadovoljavajućem, a niska Energy i niska Valence tužnom raspoloženju.

Izvor: *Mood and Emotion Analysis* - ²

```
add_mood_quadrant = function(df) {
  df %>%
    mutate(
      Mood_quadrant = case_when(
        Valence >= 0.5 & Energy >= 0.5 ~ "Happy/Energetic",
        Valence < 0.5 & Energy >= 0.5 ~ "Anger/Unpleasant",
        Valence >= 0.5 & Energy < 0.5 ~ "Calm/Relaxing",
        Valence < 0.5 & Energy < 0.5 ~ "Sad/Melancholic"
      ),
      Mood_quadrant = factor(Mood_quadrant)
    )
}

train = add_mood_quadrant(train)
test = add_mood_quadrant(test)

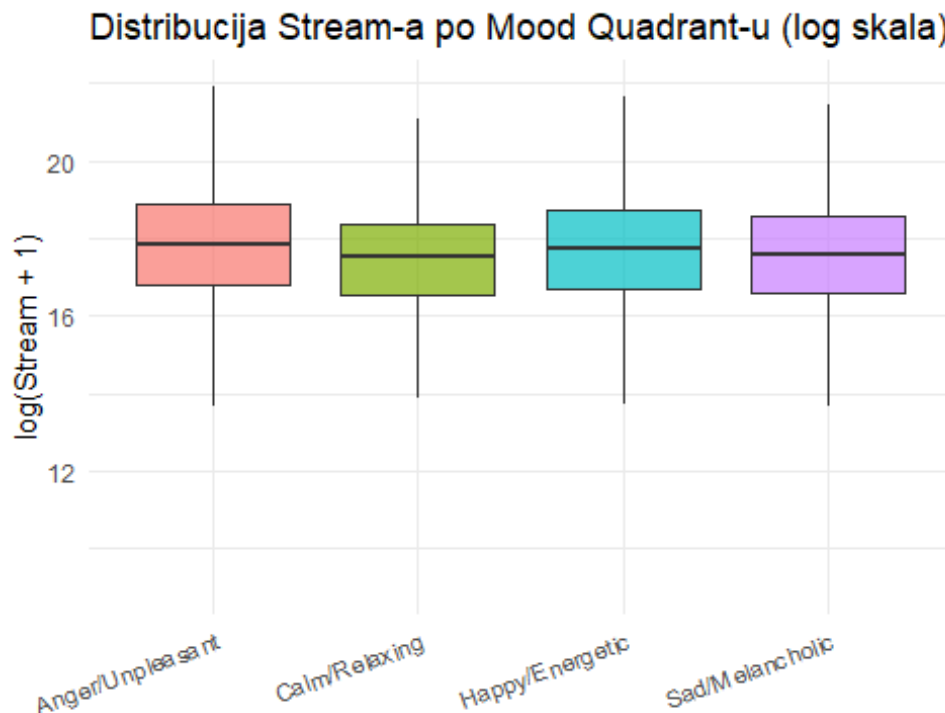
kruskal_effsize(train, Stream ~ Mood_quadrant)

## # A tibble: 1 × 5
##   .y.      n effsize method magnitude
## * <chr> <int>   <dbl> <chr>   <ord>
## 1 Stream 16572 0.00482 eta2[H] small
```

² <https://mixanalytic.com/guides/mood-analysis>

Kruskal-Wallis effect size za Mood_quadrant spada u kategoriju zanemarljivo malog efekta.

```
ggplot(train, aes(x = Mood_quadrant, y = log1p(Stream), fill =  
Mood_quadrant)) +  
  geom_boxplot(outlier.shape = NA, alpha = 0.7) +  
  theme_minimal() +  
  theme(legend.position = "none", axis.text.x = element_text(angle = 20,  
hjust = 1)) +  
  labs(title = "Distribucija Stream-a po Mood Quadrant-u (log skala)", x =  
"", y = "log(Stream + 1)")
```



Boxplot potvrđuje zanemarljiv Kruskal-Wallis effect size dobijen ranije - medijane sve četiri kategorije Mood_quadrant-a nalaze se na gotovo identičnom nivou, a i sami boxplot-ovi se u velikoj meri preklapaju.

Danceability i Energy (Party index)

Pošto kombinacija Valence i Energy (Mood_quadrant) pokazuje zanemarljiv efekat, probaćemo drugu kombinaciju audio karakteristika koja ima smisla iz domenskog znanja. Danceability i Energy zajedno bi trebalo da opisuju koliko je pesma zabavna i energična. Umesto kategorijske podele, pravimo feature množenjem dve vrednosti, kako bismo zadržali kontinualnu prirodu podataka.

Izvor: Spotify developer documentation - Spotify for Developers³

```
train = train %>% mutate(Party_index = Danceability * Energy)
test = test %>% mutate(Party_index = Danceability * Energy)

cor.test(train$Party_index, train$Stream, method = "spearman")

## Warning in cor.test.default(train$Party_index, train$Stream, method =
## "spearman"): Cannot compute exact p-value with ties

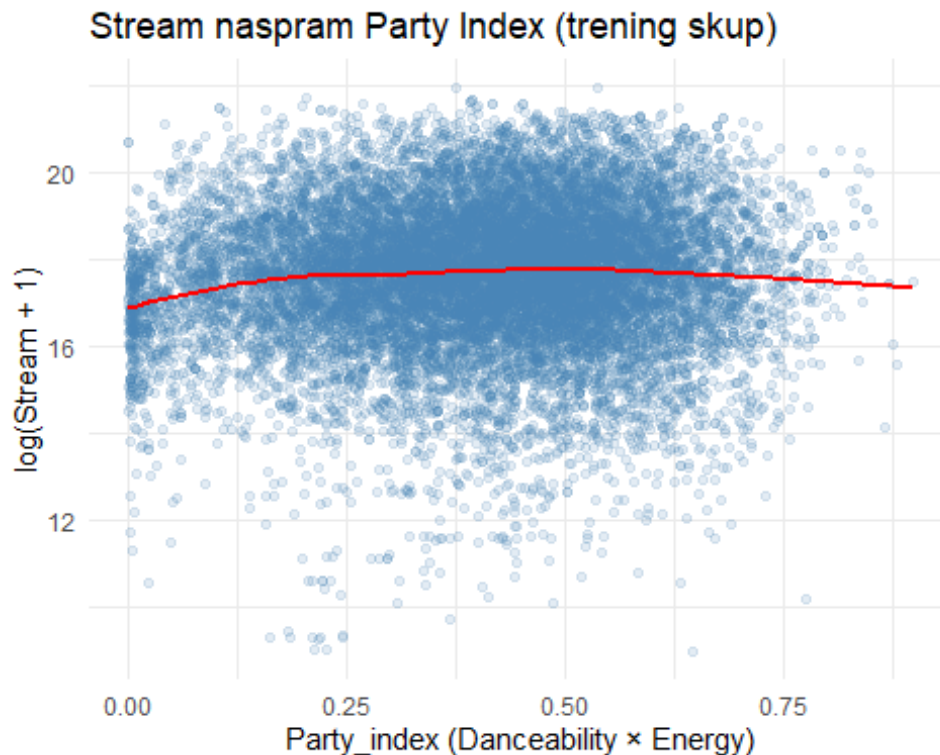
##
## Spearman's rank correlation rho
##
## data: train$Party_index and train$Stream
## S = 7.1721e+11, p-value = 2.273e-12
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## 0.05446904
```

Party_index ima nešto jaču vezu sa Stream-om nego njegove pojedinačne komponente, ali su i dalje sve vrednosti veoma slabo korelisane.

```
train %>%
  ggplot(aes(x = Party_index, y = log1p(Stream))) +
  geom_point(alpha = 0.15, color = "steelblue") +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  theme_minimal() +
  labs(title = "Stream naspram Party Index (trening skup)", x = "Party_index
(Danceability × Energy)", y = "log(Stream + 1)")

## `geom_smooth()` using formula = 'y ~ x'
```

³ <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>



Scatter plot potvrđuje ono što je Spearman korelacija već pokazala, vidljiva je izuzetno slaba veza između Party_index i Stream-a. Crvena loess linija je gotovo horizontalna kroz ceo opseg vrednosti, sa jedva приметnim povećanjem u opsegu Party_index vrednosti od 0.2 do 0.5, nakon čega ostaje ravna.

Binarna klasifikacija audio karakteristika

Spotify-jeva zvanična dokumentacija predlaže konkretne pragove za pojedine audio karakteristike koji ukazuju na specifičnu vrstu sadržaja, npr. Speechiness iznad 0.66 ukazuje da je numera pretežno govorni sadržaj, Instrumentalness iznad 0.5 ukazuje na instrumentalnu numeru bez vokala, a Liveness iznad 0.8 ukazuje na veliku verovatnoću da je snimak uživo izvođenje.

```
dodaj_binarne_flagove = function(df) {
  df %>%
    mutate(
      Is_spoken = ifelse(Speechiness > 0.66, TRUE, FALSE),
      Is_instrumental = ifelse(Instrumentalness > 0.5, TRUE, FALSE),
      Is_live = ifelse(Liveness > 0.8, TRUE, FALSE)
    )
}

train = dodaj_binarne_flagove(train)
test = dodaj_binarne_flagove(test)

wilcox_effsize(Stream ~ Is_spoken, data = train)
```

```
## # A tibble: 1 × 7
##   .y.    group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr> <chr>    <dbl> <int> <int> <ord>
## 1 Stream FALSE  TRUE    0.0842 16471   101 small
```

```
wilcox_effsize(Stream ~ Is_instrumental, data = train)
```

```
## # A tibble: 1 × 7
##   .y.    group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr> <chr>    <dbl> <int> <int> <ord>
## 1 Stream FALSE  TRUE    0.125 15680   892 small
```

```
wilcox_effsize(Stream ~ Is_live, data = train)
```

```
## # A tibble: 1 × 7
##   .y.    group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr> <chr>    <dbl> <int> <int> <ord>
## 1 Stream FALSE  TRUE    0.0336 16269   303 small
```

Sve tri binarne klasifikacije pokazuju po rstatix klasifikaciji malu (*small*) razliku u Stream-u. **Is_instrumental** pokazuje relativno najizraženiji efekat, ali i dalje po rstatix klasifikaciji ima mali uticaj.

Key

Key predstavlja muzički tonalitet pesme, kodiran kao ceo broj od 0 do 11, pri čemu vrednost -1 označava da tonalitet nije detektovan. Iako je Key tehnički numerički kodiran, on je po prirodi kategorijska promenljiva - brojevi ne nose redosled, pa možemo da ga tretiramo kao faktor.

```
train = train %>% mutate(Key_factor = factor(Key))
test = test %>% mutate(Key_factor = factor(Key))
```

```
table(train$Key_factor)
```

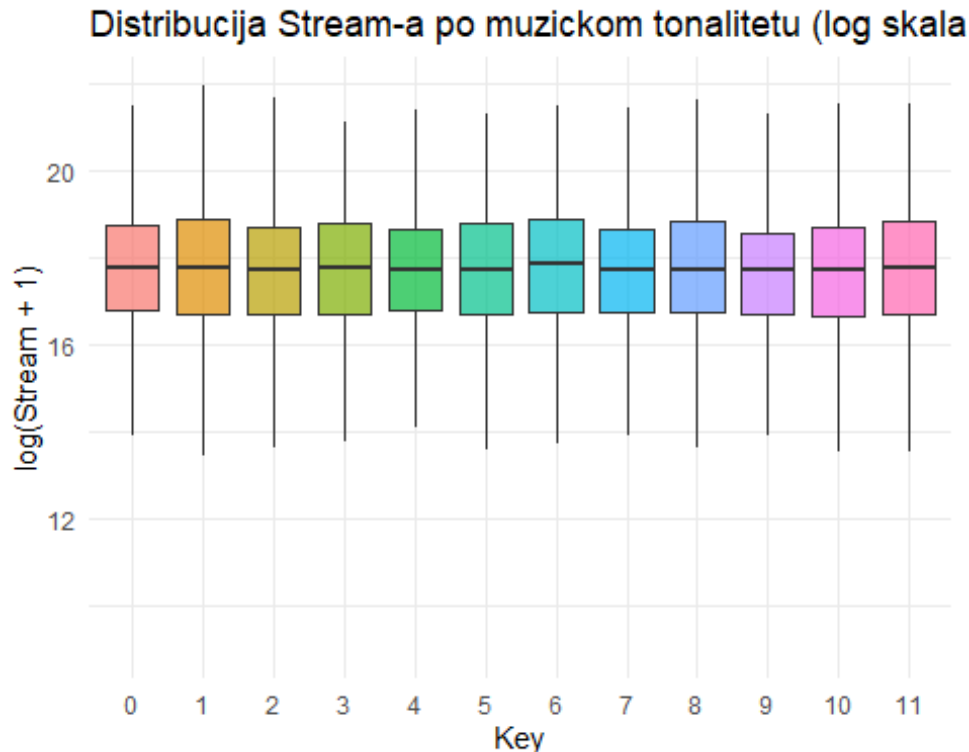
```
##
##    0    1    2    3    4    5    6    7    8    9   10   11
## 1832 1791 1583  537 1204 1372 1151 1839 1177 1606 1144 1336
```

```
kruskal_effsize(train, Stream ~ Key_factor)
```

```
## # A tibble: 1 × 5
##   .y.      n effsize method magnitude
## * <chr> <int>    <dbl> <chr>    <ord>
## 1 Stream 16572 0.000127 eta2[H] small
```

```
ggplot(train, aes(x = Key_factor, y = log1p(Stream), fill = Key_factor)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.7) +
  theme_minimal() +
  labs(title = "Distribucija Stream-a po muzickom tonalitetu (log skala,
```

```
trening skup)", x = "Key", y = "log(Stream + 1)") +  
  theme(legend.position = "none")
```



Na boxplot-u se ne vidi neka preterana razlika u odnosu na grupe, a Kruskal-Wallis test za Key_factor pokazuje jako mali effect size, koji je zanemarljiv.

Kolaboracija (feat. u nazivu pesme)

Pesme sa više izvođača (imaju “feat.” ili “ft.” u nazivu) mogle bi da imaju veći broj stream-ova, jer kombinuju publiku više izvođača. Ovu informaciju uzimamo direktno iz Track kolone, pretragom teksta na oznake koje ukazuju na gostujuće izvođače.

```
train = train %>% mutate(Is_collab = grepl("feat\\.|ft\\.|featuring", Track,  
ignore.case = TRUE))  
test = test %>% mutate(Is_collab = grepl("feat\\.|ft\\.|featuring", Track,  
ignore.case = TRUE))
```

```
table(train$Is_collab)
```

```
##  
## FALSE TRUE  
## 15154 1418
```

```
wilcox_effsize(Stream ~ Is_collab, data = train)
```

```
## # A tibble: 1 × 7  
## .y. group1 group2 effsize n1 n2 magnitude
```

```
## * <chr> <chr> <chr> <dbl> <int> <int> <ord>
## 1 Stream FALSE TRUE 0.0517 15154 1418 small
```

Ova kolona takođe pokazuje jako slabu povezanost sa kolonom Stream.

Licensed i Official_video

Licensed i Official_video pojedinačno imaju mali, ali statistički značajan efekat na Stream, i međusobno su povezane. Umesto da ih koristimo kao dve odvojene binarne kolone, kombinujemo ih u jednu kategorijsku promenljivu sa 4 nivoa.

```
dodaj_licensed_video_combo = function(df) {
  df %>%
    mutate(
      Licensed_video_combo = case_when(
        Licensed == "True" & Official_video == "True" ~ "Licensed_official",
        Licensed == "True" & Official_video == "False" ~
"Licensed_unofficial",
        Licensed == "False" & Official_video == "True" ~
"Unlicensed_official",
        Licensed == "False" & Official_video == "False" ~
"Unlicensed_unofficial",
        TRUE ~ 'Unknown'
      ),
      Licensed_video_combo = factor(Licensed_video_combo)
    )
}

train = dodaj_licensed_video_combo(train)
test = dodaj_licensed_video_combo(test)

table(train$Licensed_video_combo)

##
##      Licensed_official      Unknown      Unlicensed_official
##             11322             378             1264
## Unlicensed_unofficial
##             3608

kruskal.test(Stream ~ Licensed_video_combo, data = train)

##
##  Kruskal-Wallis rank sum test
##
## data:  Stream by Licensed_video_combo
## Kruskal-Wallis chi-squared = 227.18, df = 3, p-value < 2.2e-16

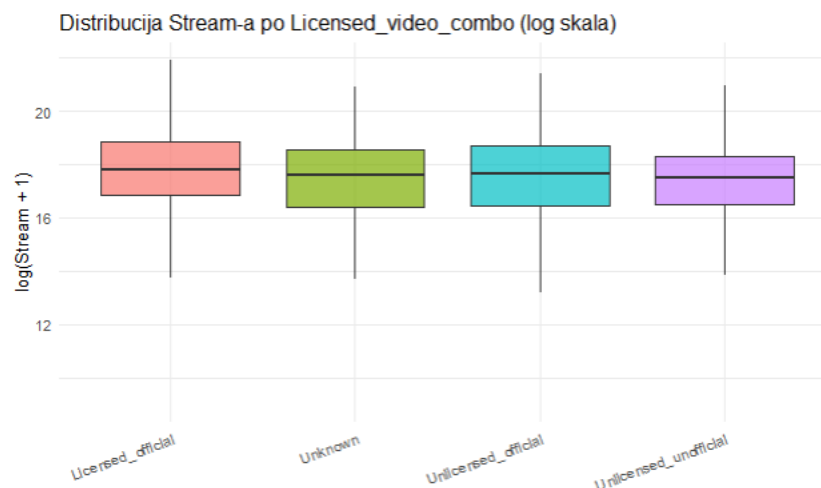
kruskal_effsize(train, Stream ~ Licensed_video_combo)

## # A tibble: 1 × 5
##   .y.      n effsize method  magnitude
```

```
## * <chr> <int> <dbl> <chr> <ord>
## 1 Stream 16572 0.0135 eta2[H] small
```

Kombinovana kategorija Licensed_video_combo pokazuje slab efekat na Stream, u skladu sa pojedinačnim efektima Licensed i Official_video iz bivarijantne analize.

```
ggplot(train, aes(x = Licensed_video_combo, y = log1p(Stream), fill =
Licensed_video_combo)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.7) +
  theme_minimal() +
  theme(legend.position = "none", axis.text.x = element_text(angle = 20,
hjust = 1)) +
  labs(title = "Distribucija Stream-a po Licensed_video_combo (log skala)", x
= "", y = "log(Stream + 1)")
```



Boxplot pokazuje blage, ali primetne razlike u medijani Stream-a između kategorija, kategorija Licensed_official ima vizuelno najvišu medijanu, dok Unlicensed_unofficial ima najnižu, sa preostale dve kategorije između njih.

Multivarijantna analiza

Bivarijantna analiza i feature engineering su nam dali uvid u to kako se svaka promenljiva pojedinačno odnosi prema Stream-u. Multivarijantna analiza nam objašnjava kako potencijalni prediktori međusobno figurišu i pruža nam mogućnost da uočimo neka specifična ponašanja, kao što je na primer sinergija. Ovu analizu radimo isključivo nad trening skupom - iz istog razloga kao i sve prethodne statistike, ne želimo da bilo koji zaključak o obliku modela (koji prediktori ulaze, da li ima multikolinearnosti) bude i najmanje oblikovan test podacima.

Izbor prediktora i dva modela

- **Model A (bez YouTube atributa):** Artist_LOO_stream, Album_type, audio karakteristike (Danceability, Energy, Loudness, Speechiness, Acousticness, Instrumentalness, Liveness, Valence, Tempo, Duration_ms). Ovo su promenljive koje (osim Genre iz API-ja) postoje nezavisno od toga da li pesma ima YouTube spot.
- **Model B (sa YouTube signalom):** Model A i Views, Licensed, Official_video, uključujemo i signal popularnosti sa druge platforme.

Views, Licensed i Official_video se mere u istom vremenskom okviru kao i sam Stream, ne pre objave pesme. Likes i Comments ostaju izostavljeni iz oba modela zbog kolinearnosti sa Views.

Oba modela grade se samo nad pesmama koje imaju YouTube spot (Has_youtube), jer Views, Licensed i Official_video imaju smisla samo tada - ovo obezbeđuje da se Model A i B upoređuju nad istim podacima.

```
data_model = train %>% filter(Has_youtube) %>% mutate(Licensed =  
droplevels(Licensed), Official_video = droplevels(Official_video))  
  
nrow(data_model)  
  
## [1] 16194
```

Provera multikolinearnosti (VIF)

Da bismo proverili da li postoji bilo kakav oblik multikolinearnosti, koristimo funkciju vif, i ukoliko neki faktor ima vrednost veću od 5 smatraćemo da ta kolona izaziva problem multikolinearnosti.

```
model_A = lm(log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +  
Danceability + Energy + Loudness + Speechiness + Acousticness +  
Instrumentalness + Liveness + Valence + Tempo + Duration_ms,  
data = data_model)  
  
vif(model_A)  
  
##                GVIF Df  GVIF^(1/(2*Df))  
## log1p(Artist_LOO_stream) 1.096823  1      1.047293  
## Album_type                1.084338  2      1.020449  
## Danceability              1.655977  1      1.286848  
## Energy                    3.500651  1      1.871003  
## Loudness                  3.325094  1      1.823484  
## Speechiness               1.107101  1      1.052189  
## Acousticness              1.937075  1      1.391789  
## Instrumentalness          1.577759  1      1.256089  
## Liveness                  1.068742  1      1.033800  
## Valence                   1.562142  1      1.249857
```

```
## Tempo                1.064414  1      1.031705
## Duration_ms          1.035322  1      1.017508

model_B = lm(
  log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +
  Danceability + Energy + Loudness + Speechiness + Acousticness +
  Instrumentalness + Liveness + Valence + Tempo + Duration_ms +
  log1p(Views) + Licensed + Official_video,
  data = data_model
)

vif(model_B)

##                GVIF Df GVIF^(1/(2*Df))
## log1p(Artist_LOO_stream) 1.229658  1      1.108899
## Album_type              1.117172  2      1.028087
## Danceability            1.668624  1      1.291752
## Energy                  3.503861  1      1.871860
## Loudness                3.378903  1      1.838179
## Speechiness             1.115037  1      1.055953
## Acousticness            1.938556  1      1.392320
## Instrumentalness        1.582666  1      1.258040
## Liveness                1.068844  1      1.033849
## Valence                 1.565923  1      1.251369
## Tempo                   1.065219  1      1.032094
## Duration_ms             1.042991  1      1.021269
## log1p(Views)            1.469205  1      1.212107
## Licensed                3.083515  1      1.755994
## Official_video          3.183533  1      1.784246
```

Ni u jednom ni u drugom modelu nema problema multikolinearnosti, kako su sve VIF vrednosti manje od 5.

Osnovni modeli

Osnovni modeli

```
summary(model_A)

##
## Call:
## lm(formula = log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +
##     Danceability + Energy + Loudness + Speechiness + Acousticness +
##     Instrumentalness + Liveness + Valence + Tempo + Duration_ms,
##     data = data_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.7829 -0.6741  0.0281  0.7193  7.0405
```



```
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    3.989e+00  1.816e-01  21.964 < 2e-16 ***
## log1p(Artist_LOO_stream) 7.866e-01  7.624e-03 103.171 < 2e-16 ***
## Album_typecompilation -1.415e-01  5.141e-02  -2.753  0.00591 **
## Album_typesingle     -4.865e-01  2.301e-02 -21.148 < 2e-16 ***
## Danceability       3.287e-01  7.357e-02   4.467 7.97e-06 ***
## Energy            -3.652e-01  8.262e-02  -4.420 9.95e-06 ***
## Loudness          2.452e-02  3.736e-03   6.563 5.43e-11 ***
## Speechiness       -4.890e-01  8.979e-02  -5.446 5.22e-08 ***
## Acousticness      -8.981e-02  4.609e-02  -1.949  0.05135 .
## Instrumentalness  -2.245e-02  6.098e-02  -0.368  0.71279
## Liveness          -1.369e-01  6.007e-02  -2.279  0.02268 *
## Valence           -8.224e-02  4.801e-02  -1.713  0.08676 .
## Tempo             1.864e-04  3.286e-04   0.567  0.57051
## Duration_ms       9.020e-07  1.124e-07   8.022 1.12e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.208 on 16187 degrees of freedom
## Multiple R-squared:  0.4516, Adjusted R-squared:  0.4512
## F-statistic: 1025 on 13 and 16187 DF, p-value: < 2.2e-16
```

Model A objašnjava oko 45% varijanse, što je solidno za model koji se oslanja samo na osobine same pesme, kako smo u toku analize zaključili da ove kolone nemaju mnogo uticaja na Stream, bez ijednog spoljašnjeg signala popularnosti, s tim što je Artist_LOO najjači prediktor.

summary(model_B)

```
##
## Call:
## lm(formula = log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +
##     Danceability + Energy + Loudness + Speechiness + Acousticness +
##     Instrumentalness + Liveness + Valence + Tempo + Duration_ms +
##     log1p(Views) + Licensed + Official_video, data = data_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.9892 -0.6235 -0.0089  0.6366  5.2350
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.760e+00  1.609e-01  17.159 < 2e-16 ***
## log1p(Artist_LOO_stream) 6.329e-01  7.046e-03  89.826 < 2e-16 ***
## Album_typecompilation -1.228e-01  4.516e-02  -2.719  0.00655 **
## Album_typesingle     -3.870e-01  2.037e-02 -18.997 < 2e-16 ***
## Danceability       -3.620e-02  6.478e-02  -0.559  0.57628
## Energy            -2.910e-01  7.248e-02  -4.015 5.97e-05 ***
```

```
## Loudness -6.111e-03 3.309e-03 -1.847 0.06480 .
## Speechiness -9.301e-02 7.902e-02 -1.177 0.23917
## Acousticness -1.469e-01 4.043e-02 -3.634 0.00028 ***
## Instrumentalness 1.569e-01 5.355e-02 2.930 0.00339 **
## Liveness -1.359e-01 5.268e-02 -2.580 0.00990 **
## Valence -1.052e-01 4.216e-02 -2.496 0.01258 *
## Tempo -2.192e-04 2.882e-04 -0.760 0.44701
## Duration_ms 3.796e-08 9.945e-08 0.382 0.70267
## log1p(Views) 2.489e-01 3.590e-03 69.345 < 2e-16 ***
## LicensedFalse 7.028e-02 3.200e-02 2.196 0.02811 *
## Official_videoFalse 3.313e-01 3.583e-02 9.246 < 2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.059 on 16184 degrees of freedom
## Multiple R-squared: 0.5784, Adjusted R-squared: 0.578
## F-statistic: 1388 on 16 and 16184 DF, p-value: < 2.2e-16
```

Kad se doda Views (uz Licensed/Official_video), R^2 na skoro 0.58, a Views je, očekivano, najjači prediktor u celom modelu.

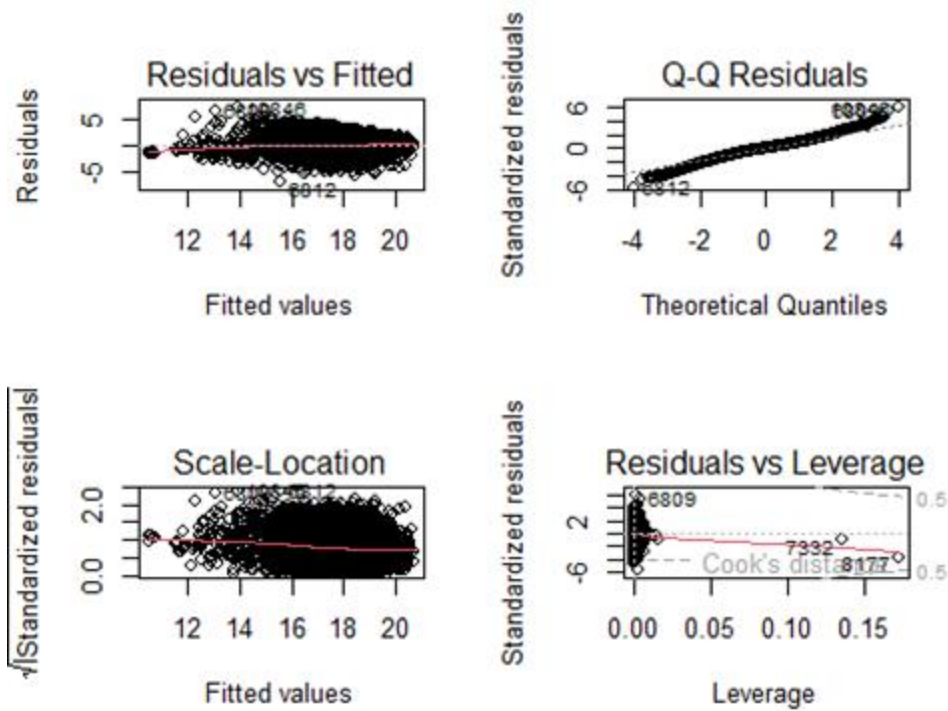
```
anova(model_A, model_B)
```

```
## Analysis of Variance Table
##
## Model 1: log1p(Stream) ~ log1p(Artist_L00_stream) + Album_type +
Danceability +
## Energy + Loudness + Speechiness + Acousticness + Instrumentalness +
## Liveness + Valence + Tempo + Duration_ms
## Model 2: log1p(Stream) ~ log1p(Artist_L00_stream) + Album_type +
Danceability +
## Energy + Loudness + Speechiness + Acousticness + Instrumentalness +
## Liveness + Valence + Tempo + Duration_ms + log1p(Views) +
## Licensed + Official_video
## Res.Df RSS Df Sum of Sq F Pr(>F)
## 1 16187 23602
## 2 16184 18145 3 5457.2 1622.5 < 2.2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

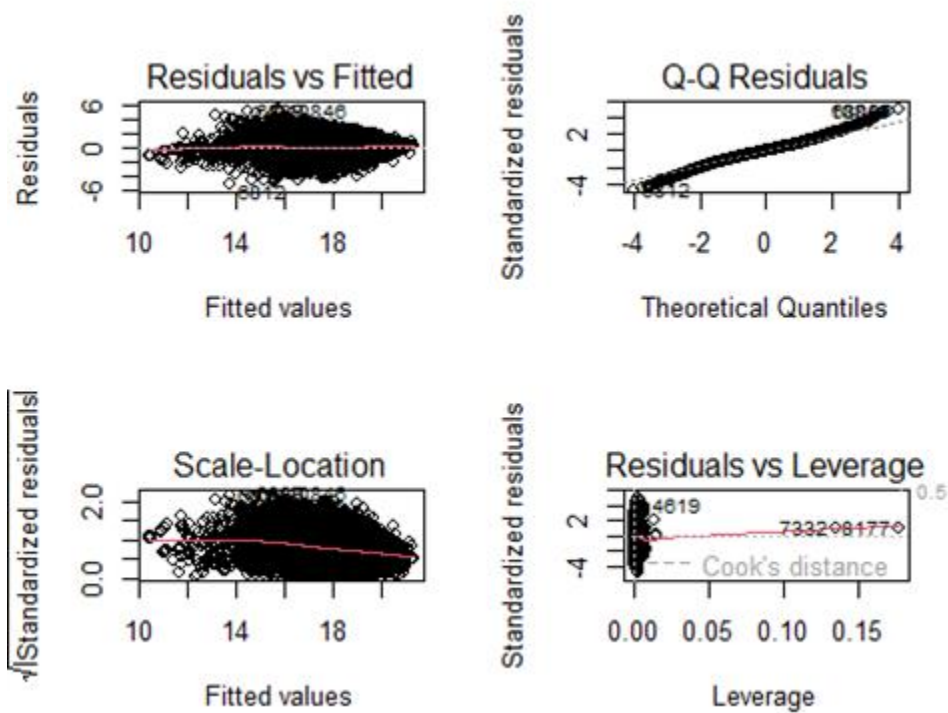
Anova test potvrđuje ono što se već moglo zaključiti, dodavanje Views/Licensed/Official_video značajno poboljšava model.

Dijagnostika modela

```
par(mfrow = c(2, 2))
plot(model_A)
```



```
par(mfrow = c(2, 2))
plot(model_B)
```



Ovakakva dijagnostika pokazuje da oba modela imaju isti problem, reziduali nisu ni savršeno homogeni ni normalni, pa postoji blaga heteroskedastičnost i primetan rep pesama koje model precenjuje, gde stvarni Stream ispadne mnogo niži nego što model predviđa. Realni zaključak je da su modeli dobro uhvatili glavni trend u podacima, ali imaju grešku sa određeneom podgrupom pesama (verovatno baš onih na velikom popularnošću), što se i moglo očekivati, obzirom na to koliko je Stream desno asimetričan čak i posle log transformacije.

Sinergija (interakcioni efekti)

Testiramo iste interakcije koje imaju smisla iz domenskog znanja kao u ranijem feature engineering-u, ovde nad Modelom B:

- **Danceability:Energy** - da li plesne i energične pesme imaju veći broj stream-ova
- **Loudness:Energy** - da li kombinacija glasnoće i energije nosi dodatni signal

```
model_B_interakcije = lm(
  log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +
  Danceability + Energy + Loudness + Speechiness + Acousticness +
  Instrumentalness + Liveness + Valence + Tempo + Duration_ms +
  log1p(Views) + Licensed + Official_video +
  Danceability:Energy + Loudness:Energy,
  data = data_model
)

summary(model_B_interakcije)

##
## Call:
## lm(formula = log1p(Stream) ~ log1p(Artist_LOO_stream) + Album_type +
##     Danceability + Energy + Loudness + Speechiness + Acousticness +
##     Instrumentalness + Liveness + Valence + Tempo + Duration_ms +
##     log1p(Views) + Licensed + Official_video + Danceability:Energy +
##     Loudness:Energy, data = data_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.9563 -0.6244 -0.0090  0.6379  5.2211
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.613e+00  1.883e-01  13.881  < 2e-16 ***
## log1p(Artist_LOO_stream) 6.321e-01  7.058e-03  89.555  < 2e-16 ***
## Album_typecompilation -1.194e-01  4.520e-02  -2.641  0.008264 **
## Album_typesingle     -3.874e-01  2.043e-02 -18.965  < 2e-16 ***
## Danceability       1.706e-01  1.629e-01   1.048  0.294822
## Energy            -4.673e-03  1.859e-01  -0.025  0.979953
```

```
## Loudness -1.168e-02 4.234e-03 -2.758 0.005826 **
## Speechiness -9.830e-02 7.915e-02 -1.242 0.214308
## Acousticness -1.464e-01 4.073e-02 -3.595 0.000325 ***
## Instrumentalness 1.444e-01 5.535e-02 2.609 0.009097 **
## Liveness -1.336e-01 5.280e-02 -2.529 0.011438 *
## Valence -1.029e-01 4.218e-02 -2.440 0.014686 *
## Tempo -2.556e-04 2.901e-04 -0.881 0.378254
## Duration_ms 5.386e-08 9.977e-08 0.540 0.589334
## log1p(Views) 2.490e-01 3.591e-03 69.349 < 2e-16 ***
## LicensedFalse 7.142e-02 3.201e-02 2.231 0.025669 *
## Official_videoFalse 3.311e-01 3.583e-02 9.242 < 2e-16 ***
## Danceability:Energy -3.248e-01 2.450e-01 -1.326 0.184909
## Energy:Loudness 1.508e-02 7.356e-03 2.050 0.040339 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.059 on 16182 degrees of freedom
## Multiple R-squared: 0.5785, Adjusted R-squared: 0.5781
## F-statistic: 1234 on 18 and 16182 DF, p-value: < 2.2e-16
```

```
anova(model_B, model_B_interakcije)
```

```
## Analysis of Variance Table
##
## Model 1: log1p(Stream) ~ log1p(Artist_L00_stream) + Album_type +
Danceability +
## Energy + Loudness + Speechiness + Acousticness + Instrumentalness +
## Liveness + Valence + Tempo + Duration_ms + log1p(Views) +
## Licensed + Official_video
## Model 2: log1p(Stream) ~ log1p(Artist_L00_stream) + Album_type +
Danceability +
## Energy + Loudness + Speechiness + Acousticness + Instrumentalness +
## Liveness + Valence + Tempo + Duration_ms + log1p(Views) +
## Licensed + Official_video + Danceability:Energy + Loudness:Energy
## Res.Df RSS Df Sum of Sq F Pr(>F)
## 1 16184 18145
## 2 16182 18140 2 4.9759 2.2195 0.1087
```

Iz ovoga možemo zaključiti da baš i nema sinergije među ovim prediktorima, jer se vrednost R^2 nije drastično povećala, a F-statistika se dodatno smanjila.

Pretprocesiranje podataka

Brišemo linkove i kolone bez prediktivne vrednosti za modele, kao i kolone koje su višak zbog feature engineering-a, ili su nastale feature engineering-om, a nisu se pokazale kao da bi imale veći potencijal od inicijalnih verzija kolona.

Pored toga, sklonićemo kolone za koje smo zaključili da nam neće biti relevantne za dalji rad: - Likes i Comments - za linearnu regresiju se svakako ne mogu koristiti zbog velike multikolinearnosti, a PCA nam je pokazao da ove kolone ne nose nove informacije pored Views-a - Official_video i Licensed - napravljena je kombinovana kolona koja podjednako kvalitetno utiče na ciljnu promenljivu. - Key - kreirana je faktorizovana kolona za key

izmena - nisu obrisane kolone Is_instrumental, Is_live, Is_spoken, Is_collab

```
brisanje_kolona = function(df) {
  df %>%
    select(-X, -Url_spotify, -Uri, -Url_youtube, -Title, -Description, -Channel, -Album, -Key, -
    Licensed, -Official_video, -Likes, -Comments)
}
```

```
train = brisanje_kolona(train)
```

```
test = brisanje_kolona(test)
```

```
ncol(train)
```

```
## [1] 27
```

```
names(train)
```

```
## [1] "Artist"           "Track"             "Album_type"
## [4] "Danceability"     "Energy"            "Loudness"
## [7] "Speechiness"      "Acousticness"      "Instrumentalness"
## [10] "Liveness"         "Valence"           "Tempo"
## [13] "Duration_ms"      "Views"             "Stream"
## [16] "Has_youtube"      "Artist_L00_stream" "Likes_rate"
## [19] "Comments_rate"    "Mood_quadrant"     "Party_index"
## [22] "Is_spoken"        "Is_instrumental"   "Is_live"
## [25] "Key_factor"       "Is_collab"         "Licensed_video_combo"
```

Radimo faktorizaciju kolona za koje je to potrebno.

```
faktorizuj_final = function(df) {
  df %>%
    mutate(
      Licensed_video_combo = as.character(Licensed_video_combo),
      Licensed_video_combo = factor(Licensed_video_combo, levels = c("Licensed_official",
      "Licensed_unofficial", "Unlicensed_official", "Unlicensed_unofficial", "Unknown")),
      Album_type = factor(Album_type)
    ) %>%
    mutate(across(where(is.logical), as.factor))
}
```

```
train = faktorizuj_final(train)
```

```
test = faktorizuj_final(test)
```

Trening i test skup su već formirani mnogo ranije (pre svake imputacije i enkodiranja koji zavise od Stream-a), pa ih ovde samo čuvamo u fajlove.

```
nrow(train)

## [1] 16572

nrow(test)

## [1] 4144

out_dir = "Podaci_za_model"
dir.create(out_dir, showWarnings = FALSE)

write_csv(train, file.path(out_dir, "train.csv"))
write_csv(test, file.path(out_dir, "test.csv"))
```

Modeli

Priprema za modele

Učitavamo pripremljene podatke za trening i test.

```
train = read_csv("Podaci_za_model/train.csv")
test = read_csv("Podaci_za_model/test.csv")

nrow(train)

## [1] 16572

nrow(test)

## [1] 4144
```

Upisivanje podataka u fajl ne čuva tip faktora i sve što je bilo faktor postaje običan tekst pri čuvanju, pa opet radimo faktorizaciju.

```
uskladi_tipove = function(df) {
  df %>%
    mutate(
      Album_type = factor(Album_type),
      Licensed_video_combo = factor(
        Licensed_video_combo,
        levels = c("Licensed_official", "Licensed_unofficial",
"Unlicensed_official", "Unlicensed_unofficial", "Unknown")
      ),
      Has_youtube = factor(Has_youtube)
    )
}
```

```
train_ml = uskladi_tipove(train)
test_ml = uskladi_tipove(test)
```

Kod pesama bez YouTube spota, Views je ranije postavljen na 0 kao deo dummy variable adjustment tehnike (nedostajuća vrednost popunjena konstantom uz indikatorsku kolonu koja to obeležava). Ovakav pristup je prihvatljiv za modele zasnovane na stablima, koji mogu da izoluju te redove razdvajanjem po Has_youtube koloni, ali je Jones (1996) pokazao da dummy variable adjustment daje pristrasne ocene parametara u linearnoj regresiji, čak i kada su podaci MCAR. Zbog toga za linearne modele izbacujemo redove bez YouTube spota, umesto da se oslanjamo na Has_youtube indikator da “ispravi” veštačke nule u Views. Iz istog razloga, kako bi poređenje sa linearnim modelima bilo fer, i Random Forest, XGBoost i LightGBM ćemo u glavnom delu rada trenirati i testirati nad ovim istim skupom podataka.

Pored toga, Mood_quadrant i Key_factor gube tip faktora prilikom čuvanja/učitavanja iz CSV-a (postaju tekst, odnosno broj), pa ih ovde vraćamo u faktor.

```
train_model = train_ml %>%
  filter(Has_youtube == TRUE) %>%
  select(-Has_youtube) %>%
  mutate(
    Mood_quadrant = factor(Mood_quadrant),
    Key_factor = factor(Key_factor)
  )

test_model = test_ml %>%
  filter(Has_youtube == TRUE) %>%
  select(-Has_youtube) %>%
  mutate(
    Mood_quadrant = factor(Mood_quadrant),
    Key_factor = factor(Key_factor)
  )

nrow(train_model)

## [1] 16194

nrow(test_model)

## [1] 4053
```

Nakon toga, neophodno je da kreiramo zajedničku funkciju za evaluaciju, jer ćemo sve kreirane modele upoređivati pomoću istih metrika, odnosno MAE, RMSE i R^2 .

```
eval_metrics = function(actual_log, pred_log) {
  rmse_log = sqrt(mean((actual_log - pred_log)^2))
  mae_log = mean(abs(actual_log - pred_log))
  r2_log = 1 - sum((actual_log - pred_log)^2) / sum((actual_log -
```



```
mean(actual_log))^2)
  tibble(RMSE_log = rmse_log, MAE_log = mae_log, R2_log = r2_log)
}
```

Linearni modeli

Pre naprednih, nelinearnih modela testiraćemo tri pristupa u kreiranju linearnih modela, odnosno običnu linearnu regresiju, Lasso i Ridge regresiju.

Feature selection

Pre same regresije, radimo formalnu selekciju prediktora, kako se ne bi oslonili isključivo na samostalni izbor kao u multivarijantnoj analizi. Prvo definišemo pun model, koji uključuje sve preostale kolone koje imaju smisla kao prediktori (izuzev Artist i Track, koji su sačuvane samo zar identifikacije redova), a zatim ćemo nad njim primeniti stepwise selekciju.

U okviru linearnih modela, nećemo koristiti Likes i Comments, kako su izuzetno jako korelisani sa Views, pa bi to dovelo do problema multikolinearnosti.

Views je, kao i Stream, izrazito desno asimetričan, pa ga radimo logaritamsku transformaciju. Takođe, logaritamsku transformaciju radićemo i za kolonu Artist_LOO_Stream jer ona nastala od Stream-ova, pa je samim tim i ona desno asimetrična.

```
full_formula = as.formula(
  log1p(Stream) ~ Album_type + Danceability + Energy + Loudness + Speechiness +
    Acousticness + Instrumentalness + Liveness + Valence + Tempo + Duration_ms +
    log1p(Views) + log1p(Artist_LOO_stream) + Likes_rate + Comments_rate +
    Mood_quadrant + Party_index + Key_factor + Licensed_video_combo
)
```

Pre treniranja modela proveravamo i raspodelu nivoa Licensed_video_combo, kako bismo znali da li neki nivo ima premalo (ili nimalo) opažanja u train_model.

```
table(train_model$Licensed_video_combo)
```

##			
##	Licensed_official	Licensed_unofficial	Unlicensed_official
##	11322	0	1264
##	Unlicensed_unofficial	Unknown	
##	3608	0	

Odavde možemo da vidimo da nivoi Licensed_unofficial i Unknown nemaju nijednu opservaciju u trening skupu, kako Unknown postoji samo kod pesama bez YouTube spota, koje smo izbacili, dok je Licensed_unofficial u ovom skupu samo odsutan). Pre nego što nastavimo, moramo da obrišemo te neiskorišćene nivoe, kako ne bismo pravili prazne kolone u kasnijim koracima.

```

train_model = train_model %>% mutate(Licensed_video_combo =
droplevels(Licensed_video_combo))
test_model = test_model %>% mutate(Licensed_video_combo =
factor(as.character(Licensed_video_combo), levels =
levels(train_model$Licensed_video_combo)))

table(train_model$Licensed_video_combo)

##
##      Licensed_official  Unlicensed_official Unlicensed_unofficial
##                11322                1264                3608

full_model = lm(full_formula, data = train_model)

```

Stepwise selekcija

Nad punim modelom primenjujemo stepwise selekciju u oba smera, zasnovanu na AIC kriterijumu, kako bismo dobili redukovani skup prediktora za linearnu regresiju. Stepwise selekcija radi tako što iterativno dodaje ili uklanja prediktore, faktički radi kao kombinacija backward i forward selekcije.

```

step_model = step(full_model, direction = "both", trace = 0)
summary(step_model)

##
## Call:
## lm(formula = log1p(Stream) ~ Album_type + Energy + Loudness +
##      Acousticness + Instrumentalness + Liveness + Valence + log1p(Views) +
##      log1p(Artist_LOO_stream) + Likes_rate + Comments_rate +
##      Licensed_video_combo,
##      data = train_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.8771 -0.6297 -0.0101  0.6334  5.2850
##
## Coefficients:
##                                     Estimate Std. Error t value
## (Intercept)                2.821076    0.148642  18.979
## Album_typecompilation      -0.122515    0.045016  -2.722
## Album_typesingle          -0.373884    0.020425 -18.305
## Energy                    -0.292383    0.070940  -4.122
## Loudness                   -0.005130    0.003254  -1.577
## Acousticness               -0.146358    0.039543  -3.701
## Instrumentalness           0.176496    0.052986   3.331
## Liveness                   -0.139885    0.052109  -2.684
## Valence                    -0.132486    0.037847  -3.501
## log1p(Views)               0.241461    0.003885  62.150
## log1p(Artist_LOO_stream)   0.637015    0.007001  90.991

```

```
## Likes_rate -2.186958 0.949409 -2.303
## Comments_rate -35.006051 9.302190 -3.763
## Licensed_video_comboUnlicensed_official 0.078933 0.031967 2.469
## Licensed_video_comboUnlicensed_unofficial 0.393617 0.022114 17.799
## Pr(>|t|)
## (Intercept) < 2e-16 ***
## Album_typecompilation 0.006503 **
## Album_typesingle < 2e-16 ***
## Energy 3.78e-05 ***
## Loudness 0.114877
## Acousticness 0.000215 ***
## Instrumentalness 0.000867 ***
## Liveness 0.007272 **
## Valence 0.000466 ***
## log1p(Views) < 2e-16 ***
## log1p(Artist_LOO_stream) < 2e-16 ***
## Likes_rate 0.021264 *
## Comments_rate 0.000168 ***
## Licensed_video_comboUnlicensed_official 0.013551 *
## Licensed_video_comboUnlicensed_unofficial < 2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.058 on 16186 degrees of freedom
## Multiple R-squared: 0.5793, Adjusted R-squared: 0.5789
## F-statistic: 1592 on 14 and 16186 DF, p-value: < 2.2e-16
```

Analizom koeficijenata možemo izvući nekoliko zapažanja:

- **Audio karakteristike** su ovde sve statistički značajne, iako nijedna pojedinačno nije prelazila korelaciju od 0.14 sa Stream-om, ali u multivarijantnom modelu, gde se kontrolišu ostali prediktori, čak i slab uticaj može postati statistički merljiv, dok koeficijenti (svi ispod 1) pokazuju da je taj uticaj i dalje praktično mali.
- **Artist_LOO_stream** ima prilično veliki koeficijent, što je u skladu sa njegovom jakom korelacijom sa Stream-om.
- **Comments_rate** pokazuje jak negativan koeficijent (manji od -30, $p < 0.001$), u skladu sa negativnom korelacijom uočenom ranije.

formula(step_model)

```
## log1p(Stream) ~ Album_type + Energy + Loudness + Acousticness +
## Instrumentalness + Liveness + Valence + log1p(Views) +
## log1p(Artist_LOO_stream) +
## Likes_rate + Comments_rate + Licensed_video_combo
```

Stepwise selekcija je izbacila Danceability, Tempo, Mood_quadrant i Key_factor, a to su sve prediktori koji su se i u bivarijantnoj analizi pokazali kao zanemarljivi. Adjusted R^2 je ostao identičan uz jednostavniji model, što je i cilj stepwise selekcije.

Obična linearna regresija

Model dobijen stepwise selekcijom koristimo kao finalni model obične linearne regresije, i evaluiramo ga uz pomoć napravljene funkcije.

```
lm_pred_train = predict(step_model, newdata = train_model)
lm_eval_train = eval_metrics(log1p(train_model$Stream), lm_pred_train) %>%
mutate(Model = "Linearna regresija (trening)")
lm_eval_train

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>   <dbl> <chr>
## 1     1.06   0.807   0.579 Linearna regresija (trening)

y_test_model = log1p(test_model$Stream)
lm_pred = predict(step_model, newdata = test_model)

lm_eval = eval_metrics(y_test_model, lm_pred) %>% mutate(Model = "Linearna
regresija")
lm_eval

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>   <dbl> <chr>
## 1     1.07   0.817   0.574 Linearna regresija
```

Obična linearna regresija na testnom skupu R^2 od oko 0.57, vrlo blizu R^2 na trening skupu s kojom se razlikuje samo za oko 0.05, što znači da model dobro generalizuje i da nema znakova overfitting-a.

Priprema za Lasso i Ridge regresiju

Funkcija `glmnet`, koju koristimo za Lasso i Ridge, zahteva eksplicitnu numeričku matricu prediktora, pa kategorijske kolone moramo da kodiramo. Koristimo celu formulu, a ne redukovanu formulu iz stepwise selekcije, jer je upravo poenta Lasso i Ridge regresije da regularizacija sama proceni koji prediktori doprinose odgovoru.

```
n_train_model = nrow(train_model)
data_matrica_model = bind_rows(train_model, test_model)

X_full_model = model.matrix(full_formula, data = data_matrica_model)[, -1]
y_full_model = log1p(data_matrica_model$Stream)

X_train_model = X_full_model[1:n_train_model, ]
X_test_model = X_full_model[(n_train_model + 1):nrow(X_full_model), ]
y_train_model = y_full_model[1:n_train_model]
```

```
dim(X_train_model)
## [1] 16194    33

dim(X_test_model)
## [1] 4053     33
```

Lasso regresija

Lasso (L1 regularizacija) dodaje kaznu na sumu apsolutnih vrednosti koeficijenata, što neke koeficijente svodi tačno na nulu. Na ovaj način, Lasso faktički radi feature selection. Parametar lambda (jačina regularizacije) biramo unakrsnom validacijom, pri čemu tražimo vrednost koja najbolje minimizuje grešku.

```
set.seed(123)
lasso_cv = cv.glmnet(X_train_model, y_train_model, alpha = 1)

lasso_model = glmnet(X_train_model, y_train_model, alpha = 1, lambda =
lasso_cv$lambda.min)
lasso_pred = as.numeric(predict(lasso_model, newx = X_test_model))

lasso_eval = eval_metrics(y_test_model, lasso_pred) %>% mutate(Model = "Lasso
regresija")
lasso_eval

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>    <dbl> <dbl> <chr>
## 1     1.07    0.818  0.573 Lasso regresija
```

Pogledaćemo koje je koeficijente Lasso sveo na nulu, tj. koje je prediktore efektivno izbacio iz modela.

```
lasso_coef = as.data.frame(as.matrix(coef(lasso_model)))
names(lasso_coef) = "Coefficient"
lasso_coef %>%
  rownames_to_column("Feature") %>%
  filter(Coefficient == 0)

lasso_coef = as.data.frame(as.matrix(coef(lasso_model)))
names(lasso_coef) = "Coefficient"
lasso_coef %>%
  rownames_to_column("Feature") %>%
  filter(Coefficient == 0)
```

	Feature	Coefficient
## 1	Danceability	0
## 2	Duration_ms	0
## 3	Mood_quadrantCalm/Relaxing	0
## 4	Mood_quadrantSad/Melancholic	0
## 5	Key_factor3	0
## 6	Key_factor4	0
## 7	Key_factor5	0
## 8	Key_factor6	0
## 9	Key_factor11	0

Lasso daje skoro identičan rezultat kao obična regresija, uz izbacivanje 5 nivoa za Keyfactor, Danceability, Durationms i 2 nivoa za Mood_quadrant.

Ridge regresija

Ridge (L2 regularizacija) dodaje kaznu na sumu kvadrata koeficijenata, čime ih smanjuje ka nuli, ali ih nikada ne svodi tačno na nulu. Ridge je pre svega način da se stabilizuju ocene koeficijenata kod korelisanih prediktora. I ovde lambda biramo unakrsnom validacijom.

```
set.seed(123)
ridge_cv = cv.glmnet(X_train_model, y_train_model, alpha = 0)

ridge_model = glmnet(X_train_model, y_train_model, alpha = 0, lambda =
ridge_cv$lambda.min)
ridge_pred = as.numeric(predict(ridge_model, newx = X_test_model))

ridge_eval = eval_metrics(y_test_model, ridge_pred) %>% mutate(Model = "Ridge
regresija")
ridge_eval

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>   <dbl> <chr>
## 1     1.08     0.823   0.572 Ridge regresija
```

Ridge regresija daje rezultat sličan prethodnim metodama, s tim što sada nema potrebe da proveramo koje je koeficijente svela na 0, jer to ona ne radi.

Poređenje linearnih modela

Na kraju kreiranja svih linearnih modela, upoređićemo njihove greške.

```
bind_rows(lm_eval, lasso_eval, ridge_eval)

## # A tibble: 3 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>   <dbl> <chr>
```

## 1	1.07	0.817	0.574	Linearna regresija
## 2	1.07	0.818	0.573	Lasso regresija
## 3	1.08	0.823	0.572	Ridge regresija

Sva tri linearna modela daju gotovo identičan rezultat, pri čemu R^2 ima vrednost oko 0.57. Regularizacija ovde ne donosi poboljšanje u odnosu na običnu regresiju, što je i očekivano, jer VIF proveru u multivarijantnoj analizi nije pokazala ozbiljnu multikolinearnost među prediktorima, a broj opservacija je mnogo veći od broja prediktora, pa obična linearna regresija nije sklona overfitting-u koji bi regularizacija trebalo da ublaži. U sva tri modela Views i Artist_LOO_stream ostaju ubedljivo najjači prediktori.

Na kraju, ostaje da vidimo da li nelinearni modeli, koji automatski hvataju interakcije i nelinearnosti, rade bolje nad ovim podacima.

Napredni modeli mašinskog učenja

Pored linearne regresije, testiraćemo i tri boosting algoritma, kako bismo videli da li nelinearni modeli i interakcije koje oni mogu da uhvate donose bolju prediktivnu moć. Koristimo algoritme Random Forest, LightGBM i XGBoost.

Random Forest je bagging algoritam (svako stablo se trenira nad bootstrap uzorkom), dok su XGBoost i LightGBM boosting algoritmi (stabla se dodaju sekvencijalno, svako ispravlja greške prethodnih). Sva tri su neparametarski modeli zasnovani na stablima odlučivanja, pa automatski hvataju nelinearnosti i interakcije između prediktora bez potrebe da ih ručno definišemo, za razliku od linearne regresije.

Priprema za napredne modele

Definisaćemo formulu za sva tri modela, sličnu formuli korišćenoj kod linearnih modela, s tim što ovde ne moramo da radimo logaritamsku transformaciju Views i Artist_LOO_stream, jer ovde nemamo problem sa nelinearnim odnosima.

Izmena, vraćamo kolone nastale feature engineering-om koje smo izbacili neopravdano u preprocesiranju podataka, tako da će se možda promeniti spisak relevantnih kolona, trenutno top 15 najbitnijih kolona za modele su:

Random Forest - Views, Artist_LOO_Stream, Likes_rate, Comments_rate, Licensed_video_combo, Album_type, Loudness, Duration_ms, Party_index, Acousticness, Energy, Danceability, Instrumentalness, Valence, Tempo

#XGBoost - Artist_LOO_Stream, Views, Likes_rate, Comments_rate, Duration_ms, Loudness, Album_type, Acousticness, Valence, Liveness, Tempo, Danceability, Party_index, Licensed_video_combo: Unlicensed_unofficial, Energy

#LightGBM - Views, Artist_LOO_Stream, Comments_rate, Likes_rate, Duration_ms, Loudness, Valence, Acousticness, Tempo, Liveness, Danceability, Party_index, Energy, Album_typesingle, Licensed_video_comboUnlicensed_unofficial

formula_model = as.formula(log1p(Stream) ~ Album_type + Loudness + Acousticness + Instrumentalness + Views + Artist_LOO_stream + Likes_rate + Comments_rate + Energy +

Duration_ms + Tempo + Liveness + Danceability + Valence + Party_index + Licensed_video_combo)

```
formula_model = as.formula(log1p(Stream) ~ Album_type + Loudness + Acousticness +  
Instrumentalness + Views + Artist_LOO_stream + Likes_rate + Comments_rate + Energy +  
Duration_ms + Tempo + Liveness + Danceability + Valence + Party_index +  
Licensed_video_combo + Is_live + Is_collab + Is_spoken + Is_instrumental +  
Mood_quadrant + Key_factor)
```

XGBoost i LightGBM zahtevaju numeričku matricu, pa moramo da kodiramo kategorijske kolone. Kao i kod Lasso/Ridge regresije, a train i test moramo privremeno da spojimo kako bi matrica imala identične kolone za oba skupa.

```
n_train = nrow(train_model)  
  
data_matrica = bind_rows(train_model, test_model)  
X_full = model.matrix(formula_model, data = data_matrica)[, -1]  
y_full = log1p(data_matrica$Stream)  
  
X_train = X_full[1:n_train, ]  
X_test = X_full[(n_train + 1):nrow(X_full), ]  
y_train = y_full[1:n_train]  
y_test = y_full[(n_train + 1):length(y_full)]  
  
dim(X_train)  
## [1] 16201 36  
  
dim(X_test)  
## [1] 4046 36
```

Modeli sa Youtube signalom

Random Forest

Random Forest je algoritam mašinskog učenja koji koristi veliki broj stabala odlučivanja kako bi pravio bolje predikcije. Svako stablo posmatra različite, nasumično izabrane delove podataka, a njihovi rezultati se kombinuju. Zbog toga se Random Forest smatra tehnikom ansambl učenja (ensemble learning), u kome više modela radi zajedno, što može da pomogne u povećanju tačnosti i smanjenju grešaka.

Pre kreiranja modela, sprovedemo pretragu hiperparametara kako bismo pronašli kombinaciju koja daje najbolje performanse. Za Random Forest ćemo tražiti optimalne vrednosti za `mtry`, odnosno broj nasumično izabranih prediktora koji se razmatraju pri svakom razdvajanju čvora i `min.node.size`, što je minimalna veličina lista stabla. Implementacija Random Forest-a iz `ranger` biblioteke nudi out-of-bag (OOB) grešku kao

ugrađenu procenu greške modela na neviđenim podacima, pa nam nije potrebna eksplicitna unakrsna validacija.

```
grid_rf = expand.grid(
  mtry = c(3, 5, 7),
  min.node.size = c(1, 5, 10)
)

rf_results = vector("list", nrow(grid_rf))

for (i in seq_len(nrow(grid_rf))) {
  set.seed(123)
  m_i = ranger(
    formula_model,
    data = train_model,
    num.trees = 500,
    mtry = grid_rf$mtry[i],
    min.node.size = grid_rf$min.node.size[i],
    importance = "none"
  )

  rf_results[[i]] = tibble(
    mtry = grid_rf$mtry[i],
    min.node.size = grid_rf$min.node.size[i],
    oob_rmse = sqrt(m_i$prediction.error)
  )
}

rf_results_df = bind_rows(rf_results) %>% arrange(oob_rmse)
rf_results_df

## # A tibble: 9 × 3
##   mtry min.node.size oob_rmse
##   <dbl>         <dbl>    <dbl>
## 1     7             1    0.912
## 2     7             5    0.914
## 3     7            10    0.916
## 4     5             1    0.918
## 5     5             5    0.919
## 6     5            10    0.923
## 7     3             1    0.943
## 8     3             5    0.945
## 9     3            10    0.951

best_rf = rf_results_df %>% slice(1)
best_rf
```

```
## # A tibble: 1 × 3
##   mtry min.node.size oob_rmse
##   <dbl>         <dbl>   <dbl>
## 1     7             1   0.912

rf_model = ranger(
  formula_model,
  data = train_model,
  num.trees = 500,
  mtry = best_rf$mtry,
  min.node.size = best_rf$min.node.size,
  importance = "permutation",
  seed = 123
)
```

Nakon kreiranja modela nad treningom skupom, izvršićemo predikciju i ocenu za test skup.

```
rf_pred = predict(rf_model, data = test_model)
rf_pred_values = rf_pred$predictions

rf_eval = eval_metrics(y_test, rf_pred_values) %>% mutate(Model = "Random Forest")
rf_eval

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl> <dbl> <chr>
## 1  0.966  0.728  0.655 Random Forest
```

Random Forest daje bolje performanse od linearne regresije, po svim metrikama, jer je model uspeo da bolje uhvati odnose između prediktora i ciljne promenljive, što nam ukazuje na to da postoje nelinearni odnosi u podacima.

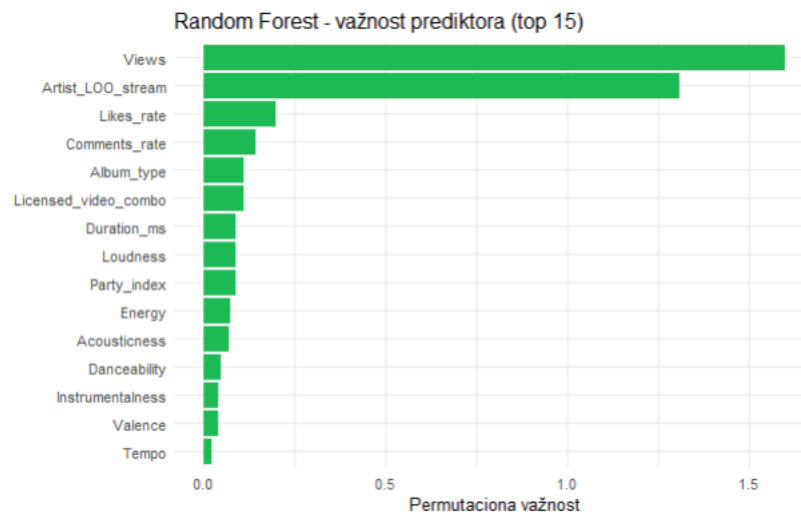
```
rf_imp = ranger::importance(rf_model) %>%
  as.data.frame() %>%
  rownames_to_column("Feature")

names(rf_imp)[2] = "Importance"

rf_imp = rf_imp %>%
  arrange(desc(Importance)) %>%
  slice_head(n = 15)

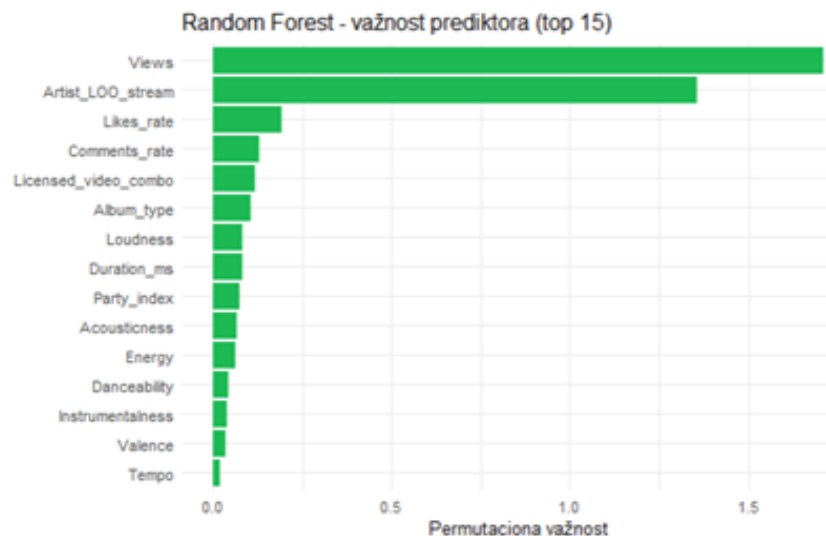
ggplot(rf_imp, aes(x = reorder(Feature, Importance), y = Importance)) +
  geom_col(fill = "#1DB954") +
  coord_flip() +
  theme_minimal() +
```

```
labs(title = "Random Forest - važnost prediktora (top 15)", x = "", y = "Permutaciona važnost")
```



Random Forest pokazuje ono što se moglo i zaključiti na osnovu bivarijantne analize, a to je da su Views i Artist_LOO_Stream najbolji prediktori, dok je ovaj model uzeo u obzir i neke prediktore za koje je koeficijent korelacije bio daleko manji, kao što su Album_type ili Loudness.

Na sledećem dijagramu nalaze se najvažniji prediktori, **pre nego što smo dodali Is_live, Is_spoken, Is_collab, Is_instrumental i Mood_quadrant.**



Nakon izmena u modelu (vraćanja određenih kolona), nisu se napravile nikakve promene u odnosu na izbor najboljih prediktora za Stream, tj. u najbitnije prediktore nisu uključene kolone koje su pre bile obrisane. Kolone koje su pre bile izabrane: Views,

Artist_LOO_Stream, Likes_rate, Comments_rate, Licensed_video_combo, Album_type, Loudness, Duration_ms, Party_index, Acousticness, Energy, Danceability, Instrumentalness, Valence, Tempo.

XGBoost

XGBoost (eXtreme Gradient Boosting) je optimizovani algoritam gradijentnog boostinga koji kombinuje više slabijih modela u jedan jači i efikasan model. Koristi stabla odlučivanja kao osnovne modele, gradeći ih sekvencijalno, tako da svako novo stablo pokušava da ispravi greške prethodnog stabla (boosting).

Model zavisi od broja iteracija u kojima će se izvršiti, kao i od hiperparametara poput maksimalne dubine stabla (`max_depth`) i stope učenja (`eta`), koji se ne mogu odrediti automatski, pa ćemo sprovesti unakrsnu validaciju da bismo pronašli ove parametre.

```
dtrain = xgb.DMatrix(data = X_train, label = y_train)

grid_xgb = expand.grid(
  max_depth = c(4, 6),
  eta = c(0.05, 0.1)
)

xgb_results = vector("list", nrow(grid_xgb))

for (i in seq_len(nrow(grid_xgb))) {
  params_i = list(
    objective = "reg:squarederror",
    max_depth = grid_xgb$max_depth[i],
    eta = grid_xgb$eta[i],
    subsample = 0.8,
    colsample_bytree = 0.8
  )

  set.seed(123)
  cv_i = xgb.cv(
    params = params_i,
    data = dtrain,
    nrounds = 500,
    nfold = 5,
    early_stopping_rounds = 20,
    metrics = "rmse",
    verbose = 0
  )

  best_row =
  cv_i$evaluation_log[which.min(cv_i$evaluation_log$test_rmse_mean), ]

  xgb_results[[i]] = tibble(
    max_depth = grid_xgb$max_depth[i],
```

```

    eta = grid_xgb$eta[i],
    best_iter = best_row$iter,
    best_score = best_row$test_rmse_mean
  )
}

xgb_results_df = bind_rows(xgb_results) %>% arrange(best_score)
xgb_results_df

## # A tibble: 4 × 4
##   max_depth  eta best_iter best_score
##   <dbl> <dbl>   <int>   <dbl>
## 1       6 0.05     471     0.914
## 2       6 0.1      189     0.923
## 3       4 0.1      415     0.930
## 4       4 0.05     495     0.931

```

Nakon što smo odredili najbolje vrednosti koeficijenata, fitovaćemo model nad trening skupom.

```

best_xgb = xgb_results_df %>% slice(1)

params = list(
  objective = "reg:squarederror",
  max_depth = best_xgb$max_depth,
  eta = best_xgb$eta,
  subsample = 0.8,
  colsample_bytree = 0.8
)

set.seed(123)
xgb_model = xgb.train(
  params = params,
  data = dtrain,
  nrounds = best_xgb$best_iter,
  verbose = 1
)

dtest = xgb.DMatrix(data = X_test)
xgb_pred = predict(xgb_model, dtest)

xgb_eval = eval_metrics(y_test, xgb_pred) %>% mutate(Model = "XGBoost")
xgb_eval

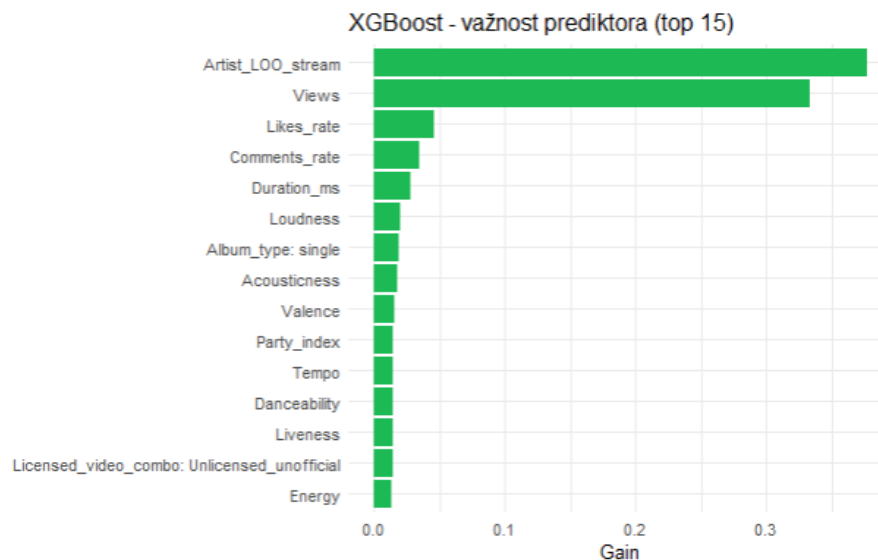
## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl> <dbl> <dbl> <chr>
## 1  0.961  0.724  0.659 XGBoost

```

XGBoost postiže malo slabije rezultate od Random Forest-a.

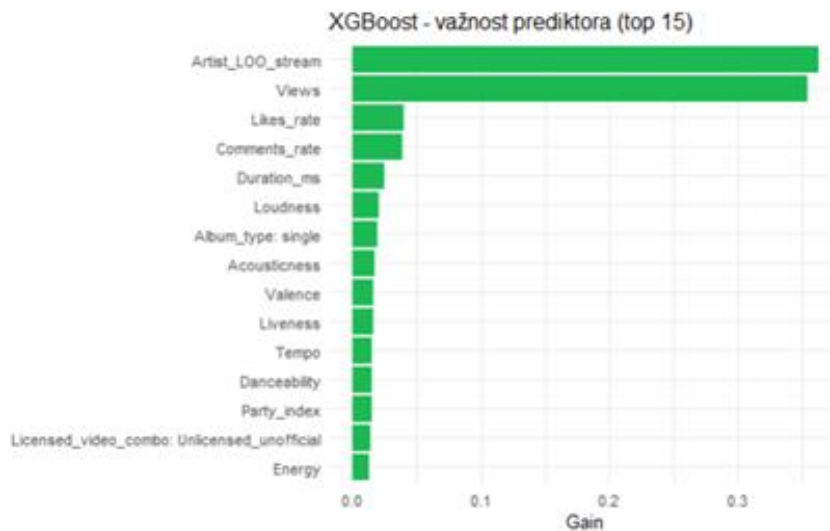
```
xgb_imp = xgb.importance(feature_names = colnames(X_train), model =
xgb_model) %>%
  as_tibble() %>%
  slice_head(n = 15) %>%
  mutate(Feature = str_replace(Feature, "^(Album_type|Licensed_video_combo)",
"\1: "))

ggplot(xgb_imp, aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col(fill = "#1DB954") +
  coord_flip() +
  theme_minimal() +
  labs(title = "XGBoost - važnost prediktora (top 15)", x = "", y = "Gain")
```



Analiza važnosti prediktora kod XGBoost-a pokazuje obrazac uočen i kod Random Forest-a, odnosno da su Views i Artist_LOO_stream ubedljivo najuticajniji prediktori, dok ostale varijable doprinose mnogo manje.

Na sledećem grafiku nalazi se analiza važnosti pre nego što smo vratili binarne kolone.



Nakon izmena u modelu (vraćanja određenih kolona), nisu se napravile nikakve promene u odnosu na izbor najboljih prediktora za Stream, tj. u najbitnije prediktore nisu uključene kolone koje su pre bile obrisane. Kolone koje su pre bile izabrane: Artist_LOO_Stream, Views, Likes_rate, Comments_rate, Duration_ms, Loudness, Album_type, Acousticness, Valence, Liveness, Tempo, Danceability, Party_index, Licensed_video_combo: Unlicensed_unofficial, Energy.

LightGBM

LightGBM je ensemble learning framework koji koristi metod gradijentnog boostinga, pri čemu se jak model gradi sekvencijalnim dodavanjem slabijih modela metodom zasnovanom na gradijentnom spustu. Dizajniran je tako da bude efikasan i veoma precizan, posebno kada se radi sa velikim skupovima podataka. Koristi stabla odlučivanja koja se efikasno grade uz smanjenje potrošnje memorije i optimizaciju vremena treniranja.

Kao i kod XGBoost-a, moramo da odaberemo optimalnu stopu učenja (`learning_rate`) i kompleksnost stabala (`num_leaves`), pa sprovodimo unakrsnu validaciju da bismo pronašli ove parametre.

```
dtrain_lgb = lgb.Dataset(data = X_train, label = y_train)

grid_lgb = expand.grid(
  learning_rate = c(0.05, 0.1),
  num_leaves = c(31, 63)
)

lgb_results = vector("list", nrow(grid_lgb))

for (i in seq_len(nrow(grid_lgb))) {
  params_i = list(
    objective = "regression",
```

```

    metric = "rmse",
    learning_rate = grid_lgb$learning_rate[i],
    num_leaves = grid_lgb$num_leaves[i],
    feature_fraction = 0.8,
    bagging_fraction = 0.8,
    bagging_freq = 1
  )

  set.seed(123)
  cv_i = lgb.cv(
    params = params_i,
    data = dtrain_lgb,
    nrounds = 2000,
    nfold = 5,
    early_stopping_rounds = 50,
    verbose = -1
  )

  lgb_results[[i]] = tibble(
    learning_rate = grid_lgb$learning_rate[i],
    num_leaves = grid_lgb$num_leaves[i],
    best_iter = cv_i$best_iter,
    best_score = cv_i$best_score
  )
}

lgb_results_df = bind_rows(lgb_results) %>% arrange(best_score)
lgb_results_df

## # A tibble: 4 × 4
##   learning_rate num_leaves best_iter best_score
##   <dbl>         <dbl>    <int>    <dbl>
## 1      0.05          63      450      0.912
## 2      0.05          31      759      0.916
## 3      0.1           63      155      0.923
## 4      0.1           31      416      0.924

best_lgb = lgb_results_df %>% slice(1)

lgb_params = list(
  objective = "regression",
  metric = "rmse",
  learning_rate = best_lgb$learning_rate,
  num_leaves = best_lgb$num_leaves,
  feature_fraction = 0.8,
  bagging_fraction = 0.8,
  bagging_freq = 1
)

set.seed(123)

```



```

lgb_model = lgb.train(
  params = lgb_params,
  data = dtrain_lgb,
  nrounds = best_lgb$best_iter,
  verbose = 1
)

## [LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.001732 seconds.
## You can set `force_row_wise=true` to remove the overhead.
## And if memory is not enough, you can set `force_col_wise=true`.
## [LightGBM] [Info] Total Bins 3578
## [LightGBM] [Info] Number of data points in the train set: 16194, number of
used features: 18
## [LightGBM] [Info] Start training from score 17.655270

lgb_pred = predict(lgb_model, X_test)

lgb_eval = eval_metrics(y_test, lgb_pred) %>% mutate(Model = "LightGBM")
lgb_eval

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>    <dbl> <dbl> <chr>
## 1    0.963    0.723  0.659 LightGBM

```

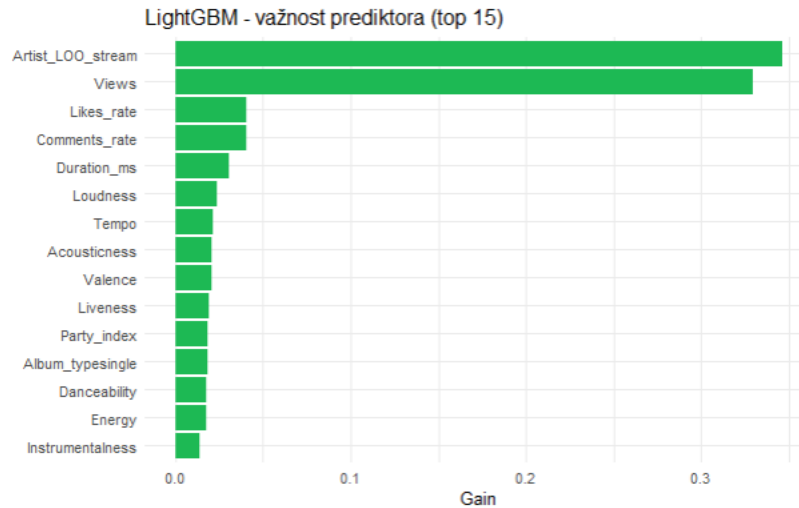
LightGBM, nakon pretraživanja hiperparametara (learning_rate, num_leaves) uz k-fold unakrsnu validaciju, postiže sličan rezultat kao XGBoost i Random Forest.

```

lgb_imp = lgb.importance(lgb_model) %>%
  as_tibble() %>%
  slice_head(n = 15)

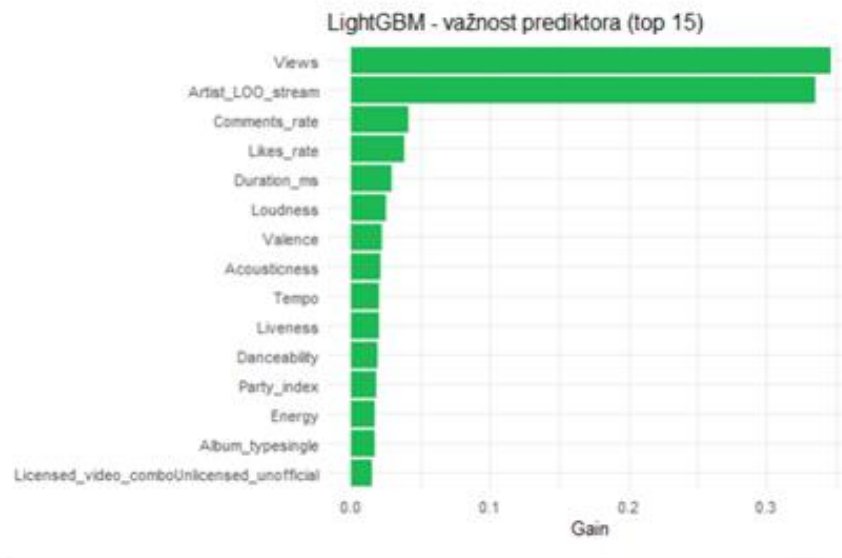
ggplot(lgb_imp, aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col(fill = "#1DB954") +
  coord_flip() +
  theme_minimal() +
  labs(title = "LightGBM - važnost prediktora (top 15)", x = "", y = "Gain")

```



Analiza važnosti prediktora pokazuje sličan obrazac kao kod prethodna dva modela, odnosno da Views i Artist_LOO_stream ubedljivo dominiraju, dok ostali prediktori doprinose znatno manje.

Na sledećeg grafiku nalazi se važnost prediktora pre izbacivanja binarnih kolona i Mood_quadrant.



Nakon izmena u modelu (vraćanja određenih kolona), nisu se napravile nikakve promene u odnosu na izbor najboljih prediktora za Stream, tj. u najbitnije prediktore nisu uključene kolone koje su pre bile obrisane. Kolone koje su pre bile izabrane: Views, Artist_LOO_Stream, Comments_rate, Likes_rate, Duration_ms, Loudness, Valence, Acousticness, Tempo, Liveness, Danceability, Party_index, Energy, Album_typesingle, Licensed_video_comboUnlicensed_unofficial.

Dodatna analiza: uticaj uključivanja pesama bez YouTube spota (Has_youtube = FALSE)

Kreirani modeli trenirani su nad pesmama koje imaju YouTube spot, na isti način kao i linearni modeli, kako bi njihovo međusobno poređenje bilo fer. Međutim, za razliku od linearne regresije, stabla odlučivanja mogu direktno da koriste i pesme bez YouTube spota, uz Has_youtube indikator koji im omogućava da izoluju te redove. Zbog toga ćemo u ovom delu proveriti da li će dodatak dodatnih pesama i indikatorske kolone doneti neko poboljšanje u odnosu na prethodne modele.

Ponovo ćemo definisati formulu, koja će uključivati i Has_youtube kao dodatni prediktor, i na isti način ćemo pripremiti podatke za modele kao u prethodnim delovima. Nakon pripreme podataka, ponovićemo proces treniranja i kreiranja modela.

```
formula_full = as.formula(log1p(Stream) ~ Album_type + Loudness + Acousticness +  
Instrumentalness +  
Views + Artist_LOO_stream + Likes_rate + Comments_rate + Energy + Duration_ms +  
Tempo + Liveness + Danceability + Valence + Party_index + Licensed_video_combo +  
Has_youtube + Is_live + Is_instrumental + Is_spoken + Is_collab + Mood_quadrant +  
Key_factor)
```

```
n_train_full = nrow(train_ml)
```

```
data_matrica_full = bind_rows(train_ml, test_ml)
```

```
X_full_all = model.matrix(formula_full, data = data_matrica_full)[, -1]
```

```
y_full_all = log1p(data_matrica_full$Stream)
```

```
X_train_full = X_full_all[1:n_train_full, ]
```

```
X_test_full = X_full_all[(n_train_full + 1):nrow(X_full_all), ]
```

```
y_train_full = y_full_all[1:n_train_full]
```

```
y_test_full = y_full_all[(n_train_full + 1):length(y_full_all)]
```

```
dim(X_train_full)
```

```
## [1] 16590    29
```

```
dim(X_test_full)
```

```
## [1] 4126    29
```

Random Forest

```
rf_results_full = vector("list", nrow(grid_rf))
```

```
for (i in seq_len(nrow(grid_rf))) {
```

```
  set.seed(123)
```

```
  m_i = ranger(
```

```

formula_full,
data = train_ml,
num.trees = 500,
mtry = grid_rf$mtry[i],
min.node.size = grid_rf$min.node.size[i],
importance = "none"
)

rf_results_full[[i]] = tibble(
  mtry = grid_rf$mtry[i],
  min.node.size = grid_rf$min.node.size[i],
  oob_rmse = sqrt(m_i$prediction.error)
)
}

## Growing trees.. Progress: 91%. Estimated remaining time: 3 seconds.
## Growing trees.. Progress: 66%. Estimated remaining time: 16 seconds.
## Growing trees.. Progress: 78%. Estimated remaining time: 8 seconds.
## Growing trees.. Progress: 81%. Estimated remaining time: 7 seconds.

rf_results_full_df = bind_rows(rf_results_full) %>% arrange(oob_rmse)
best_rf_full = rf_results_full_df %>% slice(1)
best_rf_full

## # A tibble: 1 × 3
##   mtry min.node.size oob_rmse
##   <dbl>         <dbl>    <dbl>
## 1      7             1    0.915

rf_model_full = ranger(
  formula_full,
  data = train_ml,
  num.trees = 500,
  mtry = best_rf_full$mtry,
  min.node.size = best_rf_full$min.node.size,
  importance = "none",
  seed = 123
)

## Growing trees.. Progress: 68%. Estimated remaining time: 14 seconds.

rf_pred_full = predict(rf_model_full, data = test_ml)$predictions

rf_eval_full = eval_metrics(y_test_full, rf_pred_full) %>% mutate(Model = "Random
Forest")
rf_eval_full

```

```
## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>  <dbl> <chr>
## 1    0.970    0.731  0.651 Random Forest
```

XGBoost

```
dtrain_full = xgb.DMatrix(data = X_train_full, label = y_train_full)

xgb_results_full = vector("list", nrow(grid_xgb))

for (i in seq_len(nrow(grid_xgb))) {
  params_i = list(
    objective = "reg:squarederror",
    max_depth = grid_xgb$max_depth[i],
    eta = grid_xgb$eta[i],
    subsample = 0.8,
    colsample_bytree = 0.8
  )

  set.seed(123)
  cv_i = xgb.cv(
    params = params_i,
    data = dtrain_full,
    nrounds = 500,
    nfold = 5,
    early_stopping_rounds = 20,
    metrics = "rmse",
    verbose = 0
  )

  best_row = cv_i$evaluation_log[which.min(cv_i$evaluation_log$test_rmse_mean), ]

  xgb_results_full[[i]] = tibble(
    max_depth = grid_xgb$max_depth[i],
    eta = grid_xgb$eta[i],
    best_iter = best_row$iter,
    best_score = best_row$test_rmse_mean
  )
}

xgb_results_full_df = bind_rows(xgb_results_full) %>% arrange(best_score)
best_xgb_full = xgb_results_full_df %>% slice(1)

params_full = list(
```

```

objective = "reg:squarederror",
max_depth = best_xgb_full$max_depth,
eta = best_xgb_full$eta,
subsample = 0.8,
colsample_bytree = 0.8
)

set.seed(123)
xgb_model_full = xgb.train(
  params = params_full,
  data = dtrain_full,
  nrounds = best_xgb_full$best_iter,
  verbose = 0
)

dtest_full = xgb.DMatrix(data = X_test_full)
xgb_pred_full = predict(xgb_model_full, dtest_full)

xgb_eval_full = eval_metrics(y_test_full, xgb_pred_full) %>% mutate(Model = "XGBoost")
xgb_eval_full

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl> <dbl> <chr>
## 1    0.970    0.728  0.651 XGBoost

```

LightGBM

```

dtrain_lgb_full = lgb.Dataset(data = X_train_full, label = y_train_full)

lgb_results_full = vector("list", nrow(grid_lgb))

for (i in seq_len(nrow(grid_lgb))) {
  params_i = list(
    objective = "regression",
    metric = "rmse",
    learning_rate = grid_lgb$learning_rate[i],
    num_leaves = grid_lgb$num_leaves[i],
    feature_fraction = 0.8,
    bagging_fraction = 0.8,
    bagging_freq = 1
  )

  set.seed(123)
  cv_i = lgb.cv(
    params = params_i,

```

```

data = dtrain_lgb_full,
nrounds = 2000,
nfold = 5,
early_stopping_rounds = 50,
verbose = -1
)

lgb_results_full[[i]] = tibble(
  learning_rate = grid_lgb$learning_rate[i],
  num_leaves = grid_lgb$num_leaves[i],
  best_iter = cv_i$best_iter,
  best_score = cv_i$best_score
)
}

lgb_results_full_df = bind_rows(lgb_results_full) %>% arrange(best_score)
best_lgb_full = lgb_results_full_df %>% slice(1)

lgb_params_full = list(
  objective = "regression",
  metric = "rmse",
  learning_rate = best_lgb_full$learning_rate,
  num_leaves = best_lgb_full$num_leaves,
  feature_fraction = 0.8,
  bagging_fraction = 0.8,
  bagging_freq = 1
)

set.seed(123)
lgb_model_full = lgb.train(
  params = lgb_params_full,
  data = dtrain_lgb_full,
  nrounds = best_lgb_full$best_iter,
  verbose = -1
)

lgb_pred_full = predict(lgb_model_full, X_test_full)

lgb_eval_full = eval_metrics(y_test_full, lgb_pred_full) %>% mutate(Model = "LightGBM")
lgb_eval_full

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl> <dbl> <chr>
## 1    0.968    0.727  0.653 LightGBM

```

Uticaj bez Youtube signala

Pored modela sa Youtube signalom, analogno modelu A iz multivarijatne analize, ocenićemo i kako modeli reaguju na podatke koji se odnose samo na karakteristike pesme, odnosno bez Views, Likes_rate, Comments_rate i Licensed_video_combo, kako bismo mogli da analiziramo da li se mogu predvideti Stream-ovi pesme pre objavljivanja.

Ponovo ćemo definisati formulu, isključiti navedene prediktore, i na isti način ćemo pripremiti podatke za modele kao u prethodnim delovima. Nakon pripreme podataka, ponovićemo proces treniranja i kreiranja modela.

```
formula_full = as.formula(log1p(Stream) ~ Album_type + Loudness + Acousticness +  
Instrumentalness + Artist_LOO_stream + Energy + Duration_ms +  
Tempo + Liveness + Danceability + Valence + Party_index + Is_live + Is_instrumental +  
Is_spoken + Is_collab + Mood_quadrant + Key_factor)
```

```
n_train_full = nrow(train_ml)
```

```
data_matrica_full = bind_rows(train_ml, test_ml)
```

```
X_full_all = model.matrix(formula_full, data = data_matrica_full)[, -1]
```

```
y_full_all = log1p(data_matrica_full$Stream)
```

```
X_train_full = X_full_all[1:n_train_full, ]
```

```
X_test_full = X_full_all[(n_train_full + 1):nrow(X_full_all), ]
```

```
y_train_full = y_full_all[1:n_train_full]
```

```
y_test_full = y_full_all[(n_train_full + 1):length(y_full_all)]
```

```
dim(X_train_full)
```

```
## [1] 16590    21
```

```
dim(X_test_full)
```

```
## [1] 4126     21
```

Random Forest

```
rf_results_full = vector("list", nrow(grid_rf))
```

```
for (i in seq_len(nrow(grid_rf))) {
```

```
  set.seed(123)
```

```
  m_i = ranger(
```

```
    formula_full,
```

```
    data = train_ml,
```

```
    num.trees = 500,
```

```
    mtry = grid_rf$mtry[i],
```

```
    min.node.size = grid_rf$min.node.size[i],
```



```

    importance = "none"
  )

  rf_results_full[[i]] = tibble(
    mtry = grid_rf$mtry[i],
    min.node.size = grid_rf$min.node.size[i],
    oob_rmse = sqrt(m_i$prediction.error)
  )
}

## Growing trees.. Progress: 87%. Estimated remaining time: 4 seconds.
## Growing trees.. Progress: 99%. Estimated remaining time: 0 seconds.

rf_results_full_df = bind_rows(rf_results_full) %>% arrange(oob_rmse)
best_rf_full = rf_results_full_df %>% slice(1)
best_rf_full

## # A tibble: 1 × 3
##   mtry min.node.size oob_rmse
##   <dbl>         <dbl>    <dbl>
## 1     7             1     1.15

rf_model_full = ranger(
  formula_full,
  data = train_ml,
  num.trees = 500,
  mtry = best_rf_full$mtry,
  min.node.size = best_rf_full$min.node.size,
  importance = "none",
  seed = 123
)

## Growing trees.. Progress: 93%. Estimated remaining time: 2 seconds.

rf_pred_full = predict(rf_model_full, data = test_ml)$predictions

rf_eval_noyt = eval_metrics(y_test_full, rf_pred_full) %>% mutate(Model = "Random
Forest")
rf_eval_noyt

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>    <dbl>  <dbl> <chr>
## 1    1.24    0.935  0.433 Random Forest

```

XGBoost

```
dtrain_full = xgb.DMatrix(data = X_train_full, label = y_train_full)

xgb_results_full = vector("list", nrow(grid_xgb))

for (i in seq_len(nrow(grid_xgb))) {
  params_i = list(
    objective = "reg:squarederror",
    max_depth = grid_xgb$max_depth[i],
    eta = grid_xgb$eta[i],
    subsample = 0.8,
    colsample_bytree = 0.8
  )

  set.seed(123)
  cv_i = xgb.cv(
    params = params_i,
    data = dtrain_full,
    nrounds = 500,
    nfold = 5,
    early_stopping_rounds = 20,
    metrics = "rmse",
    verbose = 0
  )

  best_row = cv_i$evaluation_log[which.min(cv_i$evaluation_log$test_rmse_mean), ]

  xgb_results_full[[i]] = tibble(
    max_depth = grid_xgb$max_depth[i],
    eta = grid_xgb$eta[i],
    best_iter = best_row$iter,
    best_score = best_row$test_rmse_mean
  )
}

xgb_results_full_df = bind_rows(xgb_results_full) %>% arrange(best_score)
best_xgb_full = xgb_results_full_df %>% slice(1)

params_full = list(
  objective = "reg:squarederror",
  max_depth = best_xgb_full$max_depth,
  eta = best_xgb_full$eta,
  subsample = 0.8,
  colsample_bytree = 0.8
)
```

```
)

set.seed(123)
xgb_model_full = xgb.train(
  params = params_full,
  data = dtrain_full,
  nrounds = best_xgb_full$best_iter,
  verbose = 0
)

dtest_full = xgb.DMatrix(data = X_test_full)
xgb_pred_full = predict(xgb_model_full, dtest_full)

xgb_eval_noyt = eval_metrics(y_test_full, xgb_pred_full) %>% mutate(Model = "XGBoost")
xgb_eval_noyt

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>    <dbl>  <dbl> <chr>
## 1     1.24     0.939   0.426 XGBoost
```

LightGBM

```
dtrain_lgb_full = lgb.Dataset(data = X_train_full, label = y_train_full)

lgb_results_full = vector("list", nrow(grid_lgb))

for (i in seq_len(nrow(grid_lgb))) {
  params_i = list(
    objective = "regression",
    metric = "rmse",
    learning_rate = grid_lgb$learning_rate[i],
    num_leaves = grid_lgb$num_leaves[i],
    feature_fraction = 0.8,
    bagging_fraction = 0.8,
    bagging_freq = 1
  )

  set.seed(123)
  cv_i = lgb.cv(
    params = params_i,
    data = dtrain_lgb_full,
    nrounds = 2000,
    nfold = 5,
    early_stopping_rounds = 50,
    verbose = -1
  )
}
```

```

)

lgb_results_full[[i]] = tibble(
  learning_rate = grid_lgb$learning_rate[i],
  num_leaves = grid_lgb$num_leaves[i],
  best_iter = cv_i$best_iter,
  best_score = cv_i$best_score
)
}

lgb_results_full_df = bind_rows(lgb_results_full) %>% arrange(best_score)
best_lgb_full = lgb_results_full_df %>% slice(1)

lgb_params_full = list(
  objective = "regression",
  metric = "rmse",
  learning_rate = best_lgb_full$learning_rate,
  num_leaves = best_lgb_full$num_leaves,
  feature_fraction = 0.8,
  bagging_fraction = 0.8,
  bagging_freq = 1
)

set.seed(123)
lgb_model_full = lgb.train(
  params = lgb_params_full,
  data = dtrain_lgb_full,
  nrounds = best_lgb_full$best_iter,
  verbose = -1
)

lgb_pred_full = predict(lgb_model_full, X_test_full)

lgb_eval_noyt = eval_metrics(y_test_full, lgb_pred_full) %>% mutate(Model = "LightGBM")
lgb_eval_noyt

## # A tibble: 1 × 4
##   RMSE_log MAE_log R2_log Model
##   <dbl>   <dbl>   <dbl> <chr>
## 1     1.25     0.951   0.416 LightGBM

```

Poređenje nelinearnih modela

```

rezultati_sve = bind_rows(
  rf_eval %>% mutate(Varijanta = "Samo pesme sa Youtube spotom"),

```

```

rf_eval_full %>% mutate(Varijanta = "Pun skup (i pesme bez Youtube spota)",
rf_eval_noyt %>% mutate(Varijanta = "Bez YouTube signala"),

xgb_eval %>% mutate(Varijanta = "Samo pesme sa Youtube spotom"),
xgb_eval_full %>% mutate(Varijanta = "Pun skup (i pesme bez Youtube spota)",
xgb_eval_noyt %>% mutate(Varijanta = "Bez YouTube signala"),

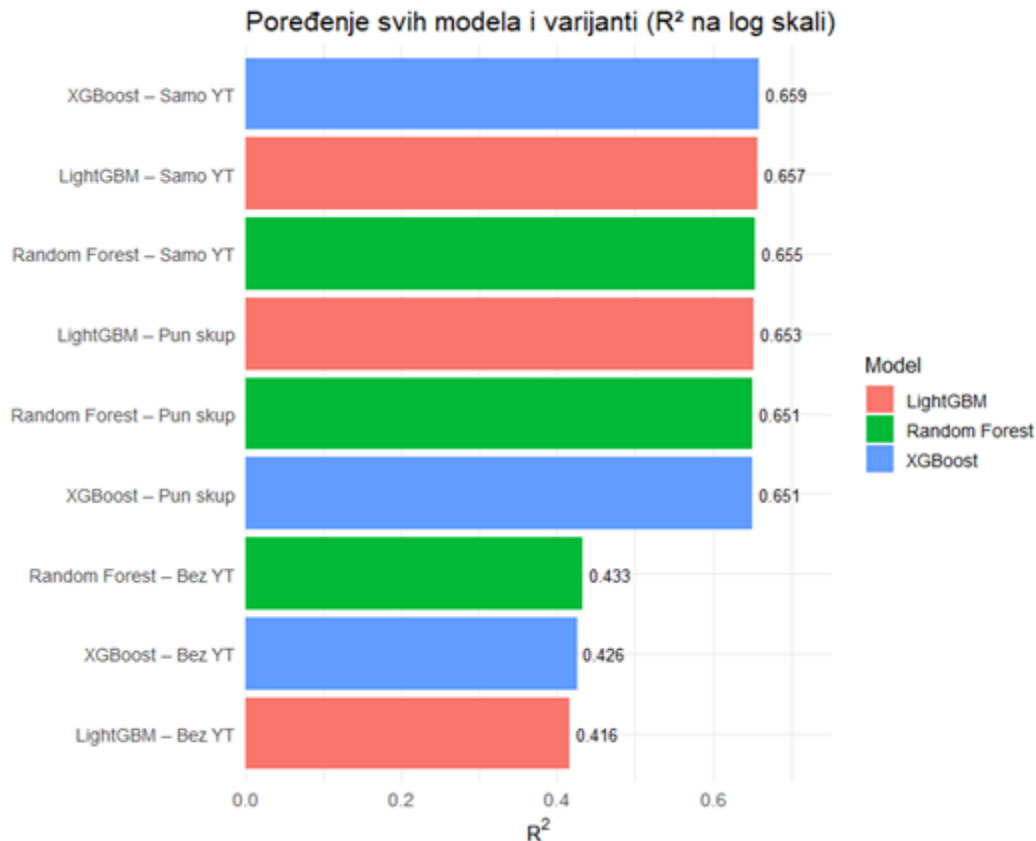
lgb_eval %>% mutate(Varijanta = "Samo pesme sa Youtube spotom"),
lgb_eval_full %>% mutate(Varijanta = "Pun skup (i pesme bez Youtube spota)",
lgb_eval_noyt %>% mutate(Varijanta = "Bez YouTube signala")
) %>%
select(Model, Varijanta, RMSE_log, MAE_log, R2_log) %>%
mutate(Varijanta = factor(Varijanta, levels = c("Samo pesme sa Youtube spotom", "Pun
skup (i pesme bez Youtube spota)", "Bez YouTube signala"))) %>%
arrange(Model, Varijanta)

rezultati_sve

## # A tibble: 9 × 5
##   Model   Varijanta                RMSE_log MAE_log R2_log
##   <chr>   <fct>                <dbl>   <dbl> <dbl>
## 1 LightGBM Samo pesme sa Youtube spotom         0.963   0.723  0.657
## 2 LightGBM Pun skup (i pesme bez Youtube spota)  0.968   0.727  0.653
## 3 LightGBM Bez YouTube signala                1.25    0.951  0.416
## 4 Random Forest Samo pesme sa Youtube spotom         0.966   0.728  0.655
## 5 Random Forest Pun skup (i pesme bez Youtube spota) 0.970   0.731  0.661
## 6 Random Forest Bez YouTube signala                1.24    0.935  0.433
## 7 XGBoost  Samo pesme sa Youtube spotom         0.961   0.724  0.659
## 8 XGBoost  Pun skup (i pesme bez Youtube spota)  0.970   0.728  0.651
## 9 XGBoost  Bez YouTube signala                1.24    0.939  0.426

rezultati_sve %>%
  mutate(Varijanta = dplyr::recode(Varijanta, "Samo pesme sa Youtube spotom" = "Samo
YT", "Pun skup (i pesme bez Youtube spota)" = "Pun skup", "Bez YouTube signala" = "Bez
YT"),
  oznaka = paste(Model, Varijanta, sep = " – ")) %>%
  ggplot(aes(x = reorder(oznaka, R2_log), y = R2_log, fill = Model)) +
  geom_col() +
  geom_text(aes(label = round(R2_log, 3)), hjust = -0.15, size = 3.5) +
  coord_flip(clip = "off") +
  scale_y_continuous(limits = c(0, max(rezultati_sve$R2_log) * 1.12),
    expand = expansion(mult = c(0, 0.02))) +
  theme_minimal(base_size = 13) +
  labs(title = "Poređenje svih modela i varijanti (R2 na log skali)",
    x = "", y = expression(R2)) +
  theme(plot.margin = ggplot2::margin(t = 10, r = 25, b = 10, l = 10))

```



Rezultati pokazuju da razlika između varijante sa i bez Youtube videa praktično ne postoji, kod sva tri modela razlike u metrikama su na nivou treće decimale. Ovo potvrđuje da modeli zasnovani na stablima uspešno koriste Has_youtube indikator da izoluju pesme kojima je Views veštački postavljen na 0, tako da uključivanje ovih redova ne narušava predikciju za pesme koje imaju YouTube podatke. Kako širi skup podataka ne dovodi do pogoršanja performansi, a omogućava modelu da predviđa Stream i za pesme bez YouTube spota, razumljivo bi bilo koristiti i ove podatke u okviru nelinearnih modela.

Za pesme za koje smo izbacili podatke o Youtube spotu, je pokazan znatno manji R^2 , pun skup (i pesme bez Youtube spota) - oko 0.66, dok modeli bez Youtube signala pokazuju oko 0.42, što potvrđuje naš zaključak iz multivarijatne analize - signal popularnosti sa Youtube-a u velikoj meri utiče na popularnost na Spotify-u.

Poređenje modela

Nakon kreiranja svih modela, trebalo bi i da ih uporedimo uz pomoć funkcije za evaluaciju, kako bismo adekvatno mogli da uporedimo modele i da donesemo potrebne zaključke. Prilikom poređenja poredimo predikacije modela nad istim skupovima, gde su uključene samo opservacije koje imaju spot na Youtube-u.

```
rezultati_modela = bind_rows(lm_eval, lasso_eval, ridge_eval, rf_eval, xgb_eval, lgb_eval)
%>%
```

```
select(Model, RMSE_log, MAE_log, R2_log) %>%
  arrange(RMSE_log)
```

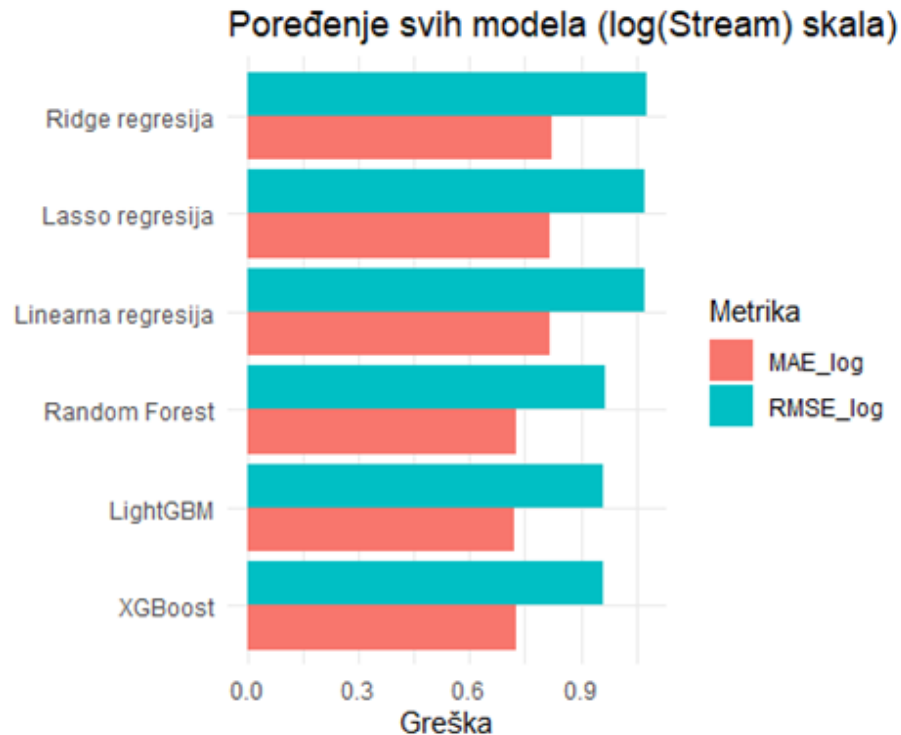
rezultati_modela

```
## # A tibble: 6 x 4
##   Model          RMSE_log MAE_log R2_log
##   <chr>          <dbl>   <dbl> <dbl>
## 1 XGBoost        0.961   0.724 0.659
## 2 LightGBM       0.963   0.723 0.657
## 3 Random Forest  0.966   0.728 0.655
## 4 Linearna regresija 1.07   0.817 0.574
## 5 Lasso regresija 1.07   0.818 0.573
## 6 Ridge regresija 1.08   0.823 0.572
```

rezultati_modela %>%

```
  pivot_longer(cols = c(RMSE_log, MAE_log), names_to = "Metrika", values_to = "Vrednost")
  %>%
```

```
  ggplot(aes(x = reorder(Model, Vrednost), y = Vrednost, fill = Metrika)) +
  geom_col(position = "dodge") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Poređenje svih modela (log(Stream) skala)", x = "", y = "Greška")
```



Nelinearni modeli nadmašuju sva tri linearna modela, pri čemu R^2 se kreće oko 0.65, dakle napravljen je rast od približno 0.1 u odnosu na linearne modele. RMSE i MAE su se smanjili

za oko 20%. Ovako izražena razlika ukazuje da odnos između prediktora i Stream-a nije čisto linearan, već sadrži interakcije i nelinearne obrasce koje stabla prirodno hvataju, a linearna regresija ne može da uhvati.

Uprkos razlici u tačnosti predviđanja, oba tipa modela su odabrala iste prediktore kao najvažnije, odnosno Views i Artist_LOO_stream dominiraju i kod linearne regresije i kod svih modela zasnovanih na stablima. Ova nam zapravo odgovara na pitanje koje smo zapravo imali na početku analize, odnosno šta najviše utiče na samu popularnost pesme. Zaključak je da su istorijska popularnost izvođača i vidljivost na YouTube-u ubedljivo najjači signali za predviđanje broja stream-ova, dok audio karakteristike pesme, čak i kada i u nelinearnim modelima, doprinose daleko manje.

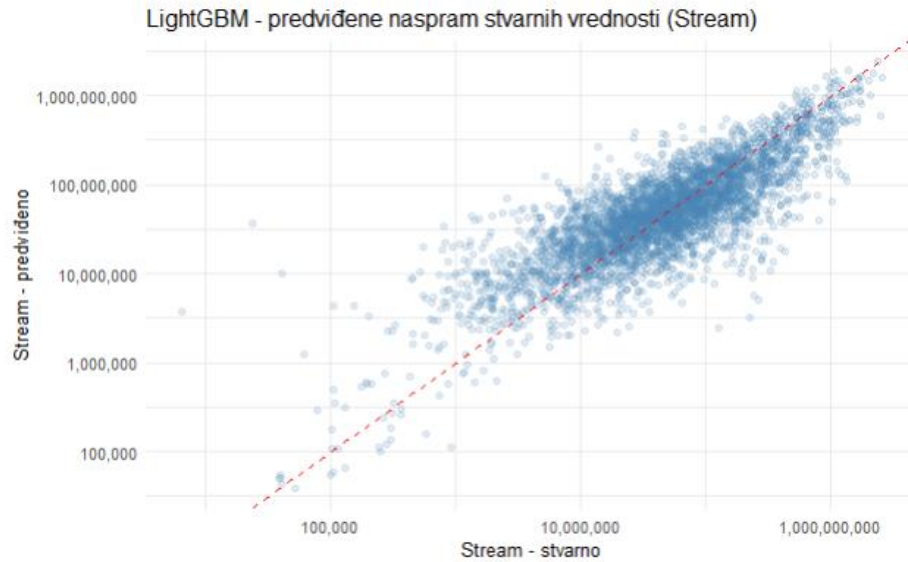
Svakako, izbor između ova dva pristupa zavisi od cilja analize:

- Ukoliko je prioritet **interpretabilnost**, tj. razumevanje zašto neka pesma ima više ili manje stream-ova, uz jasne, direktno tumačive koeficijente — linearna regresija (nakon stepwise selekcije) ostaje solidan izbor, uprkos nižoj tačnosti.
- Ako je prioritet **tačnost predikcije**, modeli zasnovani na stablima, predstavljaju znatno bolji izbor, po ceni gubitka direktne interpretabilnosti pojedinačnih koeficijenata i veće računske zahtevnosti prilikom podešavanja hiperparametara.

Predviđene naspram stvarnih vrednosti (LightGBM)

Pošto je LightGBM najbolji model, korisno je i vizuelno proveriti gde tačno greši, umesto da se oslonimo samo na metrike, pa ćemo uporediti predviđene i stvarne vrednosti na test skupu, na log skali.

```
tibble(actual = expm1(y_test), predicted = expm1(lgb_pred)) %>%  
  ggplot(aes(x = actual, y = predicted)) +  
  geom_point(alpha = 0.15, color = "steelblue") +  
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +  
  scale_x_log10(labels = scales::comma) +  
  scale_y_log10(labels = scales::comma) +  
  theme_minimal() +  
  labs(  
    title = "LightGBM - predviđene naspram stvarnih vrednosti (Stream log)",  
    x = "Stream - stvarno", y = "Stream - predviđeno"  
  )
```

Crvena isprekidana linija predstavlja stvarne podatke iz skupa. Za srednji opseg vrednosti Stream-a tačke prate tu liniju relativno dobro, što je u skladu sa umerenim R^2 od oko 0.66. Međutim, za najveće vrednosti stvarnog Stream-a na desnom delu grafika tačke sistematski padaju ispod dijagonale, odnosno model predviđa niže vrednosti od stvarnih, tj. drugim rečima, LightGBM potcenjuje najpopularnije pesme. Ovo potvrđuje zapažanje iz dijagnostike linearnih modela i direktno podržava zaključak da model najviše greši kod ekstremno popularnih pesama, verovatno zato što takve pesme dostižu popularnost usled faktora koji nisu obuhvaćeni ovim skupom podataka.

Zaključak

Ovaj seminarski rad se fokusirao na primeni metoda *Nauke o podacima* i *Mašinskog učenja* kako bismo kroz različite korake pokušali da predvidimo popularnost pesme na nekoj od platformi, pri čemu smo odlučili da predviđamo broj Stream-ova pesme.

Ono što je analiza jasno pokazala jeste da uspeh pesme mnogo manje zavisi od toga kako ona zvuči, a mnogo više od uticaja drugih platformi i od popularnosti izvođača. Većina promenljivih vezana za audio karakteristike pesme se nije pokazala kao dobar prediktor, i pored raznih pokušaja kombinacija. Dakle, komercijalni uspeh pesme, u granicama ovog skupa podataka, je u većoj meri posledica reputacije i vidljivosti nego same muzičke produkcije. I u poređenju nelinearnih modela, sa i bez Youtube signala, pokazalo se da se dobija mnogo bolji rezultat kada se uključi i Youtube signal, jer dodatno objašnjava oko 20% varijanse.

Modeli su ocenjeni na **log skali** i korišćenjem linearnih modela dobili smo najbolji R^2 od oko 0.56, dok su nelinearni modeli postigli R^2 od oko 0.66. Međutim, čak je i LightGBM ostavio oko 30% varijanse Stream-a neobjašnjenim, što govori da predviđanje uspeha jedne pesme, bar uz ove prediktore, ne može u potpunosti da izvrši, posebno jer baš u ovom polju postoje različiti faktori koji se ne mogu uzeti u obzir i koji se ne mogu kvantifikovati ili zabeležiti na neki način, kao što je već pomenuta viralnost na društvenim mrežama, ali i to i koliko se meri promoviše neka pesma (da smo imali podatke o ulaganju u marketing, možda bismo postigle bolje rezultate), kao i na primer da li se pesma koristi u filmovima, igricama i slično.

Dodatno, rad bi se mogao unaprediti kroz nekoliko koraka. Obzirom na to da su svi testirani modeli najviše grešili kod ekstremno popularnih pesama, poseban tretman repa raspodele, na primer kroz kvantilnu regresiju ili odvojen model za tu podgrupu bi mogli dodatno poboljšati preciznost predikcije. Pored toga, ponavljanje evaluacije preko više nasumičnih podela na trening i test skup umesto jedne dalo bi uvid u to koliko su dobijene metrike osetljive na konkretnu podelu podataka.

Literatura

<https://www.ef.uns.ac.rs/predmeti/oas/statistika/2020-01-10-des-deskriptivna-statistika.pd>" (*Ekonomski fakultet u Novom Sadu*, Deskriptivna statistika)

<https://mixanalytic.com/guides/mood-analysis> (*Mix Analytic*, Podaci o mood kvadrantu pesme)

<https://developer.spotify.com/documentation/web-api/reference/get-audio-features> (*Spotify* dokumentacija o pragovima za audio karakteristike)

Materijali sa predmeta Uvod u nauku o podacima, Prirodno-matematički fakultet u Kragujevcu

<https://www.geeksforgeeks.org/> (*Geeks for Geeks*, Podaci o različitim metodama)

<https://www.kaggle.com/> (*Kaggle*, Skup podataka, podaci o različitim pristupima i metodama, informacije sa foruma)